

Algoritmos y Estructuras de Datos II

Ordenación elemental

Contenidos

- 1 Introducción
- 2 Motivación
- 3 Ordenación por selección
 - Idea
 - Ejemplo
 - Algoritmo
 - Comando for
 - Análisis
- 4 Número de operaciones de un programa (función ops)
- 5 Ordenación por inserción
 - Ejemplo
 - Algoritmo
 - Análisis
- 6 Resumen

Algoritmos y Estructuras de Datos

Programación imperativa:

- Algoritmos y Estructuras de Datos I
 - pre- y post- condiciones
 - “qué” hace un algoritmo
- Algoritmos y Estructuras de Datos II
 - “cómo” hace el algoritmo

Ejemplo de “qué” y “cómo” de un algoritmo

Ejemplo:

un algoritmo para contar los ceros de una secuencia finita de enteros.

- ¿**Qué** hace?
devuelve (calcula, computa) el número de ocurrencias del cero en la secuencia dada.
- ¿**Cómo** lo hace? Hay varias posibilidades, por ejemplo:
recorre la secuencia de izquierda a derecha incrementando un contador cada vez que observa un cero.

Análisis de algoritmos

Analizar el “cómo” permite

- predecir el tiempo de ejecución (eficiencia en tiempo)
- predecir el uso de memoria (eficiencia en espacio)
- predecir el uso de otros recursos
- comparar distintos algoritmos para un mismo problema

Problema del pintor

Un pintor tarda una hora y media en pintar una línea recta de 3 metros de largo sobre el suelo. ¿Cuánto tardará en pintar una de 5 metros de largo?

3 metros $\longleftrightarrow$ 90 minutos

1 metro $\longleftrightarrow$ 30 minutos

5 metros $\longleftrightarrow$ 150 minutos

Solución: dos horas y media.

El trabajo de pintar la línea es **proporcional** a su longitud.

Problema del profe de Algoritmos 2

El profe de esta materia tarda media hora en ordenar alfabéticamente 100 exámenes. ¿Cuánto tardará en ordenar 200 exámenes?

Razonamiento similar

100 exámenes $\longleftrightarrow$ 1/2 hora

200 exámenes $\longleftrightarrow$ 1 hora

Solución: una hora.

¿Está bien? ¿Es el trabajo de ordenar exámenes **proporcional** a la cantidad de exámenes a ordenar?

Otros problemas

del pintor

*Un pintor tarda una hora y media en pintar una pared **cuadrada** de 3 metros de lado. ¿Cuánto tardará en pintar una de 5 metros de lado?*

9 metros cuadrados	↔	90 minutos
1 metro cuadrado	↔	10 minutos
25 metros cuadrados	↔	250 minutos

Solución: cuatro horas y 10 minutos.

El trabajo de pintar la pared cuadrada es **proporcional** a su superficie, que es proporcional al cuadrado del lado.

Otros problemas

el del globo esférico

Si lleva cinco horas inflar un globo aerostático esférico de 2 metros de diámetro, ¿cuánto llevará inflar uno de 4 metros de diámetro?

El trabajo de inflar el globo es **proporcional** a su volumen, que es proporcional al cubo del diámetro ($V = \frac{\pi d^3}{6}$).

diámetro = 2	↔	k metros cúbicos	↔	5 horas
diámetro = 4	↔	8k metros cúbicos	↔	40 horas

Solución: cuarenta horas.

Algoritmos de ordenación

Para resolver el problema del profe de esta materia, es necesario

- establecer a qué es proporcional la tarea de ordenar exámenes,
- estudiar/inventar métodos de ordenación,
- asumiremos la existencia de elementos o items a ordenar,
- relacionados por un orden total,
- que deben ordenarse de menor a mayor y
- que no necesariamente son diferentes entre sí.

¿Cómo?

Reflexionemos sobre lo siguiente:

- ¿Qué significa que una secuencia de exámenes, números, palabras, etc. esté ordenada?
- ¿Cómo hacen para controlar si una secuencia de números está ordenada?
 - (a esta pregunta la vamos a continuar en el práctico y en el laboratorio)
- ¿Cómo harían para ordenar de menor a mayor ciertos datos o ciertas cosas físicas que están desordenados/as?
 - números
 - cartas de un juego,
 - palabras,
 - exámenes.

Ordenación por selección

Idea

- Es el algoritmo de ordenación más sencillo (pero no el más rápido),
- **selecciona** el menor de todos, lo intercambia con el elemento que se encuentra en la primera posición.
- **selecciona** el menor de todos **los restantes**, lo intercambia con el que se encuentra en el segundo lugar.
- **selecciona** el menor de todos **los restantes**, lo intercambia con el que se encuentra en el tercer lugar.
- ... (*en cada uno de estos pasos ordena un elemento*) ...
- hasta terminar.

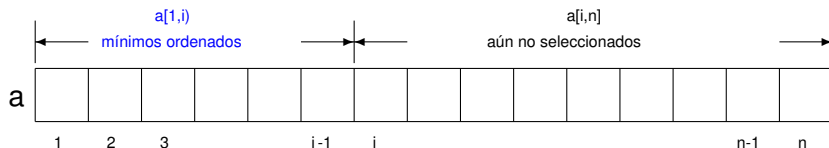
Ordenación por selección

9	3	1	3	5	2	7
9	3	1	3	5	2	7
1	3	9	3	5	2	7
1	3	9	3	5	2	7
1	2	9	3	5	3	7
1	2	9	3	5	3	7
1	2	3	9	5	3	7

1	2	3	9	5	3	7
1	2	3	3	5	9	7
1	2	3	3	5	9	7
1	2	3	3	5	9	7
1	2	3	3	5	9	7
1	2	3	3	5	7	9

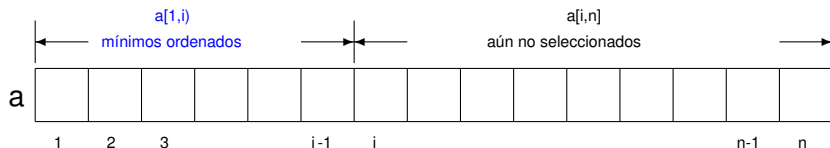
Ordenación por selección

En un arreglo



Ordenación por selección

Invariante



- Invariante:

- el arreglo a es una permutación del original,
- un segmento inicial $a[1,i]$ del arreglo está ordenado, y
- dicho segmento contiene los elementos mínimos del arreglo.

Ordenación por selección

Pseudocódigo

$$\{\text{Pre: } n \geq 0 \wedge a = A\}$$

```
proc selection_sort (in/out a: array[1..n] of T)
```

```
var i, minp: nat
```

 $i := 1$

{Inv: Invariante de recién}

do $i < n \rightarrow \text{minp} := \text{min pos from}(a, i)$

```
swap(a,i,minp)
```

$$i := i + 1$$

od

end proc

{Post: a está ordenado y es permutación de A}

Ordenación por selección

Swap o intercambio

{Pre: $a = A \wedge 1 \leq i, j \leq n$ }

proc swap (**in/out** a: **array**[1..n] **of** T, **in** i, j: **nat**)

var tmp: T

 tmp := a[i]

 a[i] := a[j]

 a[j] := tmp

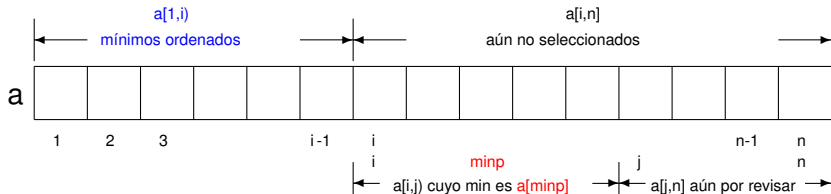
end proc

{Post: $a[i] = A[j] \wedge a[j] = A[i] \wedge \forall k. k \notin \{i, j\} \Rightarrow a[k] = A[k]$ }

¡Garantiza permutación!

Ordenación por selección

Invariante de la función de selecció



- Invariante:

- invariante anterior, y
- el mínimo del segmento $a[i,j]$ está en la posición minp .

Ordenación por selección

Función de selección

```
{Pre:  $0 < i \leq n$ }  
fun min_pos_from (a: array[1..n] of T, i: nat) ret minp: nat  
    var j: nat  
    minp:= i  
    j:= i+1                                {Inv: a[minp] es el mínimo de a[i,j]}  
    do  $j \leq n \rightarrow$  if a[j] < a[minp] then minp:= j fi  
        j:= j+1  
    od  
end fun  
{Post: a[minp] es el mínimo de a[i,n]}
```

Comando for

Fragmentos de la siguiente forma aparecen con frecuencia:

```
k:= n
do k ≤ m → C
    k:= k+1
od
```

Por simplicidad, lo reemplazaremos por

```
for k:= n to m do C od
```

siempre que k no se modifique en C.

Además, asumiremos que el **for** declara la variable k, cuya vida dura sólo durante la ejecución del ciclo.

Comando for

Reemplazo en min_pos_from

```
fun min_pos_from (a: array[1..n] of T, i: nat) ret minp: nat  
  var j: nat  
  minp:= i  
  j:= i+1  
  do  $j \leq n$  → if a[j] < a[minp] then minp:= j fi  
    j:= j+1  
  od  
end fun  
fun min_pos_from (a: array[1..n] of T, i: nat) ret minp: nat  
  minp:= i  
  for j:= i+1 to n do if a[j] < a[minp] then minp:= j fi  
  od  
end fun
```

Comando for

Reemplazo en selection_sort

```
proc selection_sort (in/out a: array[1..n] of T)
  var i,minp: nat
  i:= 1
  do i < n  $\rightarrow$  minp:= min_pos_from(a,i)
                swap(a,i,minp)
                i:= i+1
  od
end proc
```

Comando for

En selection_sort

```
proc selection_sort (in/out a: array[1..n] of T)
```

```
  var minp: nat
```

```
  for i:= 1 to n-1 do
```

```
    minp:= min_pos_from(a,i)
```

```
    swap(a,i,minp)
```

```
  od
```

```
end proc
```

```
fun min_pos_from (a: array[1..n] of T, i: nat) ret minp: nat
```

```
  minp:= i
```

```
  for j:= i+1 to n do if a[j] < a[minp] then minp:= j fi
```

```
  od
```

```
end fun
```

Comando **for ... downto**

Fragmentos de la siguiente forma también aparecen con cierta frecuencia:

```
k:= m
do k  $\geq$  n  $\rightarrow$  C
    k:= k-1
od
```

Por simplicidad, lo reemplazaremos por

```
for k:= m downto n do C od
```

siempre que k no se modifique en C.

Problema del profe

Cuando el algoritmo es la ordenación por selección

- ¿Cómo se respondería el problema del profe si el algoritmo utilizado por él fuera el de ordenación por selección?
- ¿Cuánto más trabajo resulta ordenar 200 exámenes que 100 con este algoritmo?
- ¿Cuánto trabajo es ordenar 200 exámenes (con este algoritmo)?
- ¿Cuánto trabajo es ordenar 100 exámenes (con este algoritmo)?
- ¿Cuánto trabajo es ordenar n exámenes (con este algoritmo)?

Problema del profe

Análisis

- Para contestar estas preguntas habría que **analizar** el algoritmo de ordenación por selección, es decir, contar cuántas operaciones elementales realiza.
- Cuántas sumas, asignaciones, llamadas a funciones, comparaciones, intercambios, etc.
- En vez de eso, se elige una operación **representativa**.
- ¿Qué es una operación **representativa**?
- Una tal que se repite más que o tanto como cualquier otra.
- Hay que buscar la que **más se repite**.

Analizando el procedimiento selection_sort

- selection_sort **contiene un ciclo**,
- **allí** debe estar la operación que más se repite,
- encontramos **una llamada** a la función min_pos_from y **una llamada** al procedimiento swap,
- el procedimiento swap **es constante** (siempre realiza 3 asignaciones elementales),
- la función min_pos_from, en cambio, **tiene un ciclo**,
- nuevamente **allí** debe estar la operación que más se repite,
- encontramos **una comparación** entre elementos de a, y **una asignación** (condicionada al resultado de la comparación).

Analizando ordenación por selección

Conclusión

- La **operación** que más se repite es la **comparación** entre elementos de a ,
- **toda otra operación se repite a lo sumo de manera proporcional** a esa,
- por lo tanto, la **comparación** entre elementos de a es **representativa** del trabajo de la ordenación por selección.
- Esto es habitual: para medir la eficiencia de los algoritmos de ordenación es habitual considerar el número de comparaciones entre elementos del arreglo.
- Veremos luego que acceder (o modificar) una celda de un arreglo es **constante**: su costo no depende de cuál es la celda, ni de la longitud del arreglo.

¿Cuántas comparaciones realiza la ordenación por selección?

- Al llamarse a `min_pos_from(a,i)` se realizan $n-i$ comparaciones.
- `selection_sort` llama a `min_pos_from(a,i)` para $i \in \{1, 2, \dots, n-1\}$.
- por lo tanto, en total son $(n-1) + (n-2) + \dots + (n-(n-1))$ comparaciones.
- es decir, $(n-1) + (n-2) + \dots + 1 = \frac{n*(n-1)}{2}$ comparaciones.

Resolviendo el problema del profe

Con la fórmula obtenida

Para un arreglo de tamaño n , son $\frac{n*(n-1)}{2}$ comparaciones.

100 exámenes	↔	4950 comparaciones	↔	1/2 hora
200 exámenes	↔	19900 comparaciones	↔	2 horas

Solución: 2 horas.

Resolviendo el problema del profe

Con una fórmula simplificada

Como $\frac{n*(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$, el número de comparaciones es proporcional a n^2 .

100 exámenes	↔	10000 comparaciones	↔	1/2 hora
200 exámenes	↔	40000 comparaciones	↔	2 horas

Solución: 2 horas.

Conviene utilizar la expresión n^2 para contestar la pregunta; es más sencillo y da el mismo resultado.

Número de operaciones de un programa

- Una vez que uno sabe qué **operación** quiere contar, debe imaginar una ejecución arbitraria, genérica del programa intentando contar el número de veces que esa ejecución arbitraria realizará **dicha operación**.
- Ése es el verdadero método para contar.
- Es imprescindible comprender **cómo** se ejecuta el programa.
- A modo de ayuda, en las filminas que siguen se da un método imperfecto para ir aprendiendo.
- El método supone que ya sabemos cuál **operación** queremos contar.

Número de operaciones de un programa

Secuencia de comandos

- Una secuencia de comandos se ejecuta de manera secuencial, del primero al último.
- La secuencia se puede escribir horizontalmente:
 $C_1; C_2; \dots; C_n$
- o verticalmente

 C_1 C_2 $\vdots$ C_n

Número de operaciones de un programa

Secuencia de comandos

- Para contar cuántas veces se ejecuta **la operación**, entonces, se cuenta cuántas veces se ejecuta en el primero, cuántas en el segundo, etc. y luego se suman los números obtenidos:
- $\text{ops}(C_1; C_2; \dots; C_n) = \text{ops}(C_1) + \text{ops}(C_2) + \dots + \text{ops}(C_n)$
- $\text{ops} \begin{pmatrix} C_1 \\ C_2 \\ \vdots \\ C_n \end{pmatrix} = \text{ops}(C_1) + \text{ops}(C_2) + \dots + \text{ops}(C_n)$

Número de operaciones de un programa

Comando **skip**

- El comando **skip** equivale a una secuencia vacía:
- $\text{ops}(\text{skip}) = 0$

Número de operaciones de un programa

Comando **for**

- El comando **for** $k := n$ **to** m **do** $C(k)$ **od** “equivale” también a una secuencia:
- **for** $k := n$ **to** m **do** $C(k)$ **od** “equivale” a

 $C(n)$ $C(n+1)$ $\vdots$ $C(m)$

Número de operaciones de un programa

Comando **for**

- De esta “equivalencia” resulta

$$\begin{aligned}\text{ops}(\text{for } k := n \text{ to } m \text{ do } C(k) \text{ od}) &= \\ &= \text{ops}(C(n)) + \text{ops}(C(n+1)) + \dots + \text{ops}(C(m))\end{aligned}$$

- que también se puede escribir

$$\text{ops}(\text{for } k := n \text{ to } m \text{ do } C(k) \text{ od}) = \sum_{k=n}^m \text{ops}(C(k))$$

Número de operaciones de un programa

Comando **for** (una salvedad importante)

La ecuación

$$\text{ops}(\text{for } k := n \text{ to } m \text{ do } C(k) \text{ od}) = \sum_{k=n}^m \text{ops}(C(k))$$

solamente vale cuando **no hay interés en contar las operaciones que involucran el índice k** implícitas en el **for**: inicialización, comparación con la cota m , incremento; ni el cómputo de los límites n y m . Por eso escribimos “equivale” entre comillas.

Número de operaciones de un programa

Comando condicional **if**

- El comando **if b then C else D fi** se ejecuta evaluando la condición **b** y luego, en función del valor de verdad que se obtenga, ejecutando **C** (caso verdadero) o **D** (caso falso).
- Para contar cuántas veces se ejecuta **la operación**, entonces, se cuenta cuántas veces se la ejecuta durante la evaluación de **b** y luego cuántas en la ejecución de **C** o **D**
- $$\text{ops}(\text{if } b \text{ then } C \text{ else } D \text{ fi}) = \begin{cases} \text{ops}(b) + \text{ops}(C) & \text{caso } b \text{ V} \\ \text{ops}(b) + \text{ops}(D) & \text{caso } b \text{ F} \end{cases}$$

Número de operaciones de un programa

Asignación

- El comando $x:=e$ se ejecuta evaluando la expresión e y modificando la posición de memoria donde se aloja la variable x con el valor de e .



$$\text{ops}(x:=e) = \begin{cases} \text{ops}(e)+1 & \text{si se desea contar la asignación} \\ & \text{o las modificaciones de memoria} \\ \text{ops}(e) & \text{en caso contrario} \end{cases}$$

- Tener en cuenta que la evaluación de e puede implicar la llamada a funciones auxiliares cuyas operaciones deben ser también contadas.

Número de operaciones de una expresión

- Similares ecuaciones se pueden obtener para la evaluación de expresiones.
- Por ejemplo, para evaluar la expresión $e < f$, primero se evalúa la expresión e , luego se evalúa la expresión f y luego se comparan dichos valores.



$$\text{ops}(e < f) = \begin{cases} \text{ops}(e) + \text{ops}(f) + 1 & \text{si se cuentan comparaciones} \\ \text{ops}(e) + \text{ops}(f) & \text{caso contrario} \end{cases}$$

Ejemplo: número de comparaciones de la ordenación por selección

```
proc selection_sort (in/out a: array[1..n] of T)  
  var minp: nat  
  for i:= 1 to n do  
    minp:= min_pos_from(a,i)  
    swap(a,i,minp)  
  od  
end proc  
  
fun min_pos_from (a: array[1..n] of T, i: nat) ret minp: nat  
  minp:= i  
  for j:= i+1 to n do if a[j] < a[minp] then minp:= j fi  
  od  
end fun
```

Ejemplo: número de comparaciones de la ordenación por selección

```
ops(selection_sort(a))
= ops(for i:= 1 to n do minp:= min_pos_fr...;swap... od)
=  $\sum_{i=1}^n \text{ops}(\text{minp}:= \text{min\_pos\_from}(a,i);\text{swap}(a,i,\text{minp}))$ 
=  $\sum_{i=1}^n (\text{ops}(\text{minp}:= \text{min\_pos\_from}(a,i)) + \text{ops}(\text{swap}(a,i,\text{minp})))$ 
=  $\sum_{i=1}^n \text{ops}(\text{minp}:= \text{min\_pos\_from}(a,i))$ 
=  $\sum_{i=1}^n \text{ops}(\text{min\_pos\_from}(a,i))$ 
=  $\sum_{i=1}^n \text{ops}(\text{minp}:= i; \text{for } j:= i+1 \text{ to } n \text{ do if } \dots \text{fi } \text{od})$ 
```

Ejemplo: número de comparaciones de la ordenación por selección

$$\begin{aligned}
 & \text{ops(selection_sort(a))} \\
 = & \sum_{i=1}^n \text{ops}(\text{minp} := i; \text{for } j := i+1 \text{ to } n \text{ do if } \dots \text{fi od}) \\
 = & \sum_{i=1}^n (\text{ops}(\text{minp} := i) + \text{ops}(\text{for } j := i+1 \text{ to } n \text{ do if } \dots \text{fi od})) \\
 = & \sum_{i=1}^n \text{ops}(\text{for } j := i+1 \text{ to } n \text{ do if } \dots \text{fi od}) \\
 = & \sum_{i=1}^n \sum_{j=i+1}^n \text{ops}(\text{if } a[j] < a[\text{minp}] \text{ then minp} := j \text{ if}) \\
 = & \sum_{i=1}^n \sum_{j=i+1}^n (\text{ops}(a[j] < a[\text{minp}]) + \text{ops}(\text{minp} := j)) \text{ o ops}(\text{skip}) \\
 = & \sum_{i=1}^n \sum_{j=i+1}^n \text{ops}(a[j] < a[\text{minp}])
 \end{aligned}$$

Ejemplo: número de comparaciones de la ordenación por selección

$$\begin{aligned}\text{ops}(\text{selection_sort}(a)) &= \sum_{i=1}^n \sum_{j=i+1}^n \text{ops}(a[j] < a[\text{minp}]) \\ &= \sum_{i=1}^n \sum_{j=i+1}^n 1 \\ &= \sum_{i=1}^n (n-i) \\ &= \sum_{i=0}^{n-1} i \\ &= \frac{n \cdot (n-1)}{2} \\ &= \frac{n^2}{2} - \frac{n}{2}\end{aligned}$$

Ejemplo: número de intercambios de la ordenación por selección

```
ops(selection_sort(a))
= ops(for i:= 1 to n do minp:= min_pos_fr...;swap... od)
=  $\sum_{i=1}^n \text{ops}(\text{minp}:= \text{min\_pos\_from}(a,i);\text{swap}(a,i,\text{minp}))$ 
=  $\sum_{i=1}^n (\text{ops}(\text{minp}:= \text{min\_pos\_from}(a,i)) + \text{ops}(\text{swap}(a,i,\text{minp})))$ 
= ... =  $\sum_{i=1}^n (0 + \text{ops}(\text{swap}(a,i,\text{minp})))$ 
=  $\sum_{i=1}^n \text{ops}(\text{swap}(a,i,\text{minp}))$ 
=  $\sum_{i=1}^n 1$ 
= n
```

Conclusión del ejemplo

- Número de comparaciones de la ordenación por selección:
 $\frac{n^2}{2} - \frac{n}{2}$
- Número de intercambios de la ordenación por selección: n
- Esto significa que la operación de **intercambio no es representativa** del comportamiento de la ordenación por selección, ya que el número de comparaciones crece más que proporcionalmente respecto a los intercambios.
- Por otro lado, pudimos contar las operaciones de manera **exacta**.

Ordenación por inserción

- **No siempre** es posible contar el **número exacto** de operaciones.
- Un ejemplo de ello lo brinda otro algoritmo de ordenación: la ordenación por inserción.
- Es un algoritmo que se utiliza por ejemplo en juegos de cartas, cuando es necesario mantener un gran número de cartas en las manos, en forma ordenada.
- Cada carta que se levanta de la mesa, se inserta en el lugar correspondiente entre las que ya están en las manos, manteniéndolas ordenadas.

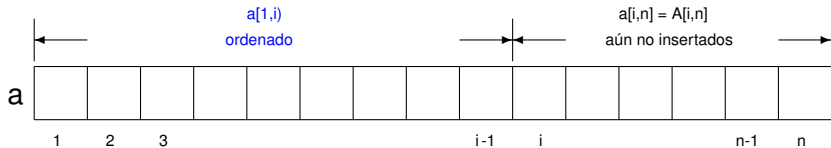
Ordenación por inserción

Ejemplo

9	3	1	3	5	2	7
9	3	1	3	5	2	7
3	9	1	3	5	2	7
3	1	9	3	5	2	7
1	3	9	3	5	2	7
1	3	3	9	5	2	7
1	3	3	5	9	2	7

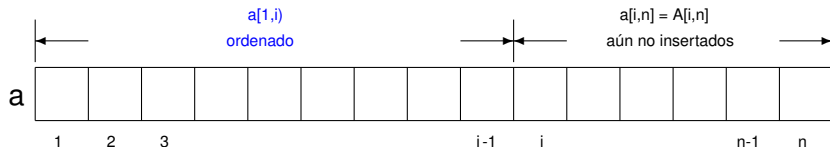
1	3	3	5	2	9	7
1	3	3	2	5	9	7
1	3	2	3	5	9	7
1	2	3	3	5	9	7
1	2	3	3	5	7	9

Ordenación por inserción



Ordenación por inserción

Invariante



- Invariante:

- el arreglo a es una permutación del original y
- un segmento inicial $a[1,i]$ del arreglo está ordenado.
- (pero en general $a[1,i]$ **no** contiene los mínimos del arreglo)

Ordenación por inserción

Pseudocódigo

{Pre: $n \geq 0 \wedge a = A$ }

proc insertion_sort (**in/out** a: **array**[1..n] **of** T)

for i:= 2 **to** n **do**

 {Inv: Invariante de recién}

 insert(a,i)

od

end proc

{Post: a está ordenado y es permutación de A}

Ordenación por inserción

Invariante del procedimiento de inserción



● Invariante:

- el arreglo a es una permutación del original
- $a[1, j]$ sin celda j está ordenado, y
- $a[j, i]$ también está ordenado.

Ordenación por inserción

Procedimiento de inserción

$$\{\text{Pre: } 0 < i \leq n \wedge a = A\}$$

```
proc insert (in/out a: array[1..n] of T, in i: nat)
```

```
var j: nat
```

$$j := i$$

{Inv: Invariante de recién}

do $j > 1 \wedge a[j] < a[j - 1] \rightarrow \text{swap}(a, j-1, j)$

$$j := j - 1$$

od

end proc

{Post: $a[1,i]$ está ordenado $\wedge$ a es permutación de A }

Ordenación por inserción

Todo junto

```
proc insertion_sort (in/out a: array[1..n] of T)
  for i:= 2 to n do
    insert(a,i)
  od
end proc

proc insert (in/out a: array[1..n] of T, in i: nat)
  var j: nat
  j:= i
  do  $j > 1 \wedge a[j] < a[j - 1]$   $\rightarrow$  swap(a,j-1,j)
    do
      j:= j-1
    od
  end proc
```

Número de Comparaciones e intercambios

Procedimiento insert(a,i)

	comparaciones		intercambios	
si el valor de i es ...	mín	máx	mín	máx
2	1	1	0	1
3	1	2	0	2
4	1	3	0	3
⋮	⋮	⋮	⋮	⋮
n	1	n-1	0	n-1
total insertion_sort	n - 1	$\frac{n^2}{2} - \frac{n}{2}$	0	$\frac{n^2}{2} - \frac{n}{2}$

Ordenación por inserción, casos

- mejor caso: arreglo ordenado, n comparaciones y 0 intercambios.
- peor caso: arreglo ordenado al revés, $\frac{n^2}{2} - \frac{n}{2}$ comparaciones e intercambios, es decir, del orden de n^2 .
- caso promedio: del orden de n^2 .

Número de operaciones de un programa

El ciclo **do**

- El ciclo **do** $b \rightarrow C$ **od** (o equivalente **while** b **do** C **od**) se ejecuta evaluando la condición b , y dependiendo de si su valor es V o F se continúa de la siguiente manera:
 - si su valor fue F, la ejecución termina inmediatamente
 - si su valor fue V, la ejecución continúa con la ejecución del cuerpo C del ciclo, y luego de eso vuelve a ejecutarse todo el ciclo nuevamente.
- Es decir que su ejecución es una secuencia de evaluaciones de la condición b y ejecuciones del cuerpo C que finaliza con la primera evaluación de b que dé F.

Número de operaciones de un programa

El ciclo **do**

Es decir, la ejecución del ciclo **do** $b \rightarrow C$ **od** “equivale” a la ejecución de

if b **then** C

if b **then** C

if b **then** C

if b **then** C

 ... $\parallel$ indefinidamente !!

else skip

else skip

else skip

else skip

Número de operaciones de un programa

El ciclo **do**

$$\text{ops}(\mathbf{do\ } b \rightarrow C \mathbf{ od}) = \text{ops}(b) + \sum_{k=1}^n d_k$$

donde

- n es el número de veces que se ejecuta el cuerpo del **do**
- d_k es el número de operaciones que realiza la k -ésima ejecución del cuerpo C del ciclo y la subsiguiente evaluación de la condición o guarda b

Resumen

- Hemos analizado dos algoritmos de ordenación
 - ordenación por selección
 - ordenación por inserción
- la ordenación por selección hace siempre el mismo número de comparaciones, del orden de n^2 .
- la ordenación por inserción también es del orden de n^2 en el peor caso (arreglo ordenado al revés) y en el caso medio,
- la ordenación por inserción es del orden de n en el mejor caso (arreglo ordenado),
- la ordenación por inserción realiza del orden de n^2 swaps (contra n de la ordenación por selección) en el peor caso.

Problema del profe de algoritmos 2

- Con cualquiera de los dos algoritmos la respuesta es 2 horas,
- salvo que se trate de un conjunto ya ordenado o casi ordenado, en cuyo caso:
 - ordenación por inserción es del orden de n ,
 - y por ello la respuesta sería: 1 hora.

Repaso de la ordenación por selección

```
proc selection_sort (in/out a: array[1..n] of T)  
  var minp: nat  
  for i:= 1 to n do  
    minp:= min_pos_from(a,i)  
    swap(a,i,minp)  
  od  
end proc  
  
fun min_pos_from (a: array[1..n] of T, i: nat) ret minp: nat  
  minp:= i  
  for j:= i+1 to n do if a[j] < a[minp] then minp:= j fi  
  od  
end fun
```

Se lo puede abreviar omitiendo la función auxiliar.

Forma abreviada de la ordenación por selección

```
proc selection_sort (in/out a: array[1..n] of T)
  var minp: nat
  for i:= 1 to n do
    minp:= i
    for j:= i+1 to n do
      if a[j] < a[minp] then minp:= j fi
    od
    swap(a,i,minp)
  od
end proc
```


Repaso de la ordenación por inserción

```
proc insertion_sort (in/out a: array[1..n] of T)
  for i:= 2 to n do
    insert(a,i)
  od
end proc

proc insert (in/out a: array[1..n] of T, in i: nat)
  j:= i
  do  $j > 1 \wedge a[j] < a[j - 1]$   $\rightarrow$  swap(a,j-1,j)
    j:= j-1
  od
end proc
```

También puede abreviarse omitiendo el procedimiento auxiliar.

Forma abreviada de la ordenación por inserción

```
proc insertion_sort (in/out a: array[1..n] of T)
  for i:= 2 to n do
    j:= i
    do  $j > 1 \wedge a[j] < a[j - 1]$   $\rightarrow$  swap(a,j-1,j)
      j:= j-1
    od
  od
end proc
```

Algoritmos y Estructuras de Datos II

Ordenación avanzada

Contenidos

- 1 Ayudando al profe
- 2 Ordenación por intercalación
 - La idea
 - El algoritmo
 - Ejemplo
- 3 Ordenación rápida
 - El algoritmo
 - Ejemplo

Ayudando al profe

El profe de esta materia tarda media hora en ordenar alfabéticamente 100 exámenes. ¿Cuánto tardará en ordenar 200 exámenes?

Respuesta hasta ahora: 2 horas.

Ayuda Mauro y Santiago se ofrecen a colaborar para ordenar los 200 exámenes.

¿Cómo puede aprovechar el profe la colaboración?

Idea

Mauro Ordena 100 exámenes.

Santiago Ordena otros 100 exámenes.

Profe Al finalizar, los intercala.

¿Cuánto tarda todo esto?

Mauro + Santiago Media hora.

Profe ¿Cuánto tiempo lleva la intercalación?

Intercalando exámenes

¿Cómo intercalar dos secuencias con 100 exámenes ordenados?

- Comparar el primer examen de una, con el primer examen de la otra. Uno de esos tiene que ser el menor. Sacarlo de esa secuencia y colocarlo como primer examen.
1 comparación $\rightarrow$ 1 examen en su lugar.
- Comparar el primer examen de una, con el primer examen de la otra. Uno de esos tiene que ser el segundo menor. Sacarlo de esa secuencia y colocarlo como segundo examen.
2 comparaciones $\rightarrow$ 2 exámenes en su lugar.
- Etcétera.
- n comparaciones $\rightarrow$ n exámenes en su lugar.

Terminando de intercalar exámenes

- Puede pasar que las dos secuencias ordenadas se van vaciando en forma pareja y la intercalación termina cuando se han colocado en su lugar los primeros 99 exámenes de cada secuencia, es decir, colocado en su lugar 198 con 198 comparaciones. Una última comparación sirve para determinar cuál de los dos exámenes que quedan es el penúltimo, y cuál es el último. 199 comparaciones $\rightarrow$ 200 exámenes en su lugar.
- La otra posibilidad es que una secuencia ordenada se vacía cuando la otra todavía tiene, por ejemplo, 20 exámenes. Se han colocado 180 exámenes con 180 comparaciones. Los restantes 20 exámenes pueden colocarse en su lugar sin ninguna comparación adicional.

¿Cuánto tiempo llevó intercalar 200 exámenes?

- Peor caso: 199 comparaciones.
- Mejor caso: 100 comparaciones.

Entonces ¿cuánto lleva ordenar los 200 exámenes?

ordenar 100 exámenes	$\longleftrightarrow$	10.000 comparaciones
ordenar 100 exámenes	$\longleftrightarrow$	10.000 comparaciones
intercalar 200 exámenes	$\longleftrightarrow$	200 comparaciones

ordenar 100 exámenes	$\longleftrightarrow$	1/2 hora	} simultáneo
ordenar 100 exámenes	$\longleftrightarrow$	1/2 hora	
intercalar 200 exámenes	$\longleftrightarrow$	$\frac{1}{50}$ 1/2 hora	

$$\frac{1}{50} \text{ 1/2 hora} = 36 \text{ segundos.}$$

$$(1/2 \text{ hora} = 1800 \text{ segundos})$$

Ordenar 200 exámenes, con esta nueva idea

Tarea A Ordenar 100 exámenes como antes, 10.000 de comparaciones, 1/2 hora.

Tarea B Ordenar 100 exámenes como antes, 10.000 de comparaciones, 1/2 hora.

Tarea C Intercalar 200 exámenes, 200 comparaciones, 36 segundos.

Total 1 hora y 36 segundos (ignorando que las tareas A y B pueden realizarse en paralelo).

¡Ordenamos 200 exámenes en poco más de 1 hora! (sin la ayuda de Mauro y Santiago)

¿Podemos hacer algo mejor?

Ordenar 200 exámenes, aprovechando más la idea

Tarea A Ordenar 100 exámenes:

Tarea AA Ordenar 50, 2.500 comparaciones, $1/8$ hora.

Tarea AB Ordenar 50, 2.500 comparaciones, $1/8$ hora.

Tarea AC Intercalar 100, 100 comparaciones, 18 segundos.

Total Tarea A 5.100 comparaciones, $1/4$ hora y 18 segundos.

Tarea B Como Tarea A, 5.100 comparaciones, $1/4$ hora y 18 segundos.

Tarea C Intercalar 200, 200 comparaciones, 36 segundos.

Total 10.400 comparaciones, $1/2$ hora y 72 segundos.

Ordenar 200 exámenes, aprovechando más la idea

Tarea A Ordenar 100 exámenes:

Tarea AA Ordenar 50 exámenes:

Tarea AAA Ordenar 25, 625 comparaciones, 1/32 hora.

Tarea AAB Ordenar 25, 625 comparaciones, 1/32 hora.

Tarea AAC Intercalar 50, 50 comparaciones, 9 segundos.

Total Tarea AA 1.300 comparaciones, 1/16 hora y 9 segundos.

Tarea AB 1.300 comparaciones, 1/16 hora y 9 segundos.

Tarea AC Intercalar 100, 100 comparaciones, 18 segundos.

Total Tarea A 2.700 comparaciones, 1/8 hora y 36 segundos.

Tarea B 2.700 comparaciones, 1/8 hora y 36 segundos.

Tarea C Intercalar 200, 200 comparaciones, 36 segundos.

Total 5.600 comparaciones, 1/4 hora y 108 segundos.

Reflexionando sobre lo que acabamos de hacer

ordenar ¹ bloques de	tardanza	segundos
200 exámenes	2 horas	7200
100 exámenes	1 hora y 36 segundos	3636
50 exámenes	1/2 hora y 72 segundos	1872
25 exámenes	1/4 hora y 108 segundos	1008
13 exámenes	1/8 hora y 144 segundos	594
7 exámenes	1/16 hora y 180 segundos	405
4 exámenes	1/32 hora y 216 segundos	328
2 exámenes	1/64 hora y 252 segundos	308
1 examen	1/128 hora y 288 segundos	316

¹ usando ordenación por selección o por inserción

Conclusión

- ¿Por qué no “ordenar” bloques de 1, y luego intercalar reiteradamente?
- ¡Podríamos ordenar 200 exámenes en 5 minutos!
- Pero ordenar bloques de 1 es trivial, ¡cada bloque de 1 está ordenado!
- ¡Entonces esta manera de ordenar **solamente intercala!**
- Esto se llama **ordenación por intercalación** o **merge sort** en inglés.
- No es tan sencillo de escribir en lenguajes imperativos (porque la operación de intercalación requiere espacio auxiliar).

En el lenguaje imperativo

{Pre: $n \geq rgt \geq lft > 0 \wedge a = A$ }

proc merge_sort_rec (**in/out** a: **array**[1..n] **of** T, **in** lft,rgt: **nat**)

var mid: **nat**

if rgt > lft $\rightarrow$ mid:= (rgt+lft) $\div$ 2

 merge_sort_rec(a,lft,mid)

{a[lft,mid] permutación ordenada de A[lft,mid]}

 merge_sort_rec(a,mid+1,rgt)

{a[mid+1,rgt] permutación ordenada de A[mid+1,rgt]}

 merge(a,lft,mid,rgt)

{a[lft,rgt] permutación ordenada de A[lft,rgt]}

fi

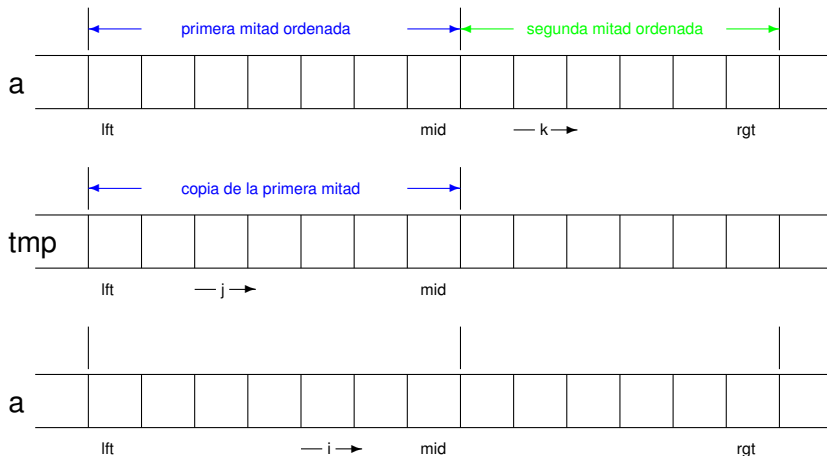
end proc

{Post: a permutación de A $\wedge$ a[lft,rgt] permutación ordenada de A[lft,rgt]}

Algoritmo principal

```
proc merge_sort (in/out a: array[1..n] of T)  
    merge_sort_rec(a,1,n)  
end proc
```


Invariante del procedimiento intercalación



Intercalación en el lenguaje de la materia

```
proc merge (in/out a: array[1..n] of T, in lft,mid,rgt: nat)  
  var tmp: array[1..n] of T  
    j,k: nat  
  for i:= lft to mid do tmp[i]:=a[i] od  
  j:= lft  
  k:= mid+1  
  for i:= lft to rgt do  
    if  $j \leq \text{mid} \wedge (k > \text{rgt} \vee \text{tmp}[j] \leq a[k])$  then a[i]:= tmp[j]  
                                          j:=j+1  
    else a[i]:= a[k]  
        k:=k+1  
    fi  
  od  
end proc
```

Ejemplo

9	3	1	3	5	2	7
9	3	1	3	5	2	7
9	3	1	3	5	2	7
9	3	1	3	5	2	7
9	3	1	3	5	2	7
9	3	1	3	5	2	7
3	9	1	3	5	2	7
3	9	1	3	5	2	7
3	9	1	3	5	2	7
3	9	1	3	5	2	7

3	9	1	3	5	2	7
1	3	3	9	5	2	7
1	3	3	9	5	2	7
1	3	3	9	5	2	7
1	3	3	9	5	2	7
1	3	3	9	5	2	7
1	3	3	9	2	5	7
1	3	3	9	2	5	7
1	3	3	9	2	5	7
1	2	3	3	5	7	9

Ejemplo de intercalación

1	3	3	9
1	3	3	9
	3	3	9
	3	3	9
		3	9
			9
			9
			9

1	3	3	9	2	5	7
				2	5	7
				2	5	7
1				2	5	7
1	2				5	7
1	2	3			5	7
1	2	3	3		5	7
1	2	3	3	5		7
1	2	3	3	5	7	
1	2	3	3	5	7	9

Una idea similar

- La idea original fue
 - Que se ordenen las dos mitades separadamente.
 - Que luego se intercalen las dos mitades ya ordenadas.
 - Este proceso, iterado, dio lugar a la ordenación por intercalación.
- otra idea parecida puede ser:
 - Separar en dos mitades: por un lado los que irían al principio y por el otro los que irían al final (por ejemplo: de la A a la L y de la M a la Z).
 - Que se ordenen las dos mitades separadamente.
 - Que finalmente se junten las dos mitades ordenadas.
 - Esta idea da lugar al algoritmo conocido por quicksort u ordenación rápida.

Implementación de Ordenación rápida

```
{Pre:  $0 \leq rgt \leq n \wedge 1 \leq lft \leq n+1 \wedge lft-1 \leq rgt \wedge a = A$ }  
proc quick_sort_rec (in/out a: array[1..n] of T, in lft,rgt: nat)  
  var ppiv: nat  
  if rgt > lft  $\rightarrow$  partition(a,lft,rgt,ppiv)  
    lft  $\leq$  ppiv  $\leq$  rgt  
    elementos en  $a[lft,ppiv-1] \leq a[ppiv]$   
    elementos en  $a[ppiv+1,rgt] \geq a[ppiv]$   
    quick_sort_rec(a,lft,ppiv-1)  
    quick_sort_rec(a,ppiv+1,rgt)  
  fi  
end proc  
{Post: a permut. de A  $\wedge$   $a[lft,rgt]$  permut. ordenada de  $A[lft,rgt]$ }
```

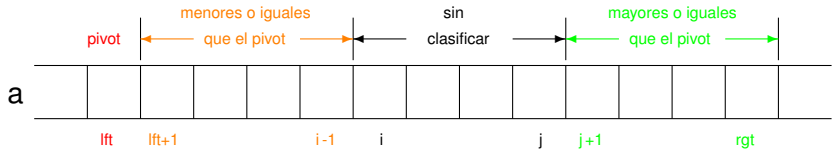
Algoritmo principal

```
proc quick_sort (in/out a: array[1..n] of T)  
    quick_sort_rec(a,1,n)  
end proc
```

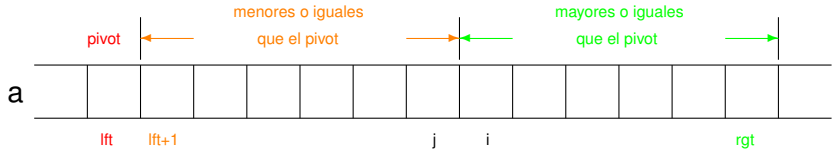
Procedimiento partition

```
proc partition (in/out a: array[1..n] of T, in lft, rgt: nat, out ppiv: nat)  
  var i,j: nat  
  ppiv:= lft  
  i:= lft+1  
  j:= rgt  
  do  $i \leq j \rightarrow$  if  $a[i] \leq a[ppiv] \rightarrow i:= i+1$   
     $a[j] \geq a[ppiv] \rightarrow j:= j-1$   
     $a[i] > a[ppiv] \wedge a[j] < a[ppiv] \rightarrow$  swap(a,i,j)  
     $i:= i+1$   
     $j:= j-1$   
  fi  
  od  
  swap(a,ppiv,j)      {dejando el pivot en una posición más central}  
  ppiv:= j            {señalando la nueva posición del pivot }  
end proc
```


Invariante del procedimiento partition



al finalizar queda así:



y se hace un swap entre las posiciones lft y j.

Pre, post e invariante

- {Pre: $1 \leq \text{lft} < \text{rgt} \leq n \wedge a = A$ }
- {Post: $a[1, \text{lft}) = A[1, \text{lft}) \wedge a(\text{rgt}, n] = A(\text{rgt}, n]$
 $\wedge a[\text{lft}, \text{rgt}]$ permutación de $A[\text{lft}, \text{rgt}]$
 $\wedge \text{lft} \leq \text{piv} \leq \text{rgt}$
 $\wedge$ los elementos de $a[\text{lft}, \text{piv}]$ son $\leq$ que $a[\text{piv}]$
 $\wedge$ los elementos de $a(\text{piv}, \text{rgt}]$ son $>$ que $a[\text{piv}]$ }
- {Inv: $\text{lft} = \text{piv} < i \leq j+1 \leq \text{rgt}+1$
 $\wedge$ todos los elementos en $a[\text{lft}, i)$ son $\leq$ que $a[\text{piv}]$
 $\wedge$ todos los elementos en $a(j, \text{rgt}]$ son $>$ que $a[\text{piv}]$ }

Ejemplo

3	9	2	1	7	3	5
3	9	2	1	7	3	5
3	1	2	9	7	3	5
2	1	3	9	7	3	5
2	1	3	9	7	3	5
2	1	3	9	7	3	5
1	2	3	9	7	3	5
1	2	3	9	7	3	5
1	2	3	9	7	3	5

1	2	3	5	7	3	9
1	2	3	5	7	3	9
1	2	3	5	3	7	9
1	2	3	3	5	7	9
1	2	3	3	5	7	9
1	2	3	3	5	7	9
1	2	3	3	5	7	9

Ejemplo de partition

3	9	2	1	7	3	5
3	9	2	1	7	3	5
3	9	2	1	7	3	5
3	9	2	1	7	3	5
3	1	2	9	7	3	5
3	1	2	9	7	3	5
2	1	3	9	7	3	5

Algoritmos y Estructuras de Datos II

Recurrencias Divide y Vencerás

Recurrencias

- Surgen al analizar algoritmos recursivos, como la ordenación por intercalación.
- El conteo de operaciones “copia” la recursión del algoritmo y se vuelve recursivo también.
- Ejemplo: máximo de comparaciones de la ordenación por intercalación.
- Es un ejemplo de algoritmo divide y vencerás.
- Es un ejemplo de recurrencia divide y vencerás:

$$t(n) = \begin{cases} 0 & \text{si } n \in \{0, 1\} \\ t(\lceil n/2 \rceil) + t(\lfloor n/2 \rfloor) + n - 1 & \text{si } n > 1 \end{cases}$$

Algoritmo divide y vencerás

Características

- hay una solución para los casos sencillos,
- para los complejos, se **divide** o **descompone** el problema en subproblemas:
 - cada subproblema es de igual naturaleza que el original,
 - el tamaño del subproblema es una **fracción** del original,
 - se resuelven los subproblemas apelando al mismo algoritmo,
- se **combinan** esas soluciones para obtener una solución del original.

Algoritmo divide y vencerás

Forma general

```
fun DyV( $x$ ) ret  $y$   
  if  $x$  suficientemente pequeño o simple then  $y := \text{ad\_hoc}(x)$   
  else descomponer  $x$  en  $x_1, x_2, \dots, x_a$   
    for  $i := 1$  to  $a$  do  $y_i := \text{DyV}(x_i)$  od  
    combinar  $y_1, y_2, \dots, y_a$  para obtener la solución  $y$  de  $x$   
  fi  
end fun
```

Normalmente los x_i son **fracciones** de x :

$$|x_i| = \frac{|x|}{b}$$

para algún b fijo mayor que 1.

Algoritmo divide y vencerás

Ejemplos

- Ordenación por intercalación:
 - “x simple” = fragmento de arreglo de longitud 0 ó 1
 - “descomponer” = partir al medio ($b = 2$)
 - $a = 2$
 - “combinar” = intercalar
- Ordenación rápida:
 - “x simple” = fragmento de arreglo de longitud 0 ó 1
 - “descomponer” = separar los menores de los mayores ($b = 2$)
 - $a = 2$
 - “combinar” = yuxtaponer

Algoritmo divide y vencerás

Forma general

```
fun DyV( $x$ ) ret  $y$   
  if  $x$  suficientemente pequeño o simple then  $y := \text{ad\_hoc}(x)$   
  else descomponer  $x$  en  $x_1, x_2, \dots, x_a$   
    for  $i := 1$  to  $a$  do  $y_i := \text{DyV}(x_i)$  od  
    combinar  $y_1, y_2, \dots, y_a$  para obtener la solución  $y$  de  $x$   
  fi  
end fun
```

- a : número de llamadas recursivas a DyV.
- b : relación entre el tamaño de x y el de x_i , satisface $|x_i| = \frac{|x|}{b}$.
- k : el orden de descomponer y combinar es n^k .

Algoritmo divide y vencerás

Conteo

Si queremos contar el costo computacional (número de operaciones) $t(n)$ de la función DyV obtenemos:

$$t(n) = \begin{cases} c & \text{si la entrada es pequeña o simple} \\ a * t(n/b) + g(n) & \text{en caso contrario} \end{cases}$$

si c es una constante que representa el costo computacional de la función ad_hoc y $g(n)$ es el costo computacional de los procesos de descomposición y de combinación.

Esta definición de $t(n)$ es recursiva (como el algoritmo DyV), se llama **recurrencia**. Existen distintos tipos de recurrencia. Ésta se llama **recurrencia divide y vencerás**.

Recurrencias divide y vencerás

- Si

$$t(n) = \begin{cases} c & \text{si la entrada es pequeña o simple} \\ a * t(n/b) + g(n) & \text{en caso contrario} \end{cases}$$

si $t(n)$ es no decreciente, y $g(n)$ es del orden de n^k ,
entonces

-

$$t(n) \text{ es del orden de } \begin{cases} n^{\log_b a} & \text{si } a > b^k \\ n^k \log n & \text{si } a = b^k \\ n^k & \text{si } a < b^k \end{cases}$$

Ejemplo: búsqueda binaria

{Pre: $1 \leq \text{lft} \leq n+1 \wedge 0 \leq \text{rgt} \leq n \wedge a$ ordenado}

fun binary_search_rec (a: **array**[1..n] **of** T, x:T, lft, rgt : **nat**) **ret** i:**nat**
 var mid: **nat**

if lft > rgt $\rightarrow$ i = 0

 lft $\leq$ rgt $\rightarrow$ mid := (lft+rgt) $\div$ 2

if x < a[mid] $\rightarrow$ i := binary_search_rec(a, x, lft, mid-1)

 x = a[mid] $\rightarrow$ i := mid

 x > a[mid] $\rightarrow$ i := binary_search_rec(a, x, mid+1, rgt)

fi

fi

end fun

{Post: (i = 0 $\Rightarrow$ x no está en a[lft,rgt]) $\wedge$ (i $\neq$ 0 $\Rightarrow$ x = a[i])}

Búsqueda binaria

Función principal

{Pre: $n \geq 0$ }

fun binary_search (a: **array**[1..n] **of** T, x:T) **ret** i:nat

i:= binary_search_rec(a, x, 1, n)

end fun

{Post: $(i = 0 \Rightarrow x \text{ no está en } a) \wedge (i \neq 0 \Rightarrow x = a[i])$ }

Búsqueda binaria

Análisis

- Sea $t(n)$ = número de comparaciones que hace en el peor caso cuando el arreglo tiene n celdas.



$$t(n) = \begin{cases} 0 & \text{si } n = 0 \\ t(n/2) + 1 & \text{si } n > 0 \end{cases}$$

- $a = 1$, $b = 2$ y $k = 0$.
- $a = b^k$.
- $t(n)$ es del orden de $n^k \log n$, es decir, del orden de $\log n$.

Análisis de algunos algoritmos

- Ordenación por selección es del orden de n^2 .
- Ordenación por inserción es del orden de n^2 (peor caso y caso medio).
- Ordenación por intercalación es del orden de $n \log_2 n$.
- Ordenación rápida es del orden de $n \log_2 n$ (caso medio).
- Búsqueda lineal es del orden de n .
- Búsqueda binaria es del orden de $\log_2 n$.

¿Cómo comparar los órdenes de los algoritmos?

- Hay funciones que crecen más rápido que otras (cuando n tiende a $+\infty$).
- Escribiremos $f(n) \sqsubset g(n)$ para decir que $g(n)$ crece más rápido que $f(n)$. Por ejemplo:
 - $n \log_2 n \sqsubset n^2$.
 - $\log_2 n \sqsubset n$.
- Escribiremos $f(n) \approx g(n)$ para decir que $f(n)$ y $g(n)$ crecen al mismo ritmo. Por ejemplo:
 - $\frac{n^2}{2} - \frac{n}{2} \approx n^2$.
 - $45n^2 \approx n^2$.

Algunas condiciones

- No nos interesan las constantes multiplicativas:
 - $\frac{1}{4}n^2 \approx n^2$
 - $4n^3 \approx n^3$
 - $1000 \log n \approx \log n$
 - $\pi n \approx n$
- No nos interesan los términos menos significativos, que crecen más lento:
 - $n^2 + n \approx n^2$
 - $n^3 + n^2 \log_2 n \approx n^3$
 - $\log n + 3456 \approx \log n$
 - $n + \sqrt{n} \approx n$

Jerarquía

$$\begin{aligned} 1 \sqsubset \log_2 n \approx \log_3 n \sqsubset n^{0.001} \sqsubset n^{1.5} \sqsubset n^2 \sqsubset \\ \sqsubset n^5 \sqsubset n^{100} \sqsubset 1.01^n \sqsubset 2^n \sqsubset 100^n \sqsubset \\ \sqsubset 10000^n \sqsubset n! \sqsubset n^n \end{aligned}$$

Propiedades

- Constantes multiplicativas no afectan.
- Términos de crecimiento despreciable no afectan.
- Sean $a, b > 1$, $\log_a n \approx \log_b n$.
- Sea $f(n) > 0$ para “casi todo $n \in \mathbb{N}$ ”. Entonces:
 - $g(n) \sqsubset h(n) \iff f(n)g(n) \sqsubset f(n)h(n)$.
 - $g(n) \approx h(n) \iff f(n)g(n) \approx f(n)h(n)$.
- Sea $\lim_{n \rightarrow \infty} h(n) = \infty$. Entonces:
 - $f(n) \sqsubset g(n) \implies f(h(n)) \sqsubset g(h(n))$.
 - $f(n) \approx g(n) \implies f(h(n)) \approx g(h(n))$.

Jerarquía

$$\begin{aligned} 1 &\sqsubset \log(\log(\log n)) \sqsubset \log(\log n) \sqsubset \log n \sqsubset n^{0.001} \sqsubset \\ &\sqsubset n \sqsubset n \log n \sqsubset n^{1.001} \sqsubset n^{100} \sqsubset 1.01^n \sqsubset \\ &\sqsubset n^{100} * 1.01^n \sqsubset 1.02^n \sqsubset 100^n \sqsubset 10000^n \sqsubset \\ &\sqsubset (n-1)! \sqsubset n! \sqsubset (n+1)! \sqsubset n^n \end{aligned}$$

Extras: Demostración de la recurrencia DyV

Por la forma de la recurrencia

$$t(n) = \begin{cases} c & \text{si la entrada es pequeña o simple} \\ a * t(n/b) + g(n) & \text{en caso contrario} \end{cases}$$

resulta más sencillo calcular $t(n)$ cuando n es potencia de b .

Se organiza la tarea en dos partes

- calcular el orden de $t(n)$ cuando n es potencia de b ,
- extender el cálculo para los demás n .

Calculando para n potencia de b

- Supongamos que



$$t(n) = \begin{cases} c & \text{si la entrada es pequeña o simple} \\ a * t(n/b) + g(n) & \text{en caso contrario} \end{cases}$$

- y $g(n)$ es del orden de n^k , es decir $g(n) \leq dn^k$ para casi todo $n \in \mathbb{N}$
 - n potencia de b , $n = b^m$



$$\begin{aligned} t(n) &= t(b^m) \\ &= a * t(b^m/b) + g(b^m) \\ &\leq a * t(b^{m-1}) + d(b^m)^k \\ &\leq a * t(b^{m-1}) + d(b^k)^m \end{aligned}$$

Iterando

$$\begin{aligned}
 t(b^m) &\leq at(b^{m-1}) + d(b^k)^m \\
 &\leq a(at(b^{m-2}) + d(b^k)^{m-1}) + d(b^k)^m \\
 &\leq a^2t(b^{m-2}) + ad(b^k)^{m-1} + d(b^k)^m \\
 &\leq a^3t(b^{m-3}) + a^2d(b^k)^{m-2} + ad(b^k)^{m-1} + d(b^k)^m \\
 &\leq \dots \\
 &\leq a^mt(1) + a^{m-1}db^k + \dots + ad(b^k)^{m-1} + d(b^k)^m \\
 &= a^mc + d(b^k)^m((a/b^k)^{m-1} + \dots + a/b^k + 1) \\
 &= a^mc + d(b^m)^k(r^{m-1} + \dots + r + 1)
 \end{aligned}$$

donde $r = a/b^k$ y recordemos $n = b^m$ y entonces $m = \log_b n$

$$\begin{aligned}
 t(n) &\leq a^{\log_b n}c + dn^k(r^{m-1} + \dots + r + 1) \\
 &= n^{\log_b a}c + dn^k(r^{m-1} + \dots + r + 1)
 \end{aligned}$$

Propiedad del logaritmo

En el último paso hemos usado que $x^{\log_y z}$ es igual a $z^{\log_y x}$.

- En efecto, si aplicamos $\log_y$ a ambos, obtenemos
- $\log_y(x^{\log_y z})$ y $\log_y(z^{\log_y x})$, que luego de simplificar quedan
- $(\log_y x)(\log_y z)$ y $(\log_y z)(\log_y x)$ que son iguales.
- Como $\log_y$ es inyectiva, $x^{\log_y z} = z^{\log_y x}$ vale.

Volvamos a los cálculos.

Finalizando

$$t(n) \leq n^{\log_b a} c + dn^k(r^{m-1} + \dots + r + 1)$$

donde $r = a/b^k$

- si $r = 1$, entonces $a = b^k$ y $\log_b a = k$ y además

$$t(n) \leq n^k c + dn^k \log_b n$$

es del orden de $n^k \log n$ para n potencia de b

- si $r \neq 1$, entonces

$$t(n) \leq n^{\log_b a} c + dn^k \left(\frac{r^m - 1}{r - 1} \right)$$

Finalizando

caso $r > 1$

- si $r > 1$, como $r = a/b^k$ entonces $a > b^k$ y $\log_b a > k$ y

además

$$\begin{aligned}
 t(n) &\leq n^{\log_b a} c + dn^k \left(\frac{r^m - 1}{r - 1} \right) \\
 &\leq n^{\log_b a} c + \frac{d}{r - 1} n^k r^m \\
 &= n^{\log_b a} c + \frac{d}{r - 1} n^k \frac{a^m}{(b^k)^m} \\
 &= n^{\log_b a} c + \frac{d}{r - 1} n^k \frac{a^m}{(b^m)^k} \\
 &= n^{\log_b a} c + \frac{d}{r - 1} n^k \frac{a^m}{n^k} \\
 &= n^{\log_b a} c + \frac{d}{r - 1} a^m \\
 &= n^{\log_b a} c + \frac{d}{r - 1} a^{\log_b n} \\
 &= n^{\log_b a} c + \frac{d}{r - 1} n^{\log_b a}
 \end{aligned}$$

es del orden de $n^{\log_b a}$ para n potencia de b

Finalizando

caso $r < 1$

- si $r < 1$, como $r = a/b^k$ entonces $a < b^k$ y $\log_b a < k$. Además, $r - 1$ y $r^m - 1$ son negativos, para evitar confusión escribimos $\frac{1-r^m}{1-r}$ en vez de $\frac{r^m-1}{r-1}$.

$$\begin{aligned} t(n) &\leq n^{\log_b a} c + dn^k \left(\frac{1-r^m}{1-r} \right) \\ &= n^{\log_b a} c + \frac{d}{1-r} n^k (1 - r^m) \\ &\leq n^{\log_b a} c + \frac{d}{1-r} n^k \\ &\text{es del orden de } n^k \text{ para } n \text{ potencia de } b \end{aligned}$$

Conclusión

- Si

$$t(n) = \begin{cases} c \\ a * t(n/b) + g(n) \end{cases}$$

si la entrada es pequeña o simple
en caso contrario

- con $g(n)$ del orden de n^k ,
- demostramos

$$t(n) \text{ es del orden de } \begin{cases} n^{\log_b a} & \text{si } a > b^k \\ n^k \log n & \text{si } a = b^k \\ n^k & \text{si } a < b^k \end{cases}$$

- para n potencia de b .

Extendiendo el resultado a todo n

- Hemos calculado el orden de cualquier algoritmo divide y vencerás, para n potencia de b .
- Queremos calcular el orden para todo n .
- Se puede comprobar que si
 - $t(n)$ es no decreciente, y
 - $t(n)$ es del orden de $h(n)$ para potencias de b para cualquiera de las tres funciones $h(n)$ que acabamos de considerar,entonces $t(n)$ es del orden de $h(n)$ (para n arbitrario, no solamente las potencias de b).
- es decir, el resultado puede extenderse para n arbitrario.

Regla del límite para comparar funciones

- Regla del límite. Sean $f(n)$ y $g(n)$ tales que

- $\lim_{n \rightarrow +\infty} f(n) = \lim_{n \rightarrow +\infty} g(n) = \infty$, y
- $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)}$ existe.

Entonces

- si $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = 0$, entonces $f(n) \sqsubset g(n)$.
- si $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = +\infty$, entonces $g(n) \sqsubset f(n)$.
- caso contrario (el límite es un número real positivo), $f(n) \approx g(n)$.

Algoritmos y Estructuras de Datos II

Tipos concretos

Tipos de datos

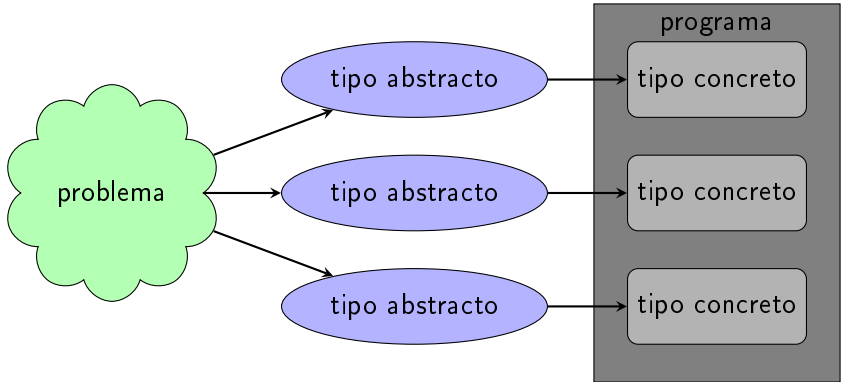
Enumerados

Tuplas

Arreglos

Punteros

Tipos abstractos de datos y su representación



Tipos de datos

Conceptualmente distinguimos dos clases de tipos de datos:

- ▶ Tipos de datos **concretos**:
 - ▶ son provistos por el lenguaje de programación, decimos que son *nativos*,
 - ▶ es un concepto **dependiente** del lenguaje de programación,
 - ▶ comúnmente: enteros, char, string, booleanos, arreglos, reales,
- ▶ Tipos de datos **abstractos**:
 - ▶ **surgen de** analizar **el problema** a resolver,
 - ▶ es un concepto **independiente** del lenguaje de programación,
 - ▶ eventualmente se implementará utilizando tipos concretos,
 - ▶ eso da lugar a una **implementación** o **representación** del tipo abstracto
 - ▶ ejemplo: si se quiere desarrollar una aplicación para un gps que calcule ciertos caminos óptimos, seguramente surgirá considerar un grafo donde las aristas son segmentos de rutas.

Tipos concretos

En estas filminas presentaremos los tipos concretos más complejos que tenemos en el lenguaje de la materia, y que están presentes en la mayoría de los lenguajes de programación.

Saltearemos los mas simples como booleanos, enteros, char, reales, string.

Presentamos a continuación los tipos enumerados, tuplas, arreglos y punteros.

Tipos enumerados

Declaración

- ▶ Un tipo enumerado consiste de una serie finita de constantes, que serán los únicos elementos del tipo.
- ▶ En nuestro lenguaje el programador puede definir un nuevo tipo enumerado de la siguiente forma:

```
type E = enumerate  
    elem1  
    elem2  
    ....  
    elemk  
end enumerate
```

Tipos enumerados

Uso

- ▶ El tipo E así definido tiene **únicamente** los k elementos elem1, elem2, , elemk.
- ▶ En un programa se puede declarar una variable del tipo enumerado E y asignarla a una de las constantes que define:

```
var e: E  
e := elem2
```

- ▶ En nuestro lenguaje permitimos realizar ciclos **for** donde el índice tome los valores de un tipo enumerado:
for i:= elem1 **to** elemk **do** **od**

Tipos enumerados

- ▶ Cuando definimos un tipo enumerado, sus elementos tendrán el orden en que fueron escritos en la definición. En el ejemplo anterior $\text{elem1} < \text{elem2} < \dots < \text{elemk}$.
- ▶ El tipo concreto **char** también puede verse como un tipo enumerado donde las constantes estarán ordenadas lexicográficamente ($'a' < 'b' < 'c' < \dots$).

Tuplas

Declaración

También llamados registros (*records*) o estructuras (*structs*). Las tuplas representan productos cartesianos de distintos tipos. Se puede definir un nuevo tipo mediante una tupla de la siguiente forma:

```
type tperson = tuple  
    name: string  
    age: nat  
    weight: real  
end tuple
```


Tuplas

Uso

- ▶ El tipo `tperson` así definido corresponde al producto $\text{string} \times \text{nat} \times \text{real}$, y `name`, `age` y `weight` se llaman **campos**.
- ▶ Para acceder a los campos de un elemento de tipo tupla utilizamos el operador de acceso escrito con un punto:

```
var manu : tperson  
manu.name := "Emmanuel"  
manu.age := 33  
manu.weight := 68
```

- ▶ En la mayoría de los lenguajes de programación, cuando se ejecuta un programa los elementos de un tipo tupla son alojados en espacios contiguos de memoria y el espacio que ocupa es igual a la suma de los espacios que ocupan sus campos.

Arreglos

Declaración

- ▶ Los arreglos son colecciones de tamaño fijo de elementos del mismo tipo.
- ▶ En general los lenguajes de programación implementan los arreglos alojándolos en espacios contiguos de memoria, y el acceso a cada uno de sus elementos se obtiene en tiempo constante.
- ▶ Para declarar un arreglo en un programa debemos indicar de qué tipo **T** son los elementos y dos números naturales **M** y **N** indicando el primero y último índice respectivamente:

var a: array[M..N] of T

- ▶ El arreglo **a** tiene **N-M** elementos de tipo **T**.

Arreglos

Índices

- ▶ Comunmente en otros lenguajes como C, los índices siempre comienzan en 0, y entonces solo debo indicar cuántos elementos tiene el arreglo.
- ▶ Nuestro lenguaje es más flexible, permitiendo indicar el índice del comienzo y final del arreglo, y también permitimos utilizar como índices a los elementos de un tipo enumerado:

var a: array[elem1..elemk] of T

donde elem1 ... elemk son los elementos del tipo enumerado definido previamente.

var page : array['a'..'z'] of nat

Arreglos

- ▶ También es frecuente la utilización de arreglos multidimensionales,
 - ▶ ejemplos:
- var b: array[1..N,1..M] of string**
var c: array[1..N,'a'..'z',1..M] of nat
- ▶ Para acceder a los elementos de un arreglo utilizamos los corchetes, separando con comas cuando tenemos más de una dimensión:

b[2,3] := "casa"
c[2,'x',1] := 25

Arreglos

Ejemplo inicialización

- Si quiero inicializar el arreglo `c` puedo utilizar tres **for** anidados, de la siguiente manera:

```
for i:= M to N do
  for k:= 'a' to 'z' do
    for d:= 1 to M do
      c[i,k,d]:= 0
    od
  od
od
```

Punteros

Declaración

Dado un tipo T , un **puntero a T** es un tipo de datos que representa el lugar en la memoria en donde está alojado un elemento de tipo T .

Por ejemplo se puede declarar un puntero a **nat** así:

var p: pointer to nat

Punteros

Operaciones

Mediante punteros el programador puede manipular la memoria disponible al ejecutar un programa.

- ▶ Para **reservar** un nuevo bloque de memoria donde pueda almacenar un elemento del tipo al que apunta se utiliza la operación **alloc**

`alloc(p)`

- ▶ Puedo acceder al valor en el bloque de memoria apuntado por `p` mediante la operación `*`

`*p := 10`

decimos que `p` *apunta* al valor 10.

Punteros

Operaciones

- ▶ Para **liberar** un bloque de memoria que haya sido reservado antes se utiliza la operación **free**

`free(p)`

Luego de realizar un `free`, `p` ya no apunta a un bloque de memoria reservado por el programa, por lo cual no se puede saber qué valor tiene la expresión `*p`.

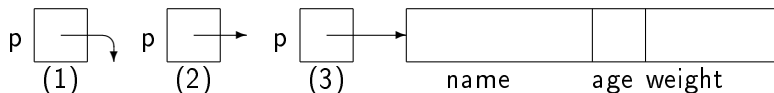
- ▶ Existe una constante para representar punteros que no apunten a nada, a la que llamamos **null**

`p := null`

Punteros

Representación gráfica

Hay distintas representaciones gráficas, una para cada una de las posibles situaciones. Dado p : **pointer to tperson**



- ▶ En la situación (1), el valor de p es **null**, p no señala ninguna posición de memoria.
- ▶ En la situación (2) la posición de memoria señalada por p no está reservada, por ejem. inmediatamente después de `free(p)`.
- ▶ En la situación (3) el valor de p es la dirección de memoria donde se aloja la `tperson` representada gráficamente al final de la flecha, por ejemplo, inmediatamente después de `alloc(p)`.

Punteros

Representación gráfica

- ▶ En la situación (3), $\star p$ denota la `tperson` que se encuentra señalada por `p`, y por lo tanto, $\star p.name$, $\star p.age$ y $\star p.weight$, sus campos.
- ▶ Esta notación permite acceder a la información alojada en la `tperson` y modificarla mediante asignaciones a sus campos (por ejemplo, $\star p.name := \text{"Juan"}$).

Punteros

Notación

- ▶ Una notación conveniente para acceder a los campos de una tupla señalada por un puntero es la flecha “ $\rightarrow$ ”.
- ▶ Así, en vez de escribir $\star p.name$, podemos escribir $p \rightarrow name$ tanto para leer ese campo como para modificarlo.
- ▶ Esta notación reemplaza el uso de dos operadores (“ $\star$ ” y “ $.$ ”) por uno visualmente más apropiado (por ejemplo, $p \rightarrow name = \text{“Juan”}$).
- ▶ La notación $\star p$ y sus derivadas $\star p.name$, $p \rightarrow name$, etc. sólo pueden utilizarse en la situación (3).

Punteros

Punteros colgantes y **null**

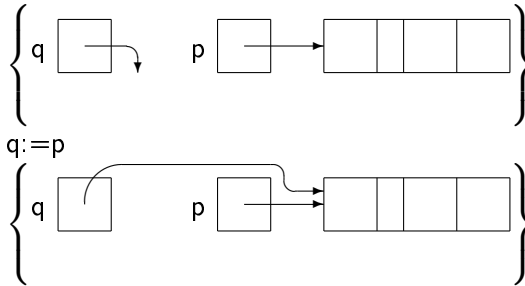
En la situación (2) el valor de `p` es inconsistente, no debe utilizarse ni accederse una dirección de memoria no reservada ya que no se sabe, a priori, qué hay en ella (en particular puede haber sido reservada para otro uso y al modificarlo se estaría corrompiendo información importante para tal uso). Los punteros que se encuentran en la situación (2) se llaman comúnmente referencias o punteros colgantes (dangling pointers).

En la situación (1) el valor de `p` es **null**, es decir que `p` no señala ninguna posición de memoria. Por ello, no tiene sentido intentar acceder a ella.

Punteros

Aliasing

Como vemos, los punteros permiten manejar explícitamente direcciones de memoria. Esto no es sencillo, aparecen situaciones que con los tipos de datos usuales no se daban. Por ejemplo:



Punteros

Aliasing

Como se ve, después de la asignación, q y p señalan a la misma tupla, por lo que cualquier modificación en campos de $\star q$ también modifican los de $\star p$ (claro, ya que son los mismos) y viceversa. Estamos en presencia de lo que se llama **aliasing**, es decir, hay 2 nombres distintos ($\star p$ y $\star q$) para el mismo objeto y al modificar uno se modifica el otro. Programar correctamente en presencia de aliasing es muy delicado y requiere gran atención.

Punteros

Administración de la memoria

Siempre hemos asumido que no es necesario ocuparse de reservar y liberar espacios de memoria para las variables. Los punteros como *p* y *q* son variables, así que **tampoco es necesario reservar y liberar espacio para ellos**. Pero las operaciones *alloc* y *free* son las responsables de reservar y liberar explícitamente espacio **para los objetos que *p* y *q* señalan**.

Esta posibilidad significa ciertas libertades: el programador puede decidir exactamente cuándo reservar espacio para una tupla. Por otro lado, significa también más responsabilidad: el programador es el que debe encargarse de liberar el espacio cuando deje de ser necesario.

Punteros

Administración de la memoria

Pero el verdadero beneficio de los punteros radica en que permiten una gran flexibilidad para representar estructuras complejas, y por lo tanto, para implementar diferentes tipos abstractos de datos que veremos en las próximas semanas.

Algoritmos y Estructuras de Datos II

Tipos Abstractos de Datos (TADs o ADTs en inglés)

Clase de hoy

- 1 Tipos abstractos de datos (TADs)
- 2 Especificación de TADs
- 3 Implementación de TADs
- 4 Listas
- 5 TAD Contador

Tipos abstractos de datos (TADs)

Tipos concretos.

- Son nativos del lenguaje.
- Tipos básicos: enteros, booleanos, char, etc.
- Tipos más complejos: arreglos, punteros, tuplas.
- En general conocemos cómo están implementados en el lenguaje.

Tipos abstractos.

- Se definen **especificando** constructores y operaciones.
- Podemos tener varias **implementaciones** para un mismo TAD.
- En general surgen de analizar un problema a resolver.
- El problema evidencia qué necesitamos representar y qué operaciones tener.

Especificación

Para especificar un TAD debemos:

- Indicar su nombre
- Especificar constructores: procedimientos o funciones mediante los cuales puedo crear elementos del tipo que estoy especificando.
- Especificar operaciones: todos los procedimientos o funciones que permitirán manipular los elementos del tipo de datos que estoy especificando.
- Indicamos los tipos de cada constructor y operación (el encabezado de los procedimientos o funciones), y mediante lenguaje natural explicamos qué hacen.
- Algunas operaciones pueden tener restricciones que las indicamos mediante **precondiciones**.
- Debemos especificar también una operación de *destrucción* que libera la memoria utilizada por los elementos del tipo, en caso que sea necesario.

Implementación

A partir de una especificación de un TAD, para implementarlo debemos:

- Definir un nuevo tipo con el nombre del TAD especificado. Para ello utilizamos tipos concretos y otros tipos definidos previamente.
- Implementar cada constructor respetando los tipos tal como fueron especificados.
- Implementar cada operación respetando los tipos tal como fueron especificados.
- Implementar operación de destrucción liberando memoria si es que se ha reservado al construir los elementos.
- Pueden surgir nuevas restricciones que dependen de cómo implementamos el tipo.
- Puedo necesitar operaciones auxiliares que no están especificadas en el tipo.

Listas

- Las listas permiten resolver una gran cantidad de problemas.
- Son colecciones de elementos de un mismo tipo, de tamaño variable.
- Toda lista o bien es vacía o bien tiene al menos un elemento al comienzo.
- Operaciones:
 - decidir si una lista es vacía
 - tomar el primer elemento
 - tirar el primer elemento
 - agregar un elemento al final
 - obtener la cantidad de elementos
 - concatenar dos listas
 - obtener el elemento en una posición específica
 - tomar una cantidad arbitraria de elementos
 - tirar una cantidad arbitraria de elementos
 - copiar una lista en una nueva

Especificación de Listas

spec List of T where

constructors

fun empty() **ret** l : List of T
{- crea una lista vacía. -}

proc addl (in e : T, in/out l : List of T)
{- agrega el elemento e al comienzo de la lista l. -}

destroy

proc destroy (in/out l : List of T)
{- Libera memoria en caso que sea necesario. -}

Especificación de Listas

operations

fun is_empty(*l* : List of T) **ret** *b* : bool
{- Devuelve True si *l* es vacía. -}

fun head(*l* : List of T) **ret** *e* : T
{- Devuelve el primer elemento de la lista *l* -}
{- PRE: not is_empty(*l*) -}

proc tail(*in/out l* : List of T)
{- Elimina el primer elemento de la lista *l* -}
{- PRE: not is_empty(*l*) -}

proc addr (*in/out l* : List of T, *in e* : T)
{- agrega el elemento *e* al final de la lista *l*. -}

fun length(*l* : List of T) **ret** *n* : nat
{- Devuelve la cantidad de elementos de la lista *l* -}

proc concat(*in/out l* : List of T, *in l0* : List of T)
{- Agrega al final de *l* todos los elementos de *l0*
en el mismo orden. -}

fun index(*l* : List of T, *n* : nat) **ret** *e* : T
{- Devuelve el *n*-ésimo elemento de la lista *l* -}
{- PRE: length(*l*) > *n* -}

proc take(*in/out l* : List of T, *in n* : nat)
{- Deja en *l* sólo los primeros *n*
elementos, eliminando el resto -}

proc drop(*in/out l* : List of T, *in n* : nat)
{- Elimina los primeros *n* elementos de *l* -}

fun copy_list(*l1* : List of T) **ret** *l2* : List of T
{- Copia todos los elementos de *l1* en la nueva lista *l2* -}

Especificación de Listas

- Para **usar** desde algún programa el tipo de las listas, alcanza con su especificación.
- Mediante sus **constructores** empty y addl pueden crearse listas vacías o agregar a una lista un elemento nuevo, respectivamente.
- Las **operaciones** permiten manipular las listas de acuerdo a la funcionalidad que el TAD provee.
- **No** es necesario conocer la implementación para poder **usar** el TAD.

Ejemplo de uso del TAD Lista

```
fun promedio (l : List of float) ret r : float
  var largo : nat
  var elem : float
  var laux : List of float

  laux := copy(l)
  r := 0.0
  largo := length(l)
  do (not is_empty(laux))
    elem := head(laux)
    r := r + elem
    tail(laux)
  od
  destroy(laux)
  r := r / largo
end proc
```

Implementación de Listas mediante punteros

- Implementaremos el TAD lista utilizando punteros, implementación conocida como **lista enlazada**.
- Cada elemento de la lista estará alojado en un *nodo* conteniendo además un puntero hacia el siguiente.
- Una lista será un puntero a un nodo.
- La lista vacía se implementa con el puntero **null**.
- Esta implementación permite tener la lista de elementos almacenada en lugares de la memoria no necesariamente contiguos.
- No existe límite teórico para almacenar elementos. En la práctica dicho límite será la cantidad de memoria.

Implementación de Listas mediante punteros

implement List **of** T **where**

type Node **of** T = **tuple**

elem : T

next : **pointer to** (Node **of** T)

end tuple

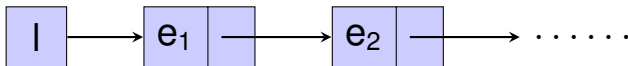
type List **of** T = **pointer to** (Node **of** T)

fun empty() **ret** l : List **of** T

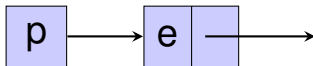
l := **null**

end fun

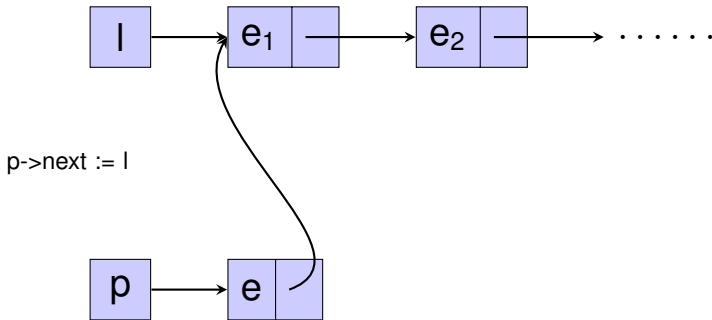
Implementación de Listas mediante punteros: addl



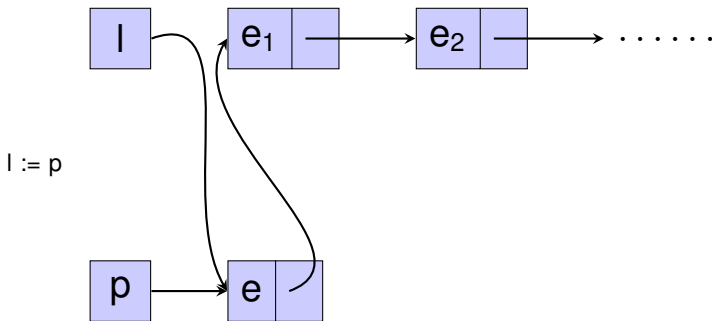
alloc(p)
p->elem := e



Implementación de Listas mediante punteros: addl



Implementación de Listas mediante punteros: addl



Implementación de Listas mediante punteros

```
proc addl (in e : T, in/out l : List of T)
  var p : pointer to (Node of T)
  alloc(p)
  p->elem := e
  p->next := l
  l := p
end proc
```


Implementación de Listas mediante punteros

```
fun is_empty(l : List of T) ret b : bool
```

```
  b := l = null
```

```
end fun
```

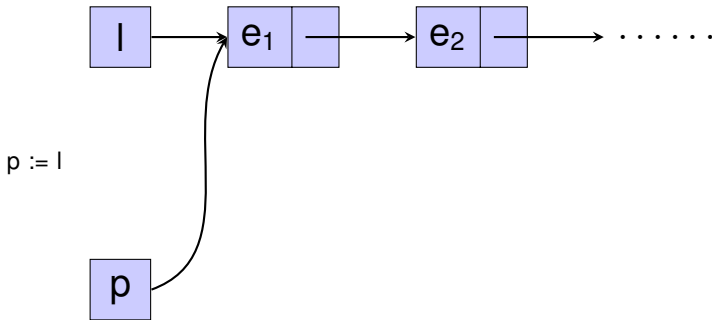
```
{- PRE: not is_empty(l) -}
```

```
fun head(l : List of T) ret e : T
```

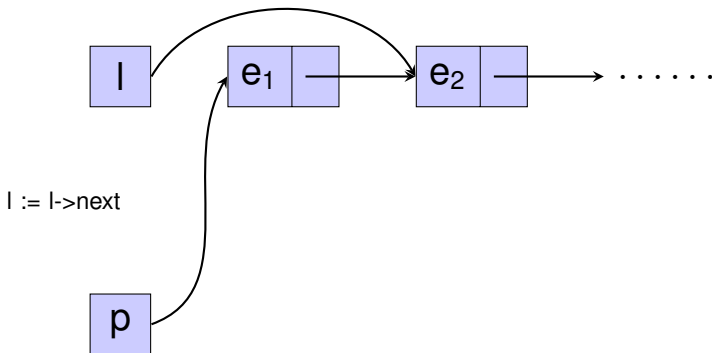
```
  e := l->elem
```

```
end fun
```

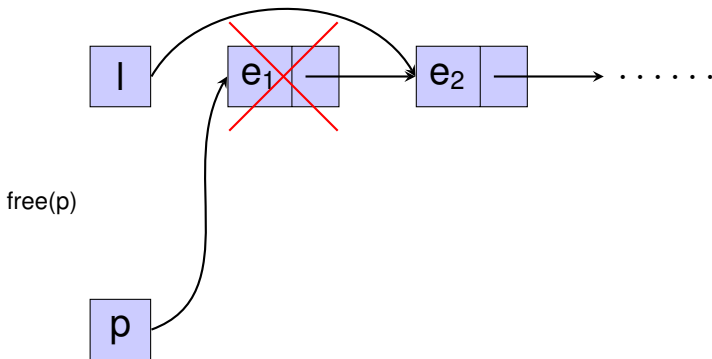
Implementación de Listas mediante punteros: tail



Implementación de Listas mediante punteros: tail



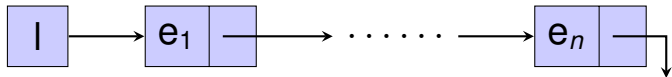
Implementación de Listas mediante punteros: tail



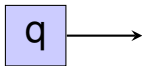
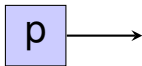
Implementación de Listas mediante punteros: tail

```
{- PRE: not is_empty(l) -}  
proc tail(in/out l : List of T)  
  var p : pointer to (Node of T)  
  p := l  
  l := l->next  
  free(p)  
end proc
```

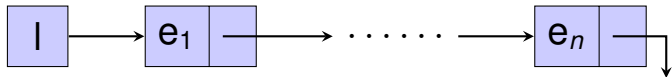
Implementación de Listas mediante punteros: addr



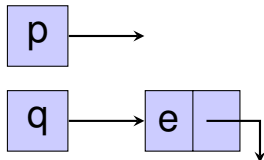
var p, q : **pointer to Node of T**



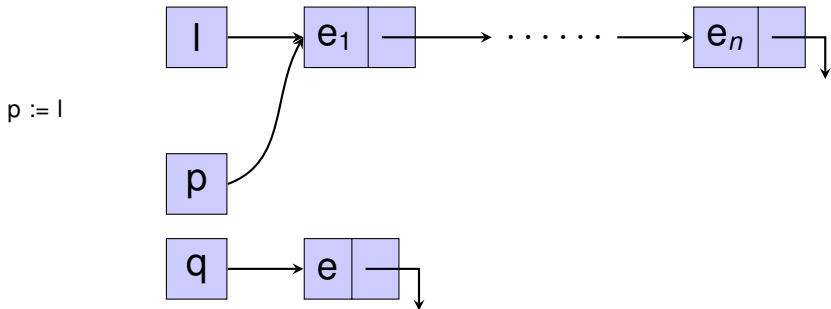
Implementación de Listas mediante punteros: addr



`alloc(q)`
`q->elem := e`
`q->next := null`

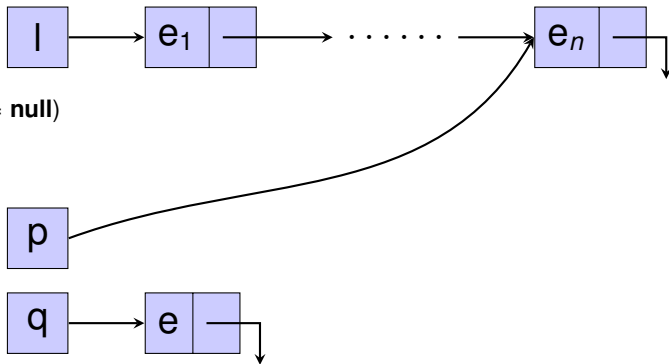


Implementación de Listas mediante punteros: addr

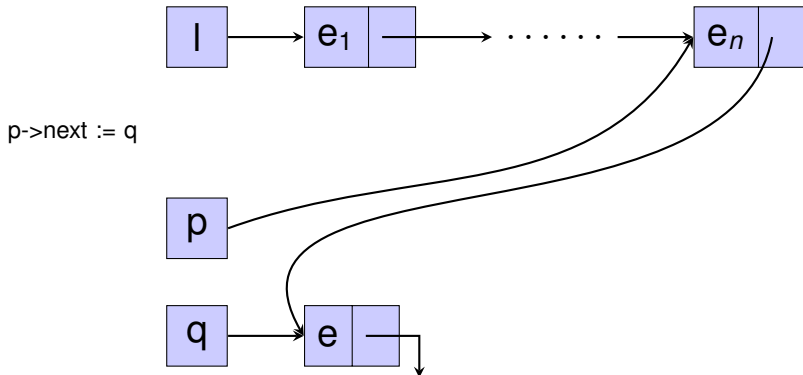


Implementación de Listas mediante punteros: addr

```
do (p->next != null)  
  p := p->next  
od
```



Implementación de Listas mediante punteros: addr



Implementación de Listas mediante punteros: addr

```
proc addr (in/out l : List of T, in e : T)
  var p,q : pointer to (Node of T)
  alloc(q)
  q->elem := e
  q->next := null
  if (not is_empty(l))
    then p := l
      do (p->next != null)
        p := p->next
      od
    p->next := q
  else l := q
fi
end proc
```

Implementación de Listas mediante punteros

```
fun length(l : List of T) ret n : nat  
  var p : pointer to (Node of T)  
  n := 0  
  p := l  
  do (p != null)  
    n := n+1  
    p := p->next  
  od  
end fun
```

Ejercicio: Implementar el resto de las operaciones.

Preguntas importantes: ¿cuándo necesito hacer **alloc** a un puntero?
¿cuándo necesito hacer **free**?

Paréntesis balanceados

- Problema:
 - Dar un algoritmo que tome una expresión,
 - dada, por ejemplo, por un arreglo de caracteres,
 - y devuelva verdadero si la expresión tiene sus paréntesis correctamente balanceados,
 - y falso en caso contrario.

Solución conocida

- Recorrer el arreglo de izquierda a derecha,
- utilizando un entero **inicializado en 0**,
- **incrementarlo** cada vez que se encuentra un paréntesis que abre,
- **decrementarlo** (comprobando previamente que no sea nulo en cuyo caso **no están balanceados**) cada vez que se encuentra un paréntesis que cierra,
- Al finalizar, **comprobar** que dicho entero sea cero.
- ¿Es necesario que sea un entero?

Contador

- No hace falta un entero (susceptible de numerosas operaciones aritméticas),
- sólo se necesita **algo** con lo que se pueda
 - inicializar
 - incrementar
 - comprobar si su valor es el inicial
 - decrementar si no lo es
- Llamaremos a ese **algo**, **contador**
- Necesitamos un contador.

TAD Contador

- El contador se define por lo que sabemos de él: sus cuatro operaciones
 - inicializar
 - incrementar
 - comprobar si su valor es el inicial
 - decrementar si no lo es
- Notamos que las operaciones **inicializar** e **incrementar** son capaces de generar todos los valores posibles del contador, por lo que serán nuestros **constructores**.
- **comprobar** en cambio solamente examina el contador,
- **decrementar** no genera más valores que los obtenibles por **inicializar** e **incrementar**

Especificación del TAD Contador

spec Counter **where**

constructors

fun init() **ret** c : Counter
{- *crea un contador inicial.* -}

proc incr (**in/out** c : Counter)
{- *incrementa el contador c.* -}

destroy

proc destroy (**in/out** c : Counter)
{- *Libera memoria en caso que sea necesario.* -}

Especificación del TAD Contador

operations

fun is_init(c : Counter) **ret** b : Bool
{- Devuelve True si el contador es inicial -}

proc decr (**in/out** c : Counter)
{- Decrementa el contador c. -}
{- **PRE:** not is_init(c) -}

Resolviendo el problema

- Queremos implementar un algoritmo que resuelve el problema de los paréntesis balanceados utilizando el TAD contador.
- La especificación nos da toda la información que necesitamos tener: constructores y operaciones con sus tipos.
- La idea es iniciar un contador y recorrer el arreglo de caracteres de izquierda a derecha.
- Si encontramos un paréntesis que abre, incrementamos el contador.
- Si encontramos un paréntesis que cierra lo decrementamos.
- Si el contador es inicial y encuentro paréntesis que cierra devuelvo False. Si termino de recorrer el arreglo y el contador no es inicial también doy False.

Algoritmo de control de paréntesis balanceados

```
fun matching_parenthesis (a: array[1..n] of char) ret b: bool  
  var i: nat  
  var c: Counter  
  b:= true  
  init(c)  
  i:= 1  
  do  $i \leq n \wedge b \rightarrow$  if a[i] = '('  $\rightarrow$  inc(c)  
    a[i] = ')'  $\wedge$  is_init(c)  $\rightarrow$  b:= false  
    a[i] = ')'  $\wedge \neg$ is_init(c)  $\rightarrow$  dec(c)  
    otherwise  $\rightarrow$  skip  
  fi  
  i:= i+1  
  
  od  
  b:= b  $\wedge$  is_init(c)  
  destroy(c)  
end fun
```

Implementación del TAD Contador

implement Counter **where**

type Counter = nat

proc init (**out** c: Counter)
 c:= 0
end proc

proc inc (**in/out** c: Counter)
 c:= c+1
end proc

fun is_init (c: Counter) **ret** b: bool
 b:= (c = 0)
end fun

{- **PRE:** not is_init(c) -}
proc dec (**in/out** c: Counter)
 c:= c-1
end proc

proc destroy (**in/out** c: Counter)
 skip
end proc

Todas las operaciones son $\mathcal{O}(1)$.

Algoritmos y Estructuras de Datos II

Tipos Abstractos de Datos: Pilas y Colas

Clase de hoy

- 1 Tipos abstractos de datos
- 2 TAD Pila
 - Generalización de paréntesis balanceados
 - Especificación del TAD Pila
 - Resolviendo el problema
- 3 TAD Cola
 - Problema del Buffer de datos
 - Cola
 - Especificación del TAD Cola
 - Resolviendo el problema

Tipos abstractos de datos (TADs)

Tipos abstractos.

- Separación entre **especificación** e **implementación**.
- Se definen especificando constructores y operaciones.
- Podemos tener varias implementaciones para un mismo tipo.
- En general surgen de analizar un problema a resolver.
- El problema evidencia qué necesitamos representar y qué operaciones tener.

TAD Contador

Paréntesis balanceados

- Problema:
 - Dar un algoritmo que tome una expresión,
 - dada, por ejemplo, por un arreglo de caracteres,
 - y devuelva verdadero si la expresión tiene sus paréntesis correctamente balanceados,
 - y falso en caso contrario.

TAD Contador

Solución conocida

- Recorrer el arreglo de izquierda a derecha y *contar* paréntesis que abren y cierran.
- Necesitamos *iniciar* el contador,
- *incrementarlo* cada vez que se encuentra un paréntesis que abre,
- *decrementarlo* (comprobando previamente que no sea nulo en cuyo caso **no están balanceados**) cada vez que se encuentra un paréntesis que cierra,
- Al finalizar, *comprobar* que la cuenta esté en el estado inicial.

TAD Contador

- El contador se define por lo que sabemos de él: sus cuatro operaciones
 - inicializar
 - incrementar
 - comprobar si su valor es el inicial
 - decrementar si no lo es
- Notamos que las operaciones **inicializar** e **incrementar** son capaces de generar todos los valores posibles del contador, por lo que serán nuestros **constructores**.
- **comprobar** en cambio solamente examina el contador,
- **decrementar** no genera más valores que los obtenibles por **inicializar** e **incrementar**

Generalización de paréntesis balanceados

- Problema:
 - Dar un algoritmo que tome una expresión,
 - dada, por ejemplo, por un arreglo de caracteres,
 - y devuelva verdadero si la expresión tiene sus paréntesis, corchetes, llaves, etc. correctamente balanceados,
 - y falso en caso contrario.

Usando contadores

- ¿Alcanza con un contador?
 - $(1+2)$
 - $\{1+(18-[4*2])\}$
 - $(1+2)$
- ¿Alcanza con tres (o n) contadores?
 - $(1+2)$
 - $(1+[3-1)+4]$

Conclusión

- No alcanza con saber cuántos delimitadores restan cerrar,
- también hay que saber en qué orden deben cerrarse,
- o lo que es igual
- en qué orden se han abierto,
- mejor dicho,
- ¿cuál fue el último que se abrió? (de los que aún no se han cerrado)
- ¿y antes de ése?
- etc.
- Hace falta una “constancia” de cuáles son los delimitadores que quedan abiertos, y en qué orden deben cerrarse.

Solución posible

- Recorrer el arreglo de izquierda a derecha,
- utilizando dicha “constancia” de delimitadores aún abiertos **inicialmente vacía**,
- **agregarle** obligación de cerrar un paréntesis (resp. corchete, llave) cada vez que se encuentra un paréntesis (resp. corchete, llave) que abre,
- **removerle** obligación de cerrar un paréntesis (resp. corchete, llave) (**comprobando** previamente que la constancia no sea vacía y que la **primera** obligación a cumplir sea justamente la de cerrar el paréntesis (resp. corchete, llave)) cada vez que se encuentra un paréntesis (resp. corchete, llave) que cierra,
- Al finalizar, **comprobar** que la constancia está vacía.

Pila

- Hace falta **algo**, una “constancia,” con lo que se pueda
 - inicializar vacía,
 - agregar una obligación de cerrar delimitador,
 - comprobar si quedan obligaciones,
 - examinar la primera obligación,
 - quitar una obligación.
- La última obligación que se agregó, es la primera que debe cumplirse y quitarse de la constancia.
- Esto se llama **pila**.

TAD Pila

- La pila se define por lo que sabemos: sus cinco operaciones
 - inicializar en vacía
 - apilar una nueva obligación (o elemento)
 - comprobar si está vacía
 - examinar la primera obligación (si no está vacía)
 - quitarla (si no está vacía).
- Nuevamente las operaciones **inicializar** y **agregar** son capaces de generar todas las pilas posibles,
- **comprobar** y **examinar**, en cambio, solamente examinan la pila,
- **quitarla** no genera más valores que los obtenibles por **inicializar** y **agregar**.

Especificación del TAD Pila

spec Stack of T where

constructors

fun empty_stack() **ret** s : Stack **of** T
{- crea una pila vacía. -}

proc push (in e : T, in/out s : Stack **of** T)
{- agrega el elemento e al tope de la pila s. -}

Especificación del TAD Pila

operations

fun is_empty_stack(s : Stack **of** T) **ret** b : Bool
{- *Devuelve True si la pila es vacía* -}

fun top(s : Stack **of** T) **ret** e : T
{- *Devuelve el elemento que se encuentra en el tope de s.* -}
{- **PRE:** not is_empty_stack(s) -}

proc pop (**in/out** s : Stack **of** T)
{- *Elimina el elemento que se encuentra en el tope de s.* -}
{- **PRE:** not is_empty_stack(s) -}

De aquí en adelante omitiremos escribir las operaciones de destrucción y copia que incluimos en todo TAD, pero las asumimos especificadas.

Algoritmo de control de delimitadores balanceados

```
fun matching_delimiters (a: array[1..n] of char) ret b: bool
  var i: nat
  var p: stack of char
  b := true
  p := empty_stack()
  i := 1
  do  $i \leq n \wedge b \rightarrow$  if left(a[i])  $\rightarrow$  push(match(a[i]),p)
    right(a[i])  $\wedge$  (is_empty(p)  $\vee$  top(p)  $\neq$  a[i])  $\rightarrow$  b := false
    right(a[i])  $\wedge$   $\neg$ is_empty(p)  $\wedge$  top(p) = a[i]  $\rightarrow$  pop(p)
    otherwise  $\rightarrow$  skip
  fi
  i := i+1
od
  b := b  $\wedge$  is_empty(p)
  destroy(p)
end fun
```

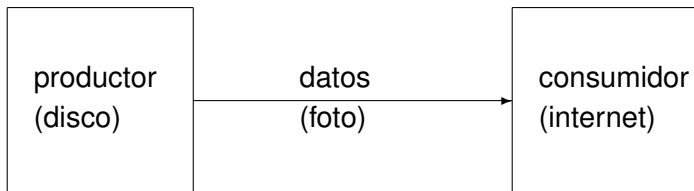
Este algoritmo asume, además de la implementación de pila,

- una función **match** tal que $\text{match}('(') = ')'$, $\text{match}('[') = ']'$, $\text{match}('{') = '}'$, etc.
- una función **left**, tal que $\text{left}('(')$, $\text{left}('[')$, $\text{left}('{')$, etc son verdadero, en los restantes casos **left** devuelve falso.
- una función **right**, tal que $\text{right}(')')$, $\text{right}(']')$, $\text{right}('}')'$, etc son verdadero, en los restantes casos, **right** devuelve falso.

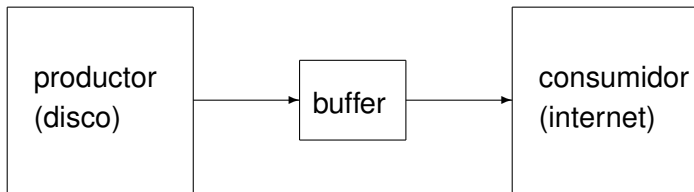
Buffer de datos

- Imaginemos cualquier situación en que ciertos datos deben transferirse desde una unidad a otra,
- por ejemplo, datos (¿una foto?) que se quiere subir a algún sitio de internet desde un disco,
- un agente suministra o produce datos (el disco) y otro que los utiliza o consume (el sitio de internet),
- esta relación se llama **productor-consumidor**
- para amortiguar el impacto por la diferencia de velocidades, se puede introducir un buffer entre ellos,
- un buffer recibe y almacena los datos a medida que se producen y los emite en el mismo orden, a medida que son solicitados.

Gráficamente



se interpone un buffer



Interés

- El programa que realiza la subida de datos puede liberar más rápidamente la lectora del disco.
- El proceso que realizaba la lectura se desocupa antes.
- El productor se ocupa de lo suyo.
- El consumidor se ocupa de lo suyo.
- El buffer se ocupa de la interacción entre ambos.

Uso del buffer

- La interposición del buffer no debe afectar el orden en que los datos llegan al consumidor.
- El propósito es sólo permitir que el productor y el consumidor puedan funcionar cada uno a su velocidad sin necesidad de sincronización.
- El buffer inicialmente está **vacío**.
- A medida que se van **agregando** datos suministrados por el productor, los mismos van siendo alojados en el buffer.
- Siempre que sea necesario enviar un dato al consumidor, habrá que comprobar que el buffer no se encuentre **vacío** en cuyo caso se enviará el **primero** que llegó al buffer y se lo **eliminará** del mismo.

Cola

- Es **algo**, con lo que se pueda
 - inicializar **vacía**,
 - agregar o **encolar** un dato,
 - comprobar si quedan datos en el buffer, es decir, si **es** o no **vacía**
 - examinar el **primer** dato (el más viejo de los que se encuentran en el buffer),
 - quitar o **decolar** un dato.
- El primer dato que se agregó, es el primero que debe enviarse y quitarse de la **cola**.
- Por eso se llama **cola** o también **cola FIFO** (First-In, First-Out).

Tad cola

- La cola se define por lo que sabemos: sus cinco operaciones
 - inicializar en **vacía**
 - **encolar** un nuevo dato (o elemento)
 - comprobar si **está vacía**
 - examinar el **primer** elemento (si no está vacía)
 - **decolarlo** (si no está vacía).
- Las operaciones **vacía** y **encolar** son capaces de generar todas las colas posibles,
- **está vacía** y **primero**, en cambio, solamente examinan la cola,
- **decolarla** no genera más valores que los obtenibles por **vacía** y **apilar**.

Especificación del TAD Cola

spec Queue **of** T **where**

constructors

fun empty_queue() **ret** q : Queue **of** T
{- crea una cola vacía. -}

proc enqueue (**in/out** q : Queue **of** T, **in** e : T)
{- agrega el elemento e al final de la cola q. -}

Especificación del TAD Cola

operations

fun is_empty_queue(q : Queue **of** T) **ret** b : Bool
{- Devuelve True si la cola es vacía -}

fun first(q : Queue **of** T) **ret** e : T
{- Devuelve el elemento que se encuentra al comienzo de q. -}
{- **PRE:** not is_empty_queue(q) -}

proc dequeue (in/out q : Queue **of** T)
{- Elimina el elemento que se encuentra al comienzo de q. -}
{- **PRE:** not is_empty_queue(q) -}

Algoritmo de transferencia de datos con buffer

```
proc buffer ()  
  var d: data  
  var q: queue of data  
  empty_queue(q)  
  do (not finish())  
    if there_is_product()  $\rightarrow$  d := get_product()  
                                enqueue(q,d)  
    there_is_demand()  $\wedge \neg$  is_empty(q)  $\rightarrow$  d:= first(q)  
                                                consume(d)  
  fi  
  od  
end proc
```

- Hemos asumido que hay varias funciones definidas:
- `finish()`. Devuelve `true` cuando el servicio de productor-consumidor haya finalizado.
- `there_is_product()` y `there_is_demand()` devuelven `si` hay producción o demanda respectivamente.
- `get_product()` obtiene un producto desde el productor, y `consume(d)` le envía al consumidor el producto `d`.

Utilización

- El TAD cola tiene numerosas aplicaciones.
- Siempre que se quieran atender pedidos, datos, etc. en el orden de llegada.
- Una aplicación interesante es el algoritmo de ordenación llamado Radix Sort.

Algoritmos y Estructuras de Datos II

Algoritmos voraces

Clase de hoy

1 Organización de la materia

2 Algoritmos voraces

- Forma general
- Problema de la moneda
- Problema de la mochila

Organización de la materia

- cómo vs. qué
- 3 partes
 - 1 análisis de algoritmos
 - 2 tipos de datos
 - 3 técnicas de resolución de problemas
 - divide y vencerás
 - **algoritmos voraces**
 - backtracking
 - programación dinámica
 - recorrida de grafos

Algoritmos Voraces (o Glotones, Golosos) (Greedy)

- Es la técnica más sencilla de resolución de problemas.
- Normalmente se trata de algoritmos que resuelven problemas de **optimización**, es decir, tenemos un problema que queremos resolver de manera **óptima**:
 - el camino más corto que une dos ciudades,
 - el valor máximo alcanzable entre ciertos objetos,
 - el costo mínimo para proveer un cierto servicio,
 - el menor número de billetes para pagar un cierto importe,
 - el menor tiempo necesario para realizar un trabajo, etc.
- Los algoritmos voraces intentan construir la solución óptima buscada **paso a paso**,
- **eligiendo** en cada paso
- la **componente** de la solución
- que **parece** más apropiada.

Características

- Nunca revisan una **elección** ya realizada,
- confían en haber elegido bien las componentes anteriores.
- Por ello, lamentablemente, no todos los problemas admiten solución voraz,
- pero varios problemas interesantes sí admiten solución voraz,
- y entonces, dichas soluciones resultan muy eficientes.

Problemas con solución voraz

- Problema de la moneda (casos especial).
- Problema de la mochila (caso especial).
- Problema del camino de costo mínimo en un grafo.
- Problema del árbol generador de costo mínimo en un grafo.
- Muchos otros problemas menos conocidos.

Ingredientes comunes de los algoritmos voraces

- se tiene un problema a resolver de manera **óptima**,
- un conjunto de **candidatos** a integrar la solución,
- los candidatos se van clasificando en 3: los aún no considerados, los **incorporados** a la solución parcial, y los **descartados**,
- tenemos una manera de saber si los candidatos ya incorporados completan una **solución** del problema (sin preocuparse por si la misma es o no óptima),
- una función que comprueba si un candidato o un conjunto de candidatos es **factible** de formar parte de la solución.
- finalmente, otra función que **selecciona** de entre los candidatos aún no considerados, el más promisorio.

Receta general de los algoritmos voraces

- Inicialmente ningún candidato ha sido considerado, es decir, ni incorporado ni descartado.
- En cada paso se utiliza la función de **selección** para elegir cuál candidato considerar.
- Se chequea que el candidato considerado sea **factible** para incorporarlo a la solución y se lo agrega o no.
- Se repiten los pasos anteriores hasta que la colección de candidatos elegidos sea una solución.

Esquema general de algoritmo voraz

```
fun voraz(C : Set of "Candidato") ret S : "Solución a construir"  
    S := "solución vacía"  
    do S "no es solución" → c := "seleccionar" de C  
        elim(C,c)  
        if "agregar c a S es factible" →  
            "agregar c a S"  
        fi  
    od  
end fun
```

Lo más importante es el criterio de selección.

Problema de la moneda

- Tenemos una cantidad infinita de monedas de cada una de las siguientes denominaciones:
 - 1 peso,
 - 50 centavos,
 - 25 centavos,
 - 10 centavos,
 - 5 centavos
 - y 1 centavo.
- Se desea pagar un cierto monto de manera exacta.
- Se debe determinar la manera de pagar dicho importe exacto con la menor cantidad de monedas posible.

Solución al problema de la moneda

- Seleccionar una moneda de la mayor denominación posible que no exceda el monto a pagar,
- utilizar exactamente el mismo algoritmo para el importe remanente.

Ya tenemos definido el **criterio de selección**.

Antes de implementar la solución, debemos pensar con qué **estructuras de datos** vamos a representar los elementos que intervienen en el problema. En este caso sencillo utilizamos naturales para cada denominación de moneda, un natural para el monto a pagar, y un natural para la solución que indica la cantidad de monedas necesarias para pagar.

Algoritmo voraz: versión 1

```
fun cambio(m: Nat, C: Set of Nat) ret S : Nat
  var c, resto: Nat
  var C_aux : Set of Nat
  S:= 0
  C_aux:= set_copy(C)
  resto:= m
  do resto > 0  $\rightarrow$ 
    c := "elemento máximo de C_aux"
    elim(C_aux,c)
    S := S + resto 'div' c
    resto := resto 'mod' c
  od
  set_destroy(C_aux)
end fun
```

Algoritmo voraz: versión 2

```
fun cambio(m: Nat, C: Set of Nat) ret S : Nat
  var c, resto: Nat
  var C_aux : Set of Nat
  S := 0
  C_aux := set_copy(C)
  resto := m
  do (not is_empty_set(C_aux))  $\rightarrow$ 
    c := "elemento máximo de C_aux" {selecciono}
    if c > resto {chequeo factibilidad}
      then elim(C_aux, c)
      else resto := resto - c
        S := S + 1 {agrego a la solución}
    fi
  od
  set_destroy(C_aux)
end fun
```

Algoritmo voraz: devolviendo valor de cada moneda

```
fun cambio(m: Nat, C: Set of Nat) ret S : List of Nat
  var c, resto: Nat
  var C_aux : Set of Nat
  S := empty_list()
  C_aux := set_copy(C)
  resto := m
  do (not is_empty_set(C_aux))  $\rightarrow$ 
    c := "elemento máximo de C_aux" {selecciono}
    if c > resto {chequeo factibilidad}
      then elim(C_aux, c)
      else resto := resto - c
        addl(S, c) {agrego a la solución}
    fi
  od
  set_destroy(C_aux)
end fun
```

Sobre el algoritmo

- La versión 1 tiene el problema de que para algunos conjuntos de denominaciones puede que el ciclo no termine nunca o bien que la ejecución falle al querer obtener el máximo de un conjunto vacío.
- En ambos casos resta por implementar la función que elige el máximo de un conjunto.
- El algoritmo no siempre obtiene la solución óptima, depende del conjunto de denominaciones.
- Para un conjunto razonable, funciona.
- Para denominaciones habituales: 100, 50, 25, 10, 5 y 1 funciona.

Conjunto de denominaciones para el que no funciona

- Sean 4, 3 y 1 las denominaciones y sea 6 el monto a pagar.
- El algoritmo voraz intenta pagar con una moneda de denominación 4, queda un saldo de 2 que solamente puede pagarse con 2 monedas de 1, en total, 3 monedas.
- Pero hay una solución mejor: dos monedas de 3.
- De todas formas, el algoritmo anda bien para todas las denominaciones de uso habitual.

Problema de la mochila

- Tenemos una mochila de capacidad W .
- Tenemos n objetos de valor $v_1, v_2, \dots, v_n$ y peso $w_1, w_2, \dots, w_n$.
- Se quiere encontrar la mejor selección de objetos para llevar en la mochila.
- Por mejor selección se entiende aquélla que totaliza el mayor valor posible sin que su peso exceda la capacidad W de la mochila.
- Para que el problema no sea trivial, asumimos que la suma de los pesos de los n objetos excede la capacidad de la mochila, obligándonos entonces a seleccionar cuáles cargar en ella.

Criterio de selección

¿Cómo conviene seleccionar un objeto para cargar en la mochila?

- El más valioso de todos.
- El menos pesado de todos.
- Una combinación de los dos.

Análisis del primer criterio de selección

El más valioso primero

- Razonabilidad: el objetivo es cargar la mochila con el mayor valor posible, escogemos los objetos más valiosos.
- Falla: puede que al elegir un objeto valioso dejemos de lado otro apenas menos valioso pero mucho más liviano.
- Ejemplo: Mochila de capacidad 10, objetos de valor 12, 11 y 9, y peso 7, 5 y 5.
- De elegir primero el de mayor valor (12) ocuparíamos 7 de los 10 kg de la mochila, no quedando lugar para otro objeto.
- En cambio, de elegir el de valor 11, ocuparíamos solamente 5 kg quedando 5 kg para el de valor 9, totalizando un valor de 20.

Análisis del segundo criterio de selección

El menos pesado primero

- Razonabilidad: hay que procurar aprovechar la capacidad de la mochila, escogemos los objetos más livianos.
- Falla: puede que al elegir un objeto liviano dejemos de lado otro apenas más pesado pero mucho más valioso.
- Ejemplo: Mochila de capacidad 13, objetos de valor 12, 11 y 7, y peso 6, 6 y 5.
- De elegir primero el de menor peso (5) obtendríamos su valor (7) más, en el mejor de los casos, 12, totalizando $12+7=19$.
- En cambio, de elegir los dos de peso 6, no se excede la capacidad de la mochila y se totaliza un valor de 23.

Análisis del tercer criterio de selección

Combinando ambos criterios

- Debemos asegurarnos de que cada kg utilizado de la mochila sea aprovechado de la mejor manera posible: que cada kg colocado en la mochila valga lo más posible.
- Criterio: elegir el de mayor valor relativo (cociente entre el valor y el peso): dicho cociente expresa el valor promedio de cada kg de ese objeto.
- Falla: puede que al elegir un objeto dejemos de lado otro de peor cociente, pero que aprovecha mejor la capacidad.
- Ejemplo: Mochila de capacidad 10, objetos de valor 12, 11 y 8, y peso 6, 5 y 4.
- El criterio elige al que pesa 5, ya que cada kg de ese objeto vale más de 2. Pero convenía elegir los otros dos.

Problema de la mochila

Versión simplificada

- El problema de la mochila no admite solución voraz.
- Se simplifica permitiendo **fraccionar** objetos.
- Ahora sí el tercer criterio funciona.
- (En el ejemplo anterior, elegimos primero el que vale 11 y luego $\frac{5}{6}$ del que vale 12 obteniendo como valor total $11 + 10 = 21$).

Ya tenemos definido criterio de selección. Antes de implementar el algoritmo definimos un tipo de datos para representar los objetos de la mochila.

Estructura de datos necesaria

```
type Objeto = tuple  
    id : Nat  
    value: Float  
    weight: Float  
end tuple
```

Resolveremos el problema asumiendo que los datos vienen dados por un conjunto $C : \text{Set of Objeto}$.

La solución la podemos representar por una lista donde cada elemento es un par conteniendo el objeto y la fracción que agrego de él:

```
type Obj_Mochila = tuple  
    obj : Objeto  
    fract : Float  
end tuple
```

Algoritmo voraz

```
fun mochila(W: Float, C: Set of Objeto) ret L : List of Obj_Mochila
  var o_m : Obj_Mochila   var resto : Float
  var C_aux : Set of Objeto
  S:= empty_list()
  C_aux:= set_copy(C)
  resto:= W
  do (resto > 0)  $\rightarrow$ 
    o_m.obj := select_obj(C_aux)
    if o_m.obj.weight <= resto
      then o_m.fract := 1.0
        resto := resto - o_m.obj.weight
      else o_m.fract := resto/o_m.obj.weight
        resto := 0
    fi
    addl(S,o_m)
    elim(C_aux,o_m.obj)
  od
  set_destroy(C_aux)
end fun
```


Algoritmo voraz

```
fun select_obj(C: Set of Objeto) ret r : Objeto
    var C_aux : Set of Objeto
    var o : Objeto
    var m : Float
    m := -∞
    C_aux := set_copy(C)
    do (not is_empty_set(C_aux)) →
        o := get(C_aux)
        if (o.value/o.weight > m)
            then m := o.value/o.weight
                r := o
        fi
        elim(C_aux,o)
    od
    set_destroy(C_aux)
end fun
```

Sobre este algoritmo

- funciona siempre que esté permitido fraccionar objetos.
- la función `select_obj` implementa el criterio de selección, eligiendo del conjunto de objetos, aquel que tiene la relación valor/peso máxima.
- La solución al problema puede implementarse también utilizando arreglos.

Algoritmos y Estructuras de Datos II

Algoritmos voraces sobre grafos

Clase de hoy

- 1 Árboles generadores de costo mínimo
 - Algoritmo de Prim
- 2 Camino de costo mínimo
 - Algoritmo de Dijkstra

Árbol generador de costo mínimo

- Sea $G = (V, A)$ un grafo conexo no dirigido con un costo no negativo asociado a cada arista.
- Se dice que $T \subseteq A$ es un árbol generador (intuitivamente, un tendido) si el grafo (V, T) es conexo y no contiene ciclos.
- Su costo es la suma de los costos de sus aristas.
- Se busca T tal que su costo sea mínimo.

Árbol generador de costo mínimo

- Sea $G = (V, A)$ un grafo conexo no dirigido con un costo no negativo asociado a cada arista.
- Se dice que $T \subseteq A$ es un árbol generador (intuitivamente, un tendido) si el grafo (V, T) es conexo y no contiene ciclos.
- Su costo es la suma de los costos de sus aristas.
- Se busca T tal que su costo sea mínimo.

Árbol generador de costo mínimo

- Sea $G = (V, A)$ un grafo conexo no dirigido con un costo no negativo asociado a cada arista.
- Se dice que $T \subseteq A$ es un árbol generador (intuitivamente, un tendido) si el grafo (V, T) es conexo y no contiene ciclos.
- Su costo es la suma de los costos de sus aristas.
- Se busca T tal que su costo sea mínimo.

Árbol generador de costo mínimo

- Sea $G = (V, A)$ un grafo conexo no dirigido con un costo no negativo asociado a cada arista.
- Se dice que $T \subseteq A$ es un árbol generador (intuitivamente, un tendido) si el grafo (V, T) es conexo y no contiene ciclos.
- Su costo es la suma de los costos de sus aristas.
- Se busca T tal que su costo sea mínimo.

Árbol generador de costo mínimo

- El problema de encontrar un árbol generador de costo mínimo tiene numerosas aplicaciones en la vida real.
- Cada vez que se quiera realizar un tendido (eléctrico, telefónico, etc) se quieren unir distintas localidades de modo que requiera el menor costo en instalaciones (por ejemplo, cables) posible.
- Se trata de realizar el tendido siguiendo la traza de un árbol generador de costo mínimo.

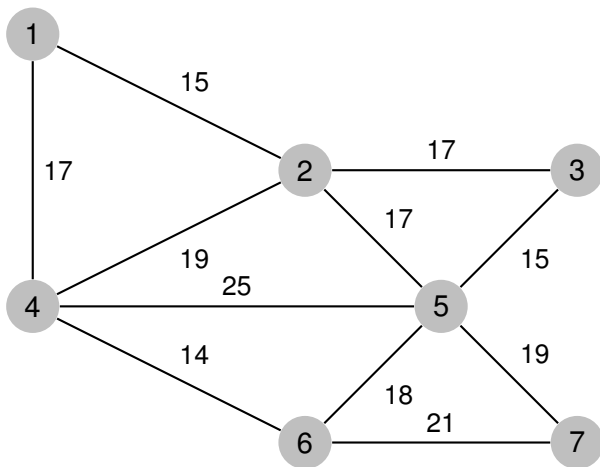
Árbol generador de costo mínimo

- El problema de encontrar un árbol generador de costo mínimo tiene numerosas aplicaciones en la vida real.
- Cada vez que se quiera realizar un tendido (eléctrico, telefónico, etc) se quieren unir distintas localidades de modo que requiera el menor costo en instalaciones (por ejemplo, cables) posible.
- Se trata de realizar el tendido siguiendo la traza de un árbol generador de costo mínimo.

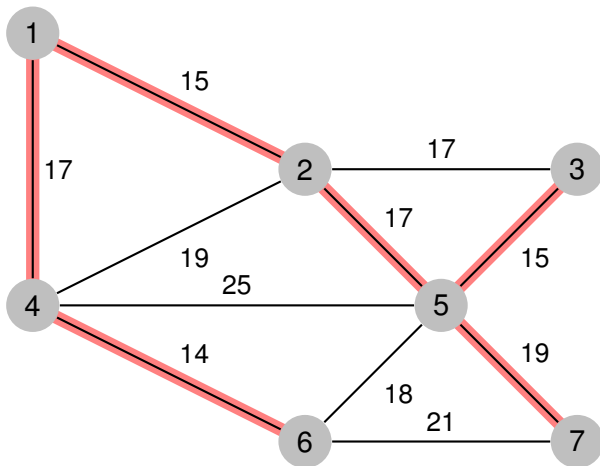
Árbol generador de costo mínimo

- El problema de encontrar un árbol generador de costo mínimo tiene numerosas aplicaciones en la vida real.
- Cada vez que se quiera realizar un tendido (eléctrico, telefónico, etc) se quieren unir distintas localidades de modo que requiera el menor costo en instalaciones (por ejemplo, cables) posible.
- Se trata de realizar el tendido siguiendo la traza de un árbol generador de costo mínimo.

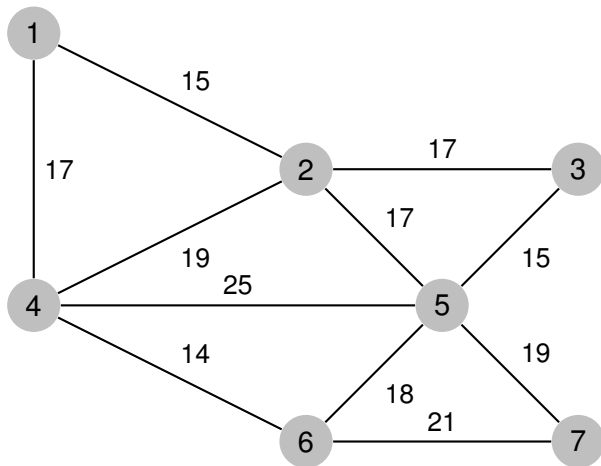
Ejemplo



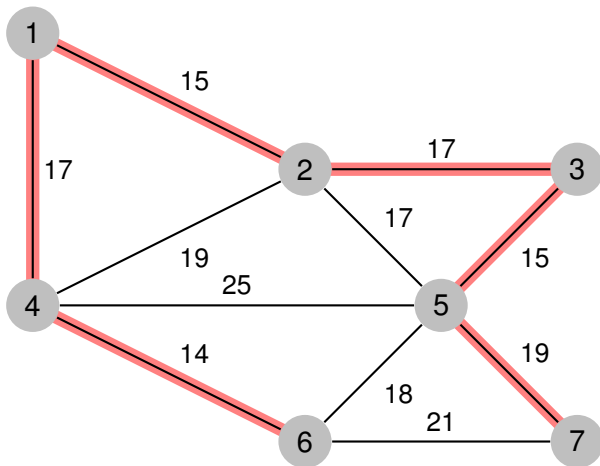
Ejemplo



Ejemplo



Ejemplo



Dos estrategias

Hay dos grandes ideas de cómo resolverlo:

- La de Prim: se parte desde un vértice origen y se va extendiendo el tendido a partir de ahí:
 - en cada paso se une el tendido ya existente con alguno de los vértices aún no alcanzados, seleccionando la arista de menor costo capaz de incorporar un nuevo vértice
- La de Kruskal: se divide el grafo en distintas componentes (originariamente una por cada vértice) y se van uniendo componentes,
 - en cada paso se selecciona la arista de menor costo capaz de unir componentes.

Este año no veremos en detalle el algoritmo de Kruskal.

Dos estrategias

Hay dos grandes ideas de cómo resolverlo:

- La de Prim: se parte desde un vértice origen y se va extendiendo el tendido a partir de ahí:
 - en cada paso se une el tendido ya existente con alguno de los vértices aún no alcanzados, seleccionando la arista de menor costo capaz de incorporar un nuevo vértice
- La de Kruskal: se divide el grafo en distintas componentes (originariamente una por cada vértice) y se van uniendo componentes,
 - en cada paso se selecciona la arista de menor costo capaz de unir componentes.

Este año no veremos en detalle el algoritmo de Kruskal.

Dos estrategias

Hay dos grandes ideas de cómo resolverlo:

- La de Prim: se parte desde un vértice origen y se va extendiendo el tendido a partir de ahí:
 - en cada paso se une el tendido ya existente con alguno de los vértices aún no alcanzados, seleccionando la arista de menor costo capaz de incorporar un nuevo vértice
- La de Kruskal: se divide el grafo en distintas componentes (originariamente una por cada vértice) y se van uniendo componentes,
 - en cada paso se selecciona la arista de menor costo capaz de unir componentes.

Este año no veremos en detalle el algoritmo de Kruskal.

Dos estrategias

Hay dos grandes ideas de cómo resolverlo:

- La de Prim: se parte desde un vértice origen y se va extendiendo el tendido a partir de ahí:
 - en cada paso se une el tendido ya existente con alguno de los vértices aún no alcanzados, seleccionando la arista de menor costo capaz de incorporar un nuevo vértice
- La de Kruskal: se divide el grafo en distintas componentes (originariamente una por cada vértice) y se van uniendo componentes,
 - en cada paso se selecciona la arista de menor costo capaz de unir componentes.

Este año no veremos en detalle el algoritmo de Kruskal.

Dos estrategias

Hay dos grandes ideas de cómo resolverlo:

- La de Prim: se parte desde un vértice origen y se va extendiendo el tendido a partir de ahí:
 - en cada paso se une el tendido ya existente con alguno de los vértices aún no alcanzados, seleccionando la arista de menor costo capaz de incorporar un nuevo vértice
- La de Kruskal: se divide el grafo en distintas componentes (originariamente una por cada vértice) y se van uniendo componentes,
 - en cada paso se selecciona la arista de menor costo capaz de unir componentes.

Este año no veremos en detalle el algoritmo de Kruskal.

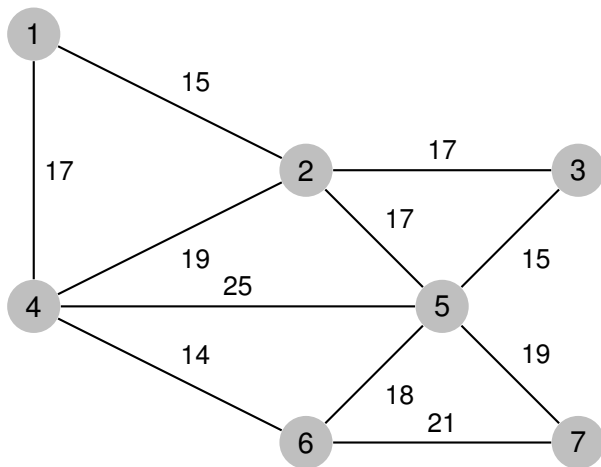
Dos estrategias

Hay dos grandes ideas de cómo resolverlo:

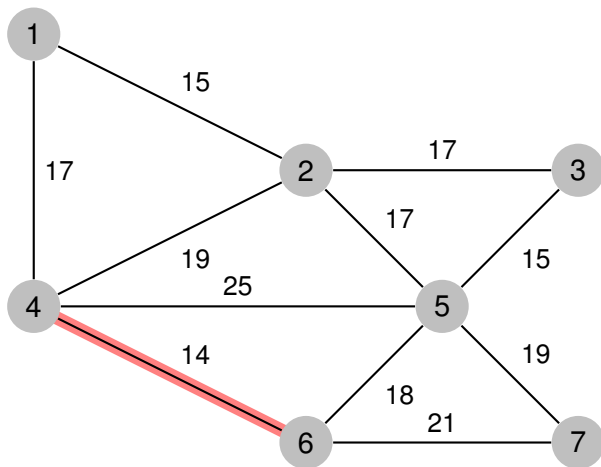
- La de Prim: se parte desde un vértice origen y se va extendiendo el tendido a partir de ahí:
 - en cada paso se une el tendido ya existente con alguno de los vértices aún no alcanzados, seleccionando la arista de menor costo capaz de incorporar un nuevo vértice
- La de Kruskal: se divide el grafo en distintas componentes (originariamente una por cada vértice) y se van uniendo componentes,
 - en cada paso se selecciona la arista de menor costo capaz de unir componentes.

Este año no veremos en detalle el algoritmo de Kruskal.

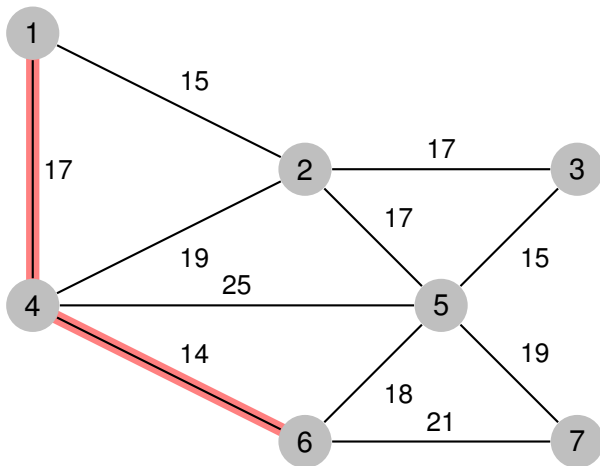
Algoritmo de Prim



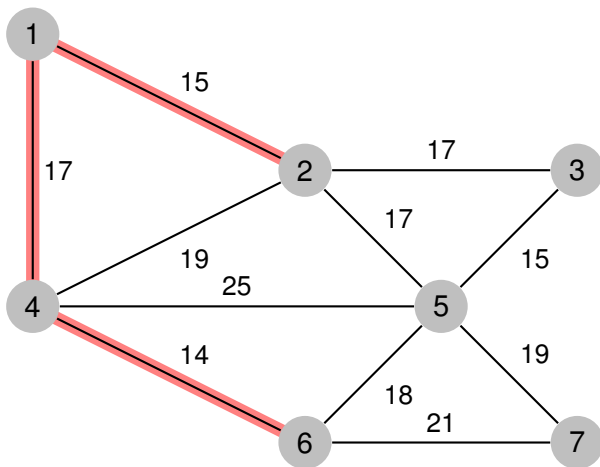
Algoritmo de Prim



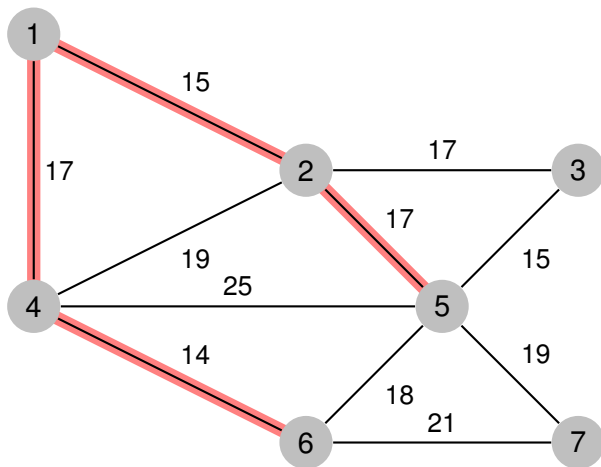
Algoritmo de Prim



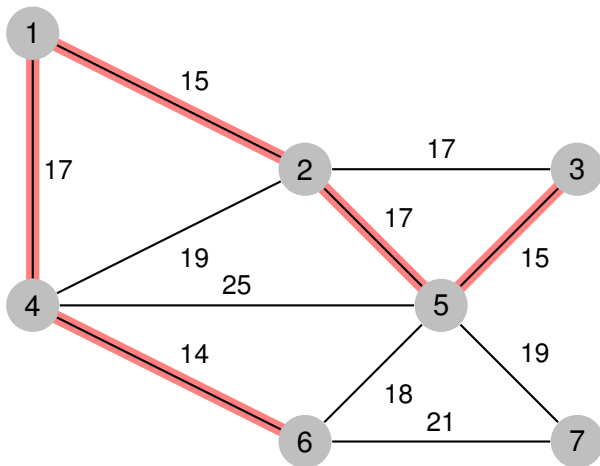
Algoritmo de Prim



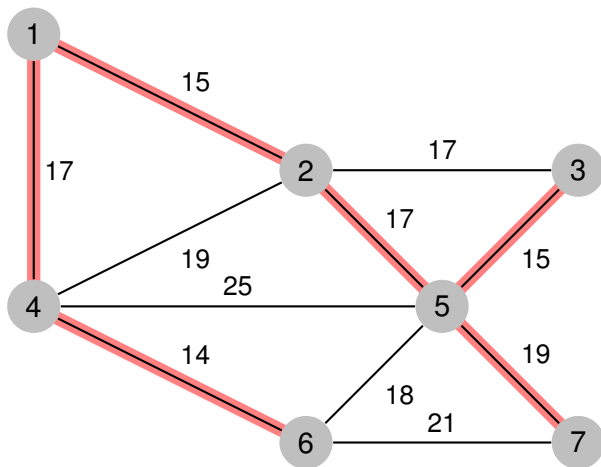
Algoritmo de Prim



Algoritmo de Prim



Algoritmo de Prim



Implementación del Algoritmo de Prim

Podemos representar los grafos como una tupla con dos conjuntos: uno para los vértices y otro para las aristas.

```
type Vertex = Nat
```

```
type Edge = tuple
```

```
    v1 : Vertex
```

```
    v2 : Vertex
```

```
    cost : Nat
```

```
end tuple
```

```
type Graph = tuple
```

```
    vertices : Set of Vertex
```

```
    edges : Set of Edge
```

```
end tuple
```

Implementación del Algoritmo de Prim

```
fun Prim(G : Graph, k: Vertex) ret T: Set of Edge
    var c: Edge
    var C: Set of Vertex
    C:= copy_set(G.vertices)
    elim(C,k)
    T:= empty_set()
    do (not is_empty_set(C))  $\rightarrow$ 
        c := "selecciono arista de costo mínimo tal que
              c.v1  $\in$  C y c.v2  $\notin$  C, ó c.v2  $\in$  C y c.v1  $\notin$  C"
        if member(c.v1,C)
            then elim(C,c.v1)
            else elim(C,c.v2)
        add(T,c)
    fi
od
end fun
```

Camino de costo mínimo

- Sea $G = (V, A)$ un grafo dirigido con costos no negativos en sus aristas, y sea $v \in V$ uno de sus vértices.
- Se busca obtener los caminos de menor costo desde v hacia cada uno de los demás vértices.

Camino de costo mínimo

- Sea $G = (V, A)$ un grafo dirigido con costos no negativos en sus aristas, y sea $v \in V$ uno de sus vértices.
- Se busca obtener los caminos de menor costo desde v hacia cada uno de los demás vértices.

Algoritmo de Dijkstra

Idea

- El algoritmo de Dijkstra realiza una secuencia de n pasos, donde n es el número de vértices.
- En cada paso, “aprende” el camino de menor costo desde v a un nuevo vértice.
- A ese nuevo vértice lo pinta de azul.
- Tras esos n pasos, conoce los costos de los caminos de menor costo a cada uno de los vértices.

Algoritmo de Dijkstra

Idea

- El algoritmo de Dijkstra realiza una secuencia de n pasos, donde n es el número de vértices.
- En cada paso, “aprende” el camino de menor costo desde v a un nuevo vértice.
- A ese nuevo vértice lo pinta de azul.
- Tras esos n pasos, conoce los costos de los caminos de menor costo a cada uno de los vértices.

Algoritmo de Dijkstra

Idea

- El algoritmo de Dijkstra realiza una secuencia de n pasos, donde n es el número de vértices.
- En cada paso, “aprende” el camino de menor costo desde v a un nuevo vértice.
- A ese nuevo vértice lo pinta de azul.
- Tras esos n pasos, conoce los costos de los caminos de menor costo a cada uno de los vértices.

Algoritmo de Dijkstra

Idea

- El algoritmo de Dijkstra realiza una secuencia de n pasos, donde n es el número de vértices.
- En cada paso, “aprende” el camino de menor costo desde v a un nuevo vértice.
- A ese nuevo vértice lo pinta de azul.
- Tras esos n pasos, conoce los costos de los caminos de menor costo a cada uno de los vértices.

Algoritmo de Dijkstra

Ejemplo

- Tratemos de entenderlo a través de un ejemplo.
- En cada paso, en los vértices azules anotamos el costo del camino de menor costo de v a ese vértice.
- En cada paso, en los vértices blancos anotamos el costo del camino azul de menor costo de v a ese vértice.
- Un camino azul es uno que a lo sumo tiene al vértice destino blanco, sus otros vértices son azules.

Algoritmo de Dijkstra

Ejemplo

- Tratemus de entenderlo a través de un ejemplo.
- En cada paso, en los vértices azules anotamos el costo del camino de menor costo de v a ese vértice.
- En cada paso, en los vértices blancos anotamos el costo del camino azul de menor costo de v a ese vértice.
- Un camino azul es uno que al menos tiene al vértice destino blanco, sus otros vértices son azules.

Algoritmo de Dijkstra

Ejemplo

- Tratemus de entenderlo a través de un ejemplo.
- En cada paso, en los vértices azules anotamos el costo del camino de menor costo de v a ese vértice.
- En cada paso, en los vértices blancos anotamos el costo del camino azul de menor costo de v a ese vértice.
- Un camino azul es uno que a lo sumo tiene al vértice destino blanco, sus otros vértices son azules.

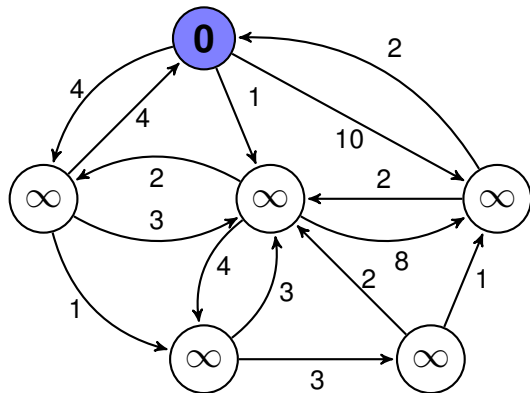
Algoritmo de Dijkstra

Ejemplo

- Tratemus de entenderlo a través de un ejemplo.
- En cada paso, en los vértices azules anotamos el costo del camino de menor costo de v a ese vértice.
- En cada paso, en los vértices blancos anotamos el costo del camino azul de menor costo de v a ese vértice.
- Un camino azul es uno que a lo sumo tiene al vértice destino blanco, sus otros vértices son azules.

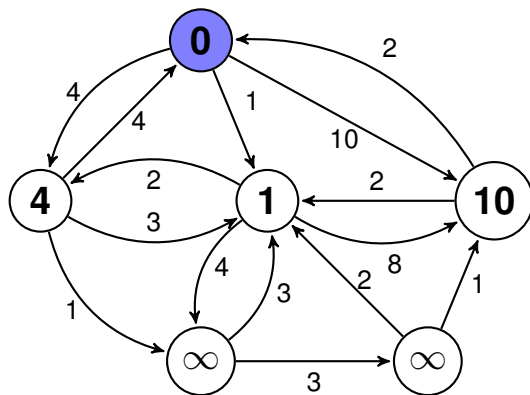
Algoritmo de Dijkstra

Paso 1 (a): sabemos lo que cuesta llegar de v a v



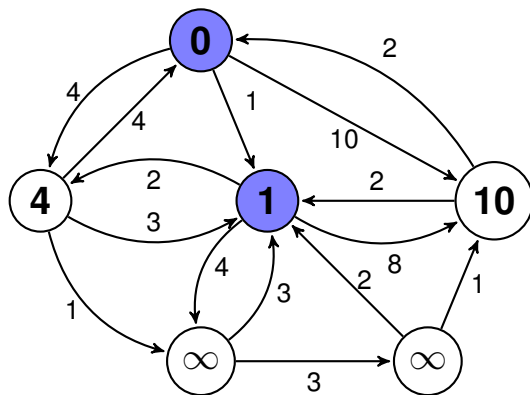
Algoritmo de Dijkstra

Paso 1 (b): Actualizamos los costos de los caminos azules óptimos



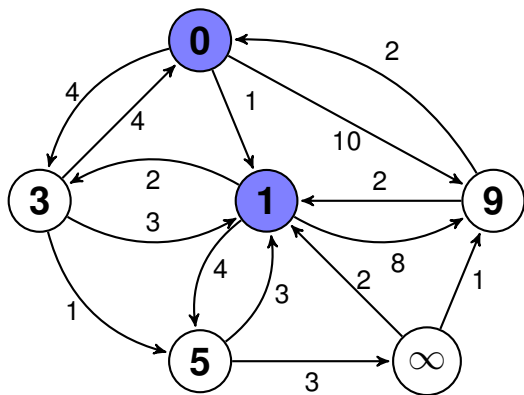
Algoritmo de Dijkstra

Paso 2 (a): sabemos lo que cuesta llegar de v a un nuevo vértice



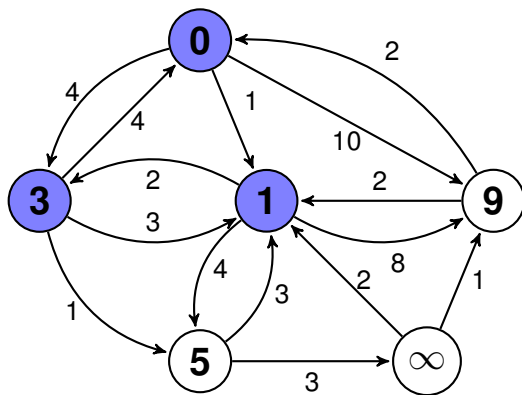
Algoritmo de Dijkstra

Paso 2 (b): Actualizamos los costos de los caminos azules óptimos



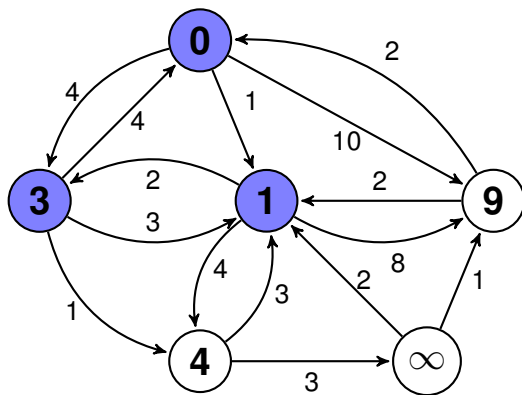
Algoritmo de Dijkstra

Paso 3 (a): sabemos lo que cuesta llegar de v a un nuevo vértice



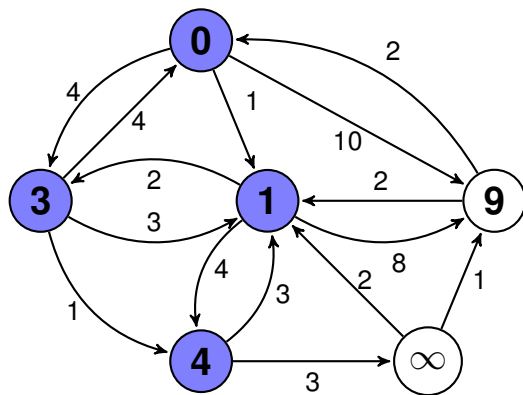
Algoritmo de Dijkstra

Paso 3 (b): Actualizamos los costos de los caminos azules óptimos



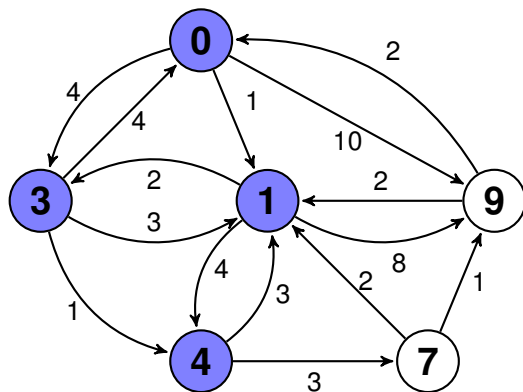
Algoritmo de Dijkstra

Paso 4 (a): sabemos lo que cuesta llegar de v a un nuevo vértice



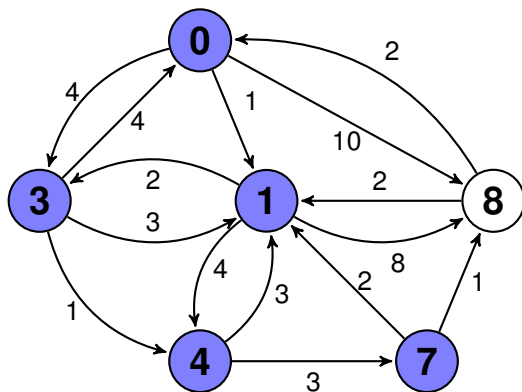
Algoritmo de Dijkstra

Paso 4 (b): Actualizamos los costos de los caminos azules óptimos



Algoritmo de Dijkstra

Paso 5 (b): Actualizamos los costos de los caminos azules óptimos



El algoritmo

- Asumiremos que el grafo viene dado por el conjunto de vértices $V = \{1, 2, \dots, n\}$
- y los costos por una matriz $L : \text{array}[1..n, 1..n] \text{ of Nat}$,
- que en $L[i, j]$ mantiene el costo de la arista que va de i a j .
- En caso de no haber ninguna arista de i a j , $L[i, j] = \infty$.
- Asumimos $L[j, j] = 0$.
- El algoritmo funciona también para grafos no dirigidos, simplemente se tiene $L[i, j] = L[j, i]$ para todo par de vértices i y j .

El algoritmo

- Asumiremos que el grafo viene dado por el conjunto de vértices $V = \{1, 2, \dots, n\}$
- y los costos por una matriz $L : \mathbf{array}[1..n, 1..n] \text{ of Nat}$,
 - que en $L[i, j]$ mantiene el costo de la arista que va de i a j .
 - En caso de no haber ninguna arista de i a j , $L[i, j] = \infty$.
 - Asumimos $L[j, j] = 0$.
- El algoritmo funciona también para grafos no dirigidos, simplemente se tiene $L[i, j] = L[j, i]$ para todo par de vértices i y j .

El algoritmo

- Asumiremos que el grafo viene dado por el conjunto de vértices $V = \{1, 2, \dots, n\}$
- y los costos por una matriz $L : \mathbf{array}[1..n, 1..n] \text{ of Nat}$,
- que en $L[i, j]$ mantiene el costo de la arista que va de i a j .
- En caso de no haber ninguna arista de i a j , $L[i, j] = \infty$.
- Asumimos $L[j, j] = 0$.
- El algoritmo funciona también para grafos no dirigidos, simplemente se tiene $L[i, j] = L[j, i]$ para todo par de vértices i y j .

El algoritmo

- Asumiremos que el grafo viene dado por el conjunto de vértices $V = \{1, 2, \dots, n\}$
- y los costos por una matriz $L : \mathbf{array}[1..n, 1..n] \text{ of Nat}$,
- que en $L[i, j]$ mantiene el costo de la arista que va de i a j .
- En caso de no haber ninguna arista de i a j , $L[i, j] = \infty$.
- Asumimos $L[j, j] = 0$.
- El algoritmo funciona también para grafos no dirigidos, simplemente se tiene $L[i, j] = L[j, i]$ para todo par de vértices i y j .

El algoritmo

- Asumiremos que el grafo viene dado por el conjunto de vértices $V = \{1, 2, \dots, n\}$
- y los costos por una matriz $L : \mathbf{array}[1..n, 1..n] \text{ of Nat}$,
- que en $L[i, j]$ mantiene el costo de la arista que va de i a j .
- En caso de no haber ninguna arista de i a j , $L[i, j] = \infty$.
- Asumimos $L[j, j] = 0$.
- El algoritmo funciona también para grafos no dirigidos, simplemente se tiene $L[i, j] = L[j, i]$ para todo par de vértices i y j .

El algoritmo

- Asumiremos que el grafo viene dado por el conjunto de vértices $V = \{1, 2, \dots, n\}$
- y los costos por una matriz $L : \mathbf{array}[1..n, 1..n] \text{ of Nat}$,
- que en $L[i, j]$ mantiene el costo de la arista que va de i a j .
- En caso de no haber ninguna arista de i a j , $L[i, j] = \infty$.
- Asumimos $L[j, j] = 0$.
- El algoritmo funciona también para grafos no dirigidos, simplemente se tiene $L[i, j] = L[j, i]$ para todo par de vértices i y j .

El algoritmo

- La versión que daremos del algoritmo, en vez de hallar el **camino de costo mínimo** desde v hasta cada uno de los demás, halla sólo el **costo** de dicho camino.
- Es decir, halla el **costo del camino de costo mínimo** desde v hasta cada uno de los demás.
- El resultado estará dado por un arreglo D : **array**[1.. n] of Nat,
- en $D[j]$ devolverá el costo del camino de costo mínimo que va de v a j .
- El conjunto C es el conjunto de los vértices hacia los que todavía desconocemos cuál es el camino de menor costo.

El algoritmo

- La versión que daremos del algoritmo, en vez de hallar el **camino de costo mínimo** desde v hasta cada uno de los demás, halla sólo el **costo** de dicho camino.
- Es decir, halla el **costo del camino de costo mínimo** desde v hasta cada uno de los demás.
- El resultado estará dado por un arreglo D : **array[1..n] of Nat**,
- en $D[j]$ devolverá el costo del camino de costo mínimo que va de v a j .
- El conjunto C es el conjunto de los vértices hacia los que todavía desconocemos cuál es el camino de menor costo.

El algoritmo

- La versión que daremos del algoritmo, en vez de hallar el **camino de costo mínimo** desde v hasta cada uno de los demás, halla sólo el **costo** de dicho camino.
- Es decir, halla el **costo del camino de costo mínimo** desde v hasta cada uno de los demás.
- El resultado estará dado por un arreglo D : **array**[1.. n] of Nat,
 - en $D[j]$ devolverá el costo del camino de costo mínimo que va de v a j .
 - El conjunto C es el conjunto de los vértices hacia los que todavía desconocemos cuál es el camino de menor costo.

El algoritmo

- La versión que daremos del algoritmo, en vez de hallar el **camino de costo mínimo** desde v hasta cada uno de los demás, halla sólo el **costo** de dicho camino.
- Es decir, halla el **costo del camino de costo mínimo** desde v hasta cada uno de los demás.
- El resultado estará dado por un arreglo D : **array**[1.. n] of Nat,
- en $D[j]$ devolverá el costo del camino de costo mínimo que va de v a j .
- El conjunto C es el conjunto de los vértices hacia los que todavía desconocemos cuál es el camino de menor costo.

El algoritmo

- La versión que daremos del algoritmo, en vez de hallar el **camino de costo mínimo** desde v hasta cada uno de los demás, halla sólo el **costo** de dicho camino.
- Es decir, halla el **costo del camino de costo mínimo** desde v hasta cada uno de los demás.
- El resultado estará dado por un arreglo D : **array**[1.. n] of Nat,
- en $D[j]$ devolverá el costo del camino de costo mínimo que va de v a j .
- El conjunto C es el conjunto de los vértices hacia los que todavía desconocemos cuál es el camino de menor costo.

Algoritmo de Dijkstra

```
fun Dijkstra(L: array[1..n,1..n] of Nat, v: Nat)
    ret D: array[1..n] of Nat

    var c: Nat
    var C: Set of Nat
    for i := 1 to n do add(C,i) od
    elim(C,v)
    for j:= 1 to n do D[j]:= L[v,j] od
    do (not is_empty_set(C))→
        c:= “elijo elemento c de C tal que D[c] sea mínimo”
        elim(C,c)
        for j in C do D[j]:= min(D[j],D[c]+L[c,j]) od
    od
end fun
```

Algoritmo de Dijkstra

- La implementación sigue el esquema de los algoritmos voraces.
- En cada paso, elijo el vértice c al que puedo llegar con menor costo.
- Luego actualizo el costo para llegar a cada uno de los demás vértices, habiendo “aprendido” el mejor camino para ir hasta c .
- Esta actualización se realiza calculando el mínimo entre lo que me costaba antes ir hasta cada vértice j , y lo que me cuesta si voy primero a c , y luego de ahí hasta j .

Modificando levemente la implementación dada, se puede obtener una versión que devuelve el camino hacia cada vértice, no solo su costo. No la veremos en detalle este año.

Algoritmo de Dijkstra

- La implementación sigue el esquema de los algoritmos voraces.
- En cada paso, elijo el vértice c al que puedo llegar con menor costo.
- Luego actualizo el costo para llegar a cada uno de los demás vértices, habiendo “aprendido” el mejor camino para ir hasta c .
- Esta actualización se realiza calculando el mínimo entre lo que me costaba antes ir hasta cada vértice j , y lo que me cuesta si voy primero a c , y luego de ahí hasta j .

Modificando levemente la implementación dada, se puede obtener una versión que devuelve el camino hacia cada vértice, no solo su costo. No la veremos en detalle este año.

Algoritmo de Dijkstra

- La implementación sigue el esquema de los algoritmos voraces.
- En cada paso, elijo el vértice c al que puedo llegar con menor costo.
- Luego actualizo el costo para llegar a cada uno de los demás vértices, habiendo “aprendido” el mejor camino para ir hasta c .
- Esta actualización se realiza calculando el mínimo entre lo que me costaba antes ir hasta cada vértice j , y lo que me cuesta si voy primero a c , y luego de ahí hasta j .

Modificando levemente la implementación dada, se puede obtener una versión que devuelve el camino hacia cada vértice, no solo su costo. No la veremos en detalle este año.

Algoritmo de Dijkstra

- La implementación sigue el esquema de los algoritmos voraces.
- En cada paso, elijo el vértice c al que puedo llegar con menor costo.
- Luego actualizo el costo para llegar a cada uno de los demás vértices, habiendo “aprendido” el mejor camino para ir hasta c .
- Esta actualización se realiza calculando el mínimo entre lo que me costaba antes ir hasta cada vértice j , y lo que me cuesta si voy primero a c , y luego de ahí hasta j .

Modificando levemente la implementación dada, se puede obtener una versión que devuelve el camino hacia cada vértice, no solo su costo. No la veremos en detalle este año.

Algoritmo de Dijkstra

- La implementación sigue el esquema de los algoritmos voraces.
- En cada paso, elijo el vértice c al que puedo llegar con menor costo.
- Luego actualizo el costo para llegar a cada uno de los demás vértices, habiendo “aprendido” el mejor camino para ir hasta c .
- Esta actualización se realiza calculando el mínimo entre lo que me costaba antes ir hasta cada vértice j , y lo que me cuesta si voy primero a c , y luego de ahí hasta j .

Modificando levemente la implementación dada, se puede obtener una versión que devuelve el camino hacia cada vértice, no solo su costo. No la veremos en detalle este año.

Algoritmo de Dijkstra

- La implementación sigue el esquema de los algoritmos voraces.
- En cada paso, elijo el vértice c al que puedo llegar con menor costo.
- Luego actualizo el costo para llegar a cada uno de los demás vértices, habiendo “aprendido” el mejor camino para ir hasta c .
- Esta actualización se realiza calculando el mínimo entre lo que me costaba antes ir hasta cada vértice j , y lo que me cuesta si voy primero a c , y luego de ahí hasta j .

Modificando levemente la implementación dada, se puede obtener una versión que devuelve el camino hacia cada vértice, no solo su costo. No la veremos en detalle este año.

Algoritmos y Estructuras de Datos II

Backtracking

Clase de hoy

- 1 Backtracking
 - Forma general de algoritmos voraces
 - ¿Y cuando no hay un buen criterio de selección?
 - Problema de la moneda
 - Problema de la mochila
 - Camino de costo mínimo entre todo par de vértices
- 2 Conclusiones
- 3 Backtracking, grafo implícito

Forma general de algoritmos voraces

```
fun voraz(C : Set of "Candidato") ret S : "Solución a construir"  
  S := "solución vacía"  
  do S "no es solución"  $\rightarrow$  c := "seleccionar" de C  
    elim(C,c)  
    if "agregar c a S es factible"  $\rightarrow$   
      "agregar c a S"  
    fi  
  od  
end fun
```

- Ser solución y ser factible no tienen en cuenta optimalidad.
- Optimalidad depende totalmente del criterio de selección.

¿Y cuando no hay un buen criterio de selección?

- A veces no hay un criterio de selección que garantice optimalidad.
- Por ejemplo:
 - Problema de la moneda para conjuntos de denominaciones arbitrarios.
 - Problema de la mochila para objetos no fraccionables.
- En este caso, si se elige un fragmento de solución puede ser necesario “volver hacia atrás” (**backtrack**) sobre esa elección e intentar otro fragmento.
- En la práctica, estamos hablando de considerar todas las selecciones posibles e intentar cada una de ellas para saber cuál de ellas conduce a la solución óptima.
(backtracking = fuerza bruta)

A diferencia de la técnica voraz

- Siempre que haya solución, backtracking la encuentra.
- En general son algoritmos ineficientes (aunque pueda que no se conozcan mejores alternativas).
- No hay buen criterio de selección: se utiliza fuerza bruta.
- A veces se puede ser un poco menos brutal...

Problema de la moneda

- Sean $d_1, d_2, \dots, d_n$ las denominaciones de las monedas (todas mayores que 0),
- no se asume que estén ordenadas,
- se dispone de una cantidad infinita de monedas de cada denominación,
- se desea pagar un monto k de manera exacta,
- utilizando el **menor número de monedas posibles**.
- Vimos que el algoritmo voraz puede no funcionar para ciertos conjuntos de denominaciones.
- Daremos un algoritmo recursivo consistente en considerar todas las combinaciones de monedas posibles.

Problema de la moneda

- Dado un conjunto con las n denominaciones, y un monto k a pagar,
- recursivamente iremos probando con cada denominación pagar el monto,
- si una denominación es factible (no se pasa del monto), pruebo usarla o no.
- Para ello calculo el resultado en cada caso, y luego obtengo el mínimo.
- El algoritmo devolverá la menor cantidad de monedas necesarias.

Problema de la moneda usando backtracking

```

fun cambio(j : Nat, C: Set of Nat) ret S : Nat
  var c, Nat
  var C_aux : Set of Nat
  if j = 0 then S := 0
  else if is_empty(C) then S :=  $\infty$ 
  else C_aux := set_copy(C)
    c := get(C)
    elim(C_aux, c)
    if (c  $\leq$  j) then S := min(1+cambio(j-c,C),cambio(j,C_aux))
      else S := cambio(j,C_aux)
    fi
    set_destroy(C_aux)
  fi
end fun

```

La solución la obtenemos llamando a cambio(k,C), donde C contiene los valores $d_1, d_2, \dots, d_n$.

Problema de la moneda

- El primer caso del if corresponde a cuando ya llegamos a pagar el total, en cuyo caso no necesito más monedas.
- El segundo caso es cuando he probado descartar tantas denominaciones que no puedo pagar el monto, por lo cual devolvemos infinito, ya que ese *intento* falló.
- En el tercer caso observo si la denominación es factible para pagar el monto j , en cuyo caso intento utilizarla o no, quedándome con el resultado mínimo.

Otra implementación

Observemos que no nos hace falta llevar un conjunto de denominaciones. Podría simplemente tomar un natural i que me indique que estoy considerando las denominaciones $d_1, d_2, \dots, d_i$, y el monto j que debo pagar:

```
fun cambio(d:array[1..n] of nat, i,j: nat) ret r: nat
  if j=0 then r:= 0
  else if i = 0 then r:=  $\infty$ 
  else if d[i] > j then r:= cambio(d,i-1,j)
  else r:= min(cambio(d,i-1,j),1+cambio(d,i,j-d[i]))
fi
end fun
```

La solución la obtenemos al llamar a $\text{cambio}(d,n,k)$, donde d es un arreglo conteniendo el valor de las denominaciones en cada posición.

Definición recursiva con otra notación

Como lo único interesante de estas soluciones con backtracking es la función recursiva, podemos definirla matemáticamente:

cambio(i, j) = “menor número de monedas necesarias para pagar exactamente el monto j con denominaciones $d_1, d_2, \dots, d_i$.”

$$cambio(i, j) = \begin{cases} 0 & j = 0 \\ \infty & j > 0 \wedge i = 0 \\ cambio(i-1, j) & d_i > j > 0 \wedge i > 0 \\ \min(cambio(i-1, j), 1 + cambio(i, j - d_i)) & j \geq d_i > 0 \wedge i > 0 \end{cases}$$

El resultado al problema lo obtengo llamando a *cambio(n, k)*.

Problema de la mochila

- Tenemos una mochila de capacidad W .
- Tenemos n objetos **no fraccionables** de valor $v_1, v_2, \dots, v_n$ y peso $w_1, w_2, \dots, w_n$.
- Se quiere encontrar la mejor selección de objetos para llevar en la mochila.
- Por mejor selección se entiende aquélla que totaliza **el mayor valor posible** sin que su peso exceda la capacidad W de la mochila.

Simplificación y generalización

- Simplificamos el problema:
 - sólo nos interesa por ahora hallar el mayor valor posible sin exceder la capacidad de la mochila,
 - no nos interesa saber cuáles son los objetos que alcanzan ese máximo.
- De manera similar al problema de la moneda, definimos una función recursiva $m(i, j)$, representando con el parámetro i que consideramos los objetos $1, 2, \dots, i$ y con j la capacidad restante de la mochila.

Problema de la mochila

Definimos $m(i, j) =$ “mayor valor alcanzable sin exceder la capacidad j con objetos $1, 2, \dots, i$.”

$$m(i, j) = \begin{cases} 0 & j = 0 \\ 0 & j > 0 \wedge i = 0 \\ m(i-1, j) & w_i > j > 0 \wedge i > 0 \\ \max(m(i-1, j), v_i + m(i-1, j - w_i)) & j \geq w_i > 0 \wedge i > 0 \end{cases}$$

El resultado al problema lo obtenemos llamando a $m(n, W)$.

Problema del camino de costo mínimo

Entre todo par de vértices

- Tenemos un grafo dirigido $G = (V, A)$,
- con costos no negativos en las aristas,
- se quiere encontrar, para cada par de vértices, el camino de menor costo que los une.
- Se asume $V = \{1, \dots, n\}$

Simplificación y generalización

- Simplificamos el problema:
 - sólo nos interesa por ahora hallar el costo de cada uno de los caminos de costo mínimo.
 - no nos interesa saber cuáles son los caminos que alcanzan ese mínimo.
- Generalizamos el problema:
 - Sean $1 \leq i, j \leq n$ y $0 \leq k \leq n$,
 - definimos $c(k, i, j)$ = “menor costo posible para caminos de i a j cuyos vértices intermedios se encuentran en el conjunto $\{1, \dots, k\}$.”
 - La solución del problema original se obtiene calculando $c(n, i, j)$ para el par i (origen) y j (destino) que se desea.

Definición recursiva de *camino*

$$c(k, i, j) = \begin{cases} L[i, j] & k = 0 \\ \min(c(k-1, i, j), c(k-1, i, k) + c(k-1, k, j)) & k \geq 1 \end{cases}$$

donde $L[i, j]$ es el costo de la arista que va de i a j , o infinito si no hay tal arista.

Conclusiones

En cada problema el algoritmo calcula cada resultado de acuerdo a la decisión de agregar un candidato a la solución. Y se queda con el mejor.

- En el problema de la moneda decido si utilizo o no utilizo la denominación i -ésima.
- En el problema de la mochila decido si utilizo o no el objeto i -ésimo.
- En el problema del camino de costo mínimo decido si paso o no por el vértice k .

Conclusiones

- Hemos visto soluciones a tres problemas.
- En general, muy ineficiente.
- Por ejemplo, para el problema de la moneda, si queremos pagar el monto 90 con denominaciones 1, 5 y 10,
 - cambio(3,90) llama a cambio(2,90) y cambio(3,80),
 - cambio(2,90) llama a cambio(1,90) y cambio(2,85),
 - cambio(2,85) llama a cambio(1,85) y **cambio(2,80)**,
 - cambio(3,80) llama a **cambio(2,80)** y cambio(3,70).
- Se ve que cambio(2,80) se calcula 2 veces.
- y muchos otros llamados se repiten, incluso varias veces.
- Los algoritmos son exponenciales.

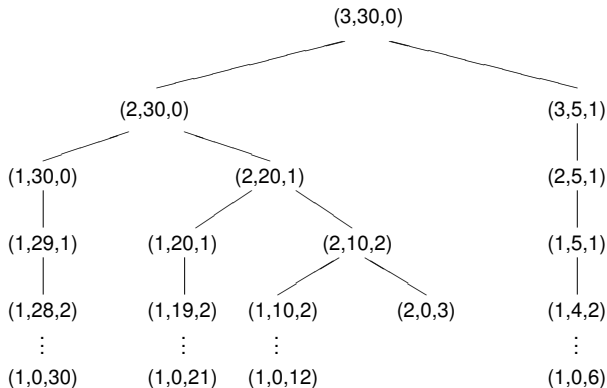
Problema de la moneda

$$cambio(i, j) = \begin{cases} 0 & j = 0 \\ \infty & j > 0 \wedge i = 0 \\ cambio(i-1, j) & d_i > j > 0 \wedge i > 0 \\ \min(cambio(i-1, j), 1 + cambio(i, j - d_i)) & j \geq d_i > 0 \wedge i > 0 \end{cases}$$

Podemos dibujar un árbol indicando en cada vértice la llamada a la función $cambio(i, j)$ indicando el resultado obtenido hasta ese momento. El vértice (i, j, x) indica que llamo a $cambio(i, j)$ y el resultado hasta ese momento es x .

Grafo implícito

Ejemplo $d_1 = 1$, $d_2 = 10$, $d_3 = 25$ y $k = 30$



Grafo implícito

Definición general

- Desde el vértice (i, j, x) , si $i, j > 0$ y $d_i < j$ existe una única arista al vértice $(i - 1, j, x)$.
- En cambio si $j \leq d_i$ existen dos aristas:
 - una a $(i - 1, j, x)$
 - y otra a $(i, j - d_i, x + 1)$.
- la raíz es el vértice $(n, k, 0)$.

Algoritmos y Estructuras de Datos II

Programación dinámica

Clase de hoy

1 Backtracking

2 Programación dinámica

- Problema de la moneda
- Problema de la mochila

Backtracking

Problema de la moneda

- Sean $0 \leq i \leq n$ y $0 \leq j \leq k$,
- definimos $cambio(i, j) =$ “menor número de monedas necesarias para pagar exactamente el monto j con denominaciones $d_1, d_2, \dots, d_i$.”



$$cambio(i, j) = \begin{cases} 0 & j = 0 \\ \infty & j > 0 \wedge i = 0 \\ cambio(i-1, j) & d_i > j > 0 \wedge i > 0 \\ \min(cambio(i-1, j), 1 + cambio(i, j - d_i)) & j \geq d_i > 0 \wedge i > 0 \end{cases}$$

- Es exponencial.

Backtracking

Problema de la mochila

- Sean $0 \leq i \leq n$ y $0 \leq j \leq W$,
- definimos $mochila(i, j) =$ “mayor valor alcanzable sin exceder la capacidad j con objetos $1, 2, \dots, i$.”



$$mochila(i, j) = \begin{cases} 0 & j = 0 \\ 0 & j > 0 \wedge i = 0 \\ mochila(i-1, j) & w_i > j > 0 \wedge i > 0 \\ \max(mochila(i-1, j), v_i + mochila(i-1, j-w_i)) & j \geq w_i > 0 \wedge i > 0 \end{cases}$$

- Es exponencial.

Backtracking

Problema de los caminos de costo mínimo entre cada par de vértices

- Sean $1 \leq i, j \leq n$ y $0 \leq k \leq n$,
- definimos $\text{camino}(k, i, j)$ = “menor costo posible para caminos de i a j cuyos vértices intermedios se encuentran en el conjunto $\{1, \dots, k\}$.”



$$\text{camino}(k, i, j) = \begin{cases} L[i, j] \\ \min(\text{camino}(k-1, i, j), \text{camino}(k-1, i, k) + \text{camino}(k-1, k, j)) \end{cases}$$

- Es exponencial (3^n).

Programación dinámica

- Método para transformar una definición recursiva en iterativa
- a través de la confección de una **tabla de valores**.
- Objetivo: evitar la reiteración de cálculos.
- Ejemplo: definición recursiva de la secuencia de Fibonacci.

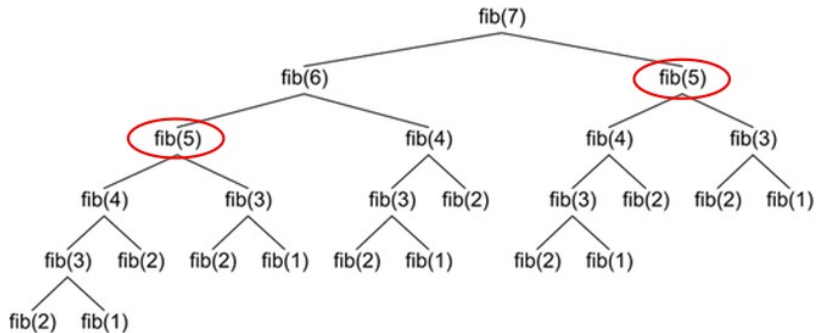
Secuencia de Fibonacci



$$f_n = \begin{cases} n & n \leq 1 \\ f_{n-1} + f_{n-2} & n > 1 \end{cases}$$

- Esta función recursiva es exponencial.
- La razón, el cálculo de f_n lleva a calcular
 - 2 veces f_{n-2} ,
 - 3 veces f_{n-3} ,
 - 5 veces f_{n-4} ,
 - etc.

Secuencia de Fibonacci



¿Cómo podemos evitar tantos recálculos?

- Llevando una tabla de valores calculados.
- Comenzando desde los casos bases.
- Sea f un arreglo de 0 a n .
 - $f[0] := 0$
 - $f[1] := 1$
 - $f[2] := f[1] + f[0]$
 - $f[3] := f[2] + f[1]$
 - etc

Fibonacci a través de una tabla

```
{PRE:  $n > 1$  }  
fun fib(n: nat) ret r: nat  
    var f: array[0..n] of nat  
    f[0]:= 0  
    f[1]:= 1  
    for i:= 2 to n do f[i]:= f[i-1] + f[i-2] od  
    r:= f[n]  
end fun
```

¡Este algoritmo es lineal!

Problema de la moneda

Backtracking

Vimos la definición

$$\text{cambio}(i, j) = \begin{cases} 0 & j = 0 \\ \infty & j > 0 \wedge i = 0 \\ \text{cambio}(i - 1, j) & d_i > j > 0 \wedge i > 0 \\ \min(\text{cambio}(i - 1, j), 1 + \text{cambio}(i, j - d_i)) & j \geq d_i > 0 \wedge i > 0 \end{cases}$$

que puede ser exponencial debido a que tiene dos llamadas recursivas en el último caso.

Problema de la moneda

Confección de una tabla

- Habiendo dos parámetros, la tabla será una matriz en vez de un vector como en el caso de Fibonacci.
- Los casos base corresponden al llenado de la primera columna y primera fila de la matriz.
- Como todas las llamadas recursivas se realizan decrementando el “parámetro i ” o manteniendolo igual pero en ese caso decrementando el “parámetro j ”, se propone el siguiente método de llenado de la matriz:
 - fila por fila, desde la primera a la última, de modo de que el valor correspondiente a $cambio(i - 1, j)$ ya esté computado al calcular el valor correspondiente a $cambio(i, j)$
 - dentro de cada fila, desde la primer columna hasta la última, de modo de que el valor correspondiente a $cambio(i, j - d_i)$ ya esté computado al calcular $cambio(i, j)$

Problema de la moneda

Programación dinámica

```
fun cambio(d:array[1..n] of nat, k: nat) ret r: nat
  var cam: array[0..n,0..k] of nat
  for i:= 0 to n do cam[i,0]:= 0 od
  for j:= 1 to k do cam[0,j]:=  $\infty$  od
  for i:= 1 to n do
    for j:= 1 to k do
      if d[i] > j then cam[i,j]:= cam[i-1,j]
      else cam[i,j]:= min(cam[i-1,j], 1+cam[i,j-d[i]])
      fi
    od
  od
  r:= cam[n,k]
end fun
```

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0																	
1																	
2																	
3																	

for $i := 0$ **to** n **do** $\text{cam}[i,0] := 0$ **od**

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0																
1	0																
2	0																
3	0																

for $i := 0$ **to** n **do** $\text{cam}[i,0] := 0$ **od**

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0																
2	0																
3	0																

for $j := 1$ **to** k **do** $\text{cam}[0,j] := \infty$ **od**

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞													
2	0																
3	0																

i = 1

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1 + cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞													
2	0																
3	0																

i = 1

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $\text{cam}[i,j] := \text{cam}[i-1,j]$

else $\text{cam}[i,j] := \min(\text{cam}[i-1,j], 1 + \text{cam}[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1												
2	0																
3	0																

i = 1

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $\text{cam}[i,j] := \text{cam}[i-1,j]$

else $\text{cam}[i,j] := \min(\text{cam}[i-1,j], 1 + \text{cam}[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞											
2	0																
3	0																

i = 1

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1+cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞										
2	0																
3	0																

i = 1

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1+cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞									
2	0																
3	0																

i = 1

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1 + cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2								
2	0																
3	0																

i = 1

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1+cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0																
3	0																

i = 1

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1+cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞															
3	0																

i = 2

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1 + cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞															
3	0																

i = 2

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1+cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞	1														
3	0																

i = 2

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1+cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞	1	∞	1	∞	2	∞	2	∞	3	∞	3	∞	4	∞	4
3	0																

i = 2

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1 + cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞	1	∞	1	∞	2	∞	2	∞	3	∞	3	∞	4	∞	4
3	0	∞	1	∞	1	∞	2										

i = 3

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1 + cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞	1	∞	1	∞	2	∞	2	∞	3	∞	3	∞	4	∞	4
3	0	∞	1	∞	1	∞	2										

i = 3

for i:= 1 to n do

for j:= 1 to k do

if d[i] > j then cam[i,j]:= cam[i-1,j]

else cam[i,j]:= min(cam[i-1,j], 1+cam[i,j-d[i]])

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞	1	∞	1	∞	2	∞	2	∞	3	∞	3	∞	4	∞	4
3	0	∞	1	∞	1	∞	2	1									

i = 3

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1+cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞	1	∞	1	∞	2	∞	2	∞	3	∞	3	∞	4	∞	4
3	0	∞	1	∞	1	∞	2	1	2								

i = 3

for i:= 1 to n do

for j:= 1 to k do

if d[i] > j then cam[i,j]:= cam[i-1,j]

else cam[i,j]:= min(cam[i-1,j], 1+cam[i,j-d[i]])

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞	1	∞	1	∞	2	∞	2	∞	3	∞	3	∞	4	∞	4
3	0	∞	1	∞	1	∞	2	1	2	2							

i = 3

for i:= 1 to n do

for j:= 1 to k do

if $d[i] > j$ **then** $cam[i,j] := cam[i-1,j]$

else $cam[i,j] := \min(cam[i-1,j], 1+cam[i,j-d[i]])$

fi

od

od

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞	1	∞	1	∞	2	∞	2	∞	3	∞	3	∞	4	∞	4
3	0	∞	1	∞	1	∞	2	1	2	2	3	2	3	3			

```

for i:= 1 to n do
  for j:= 1 to k do
    if d[i] > j then cam[i,j]:= cam[i-1,j]
    else cam[i,j]:= min(cam[i-1,j], 1+cam[i,j-d[i]])
  fi
od
od

```

Problema de la moneda

Ejemplo con denominaciones $d_1 = 4$, $d_2 = 2$ y $d_3 = 7$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	1	∞	∞	∞	2	∞	∞	∞	3	∞	∞	∞	4
2	0	∞	1	∞	1	∞	2	∞	2	∞	3	∞	3	∞	4	∞	4
3	0	∞	1	∞	1	∞	2	1	2	2	3	2	3	3	2	3	3

```

for i:= 1 to n do
  for j:= 1 to k do
    if d[i] > j then cam[i,j]:= cam[i-1,j]
    else cam[i,j]:= min(cam[i-1,j], 1+cam[i,j-d[i]])
  fi
od
od

```

Problema de la mochila

Backtracking

Vimos la definición

$$mochila(i, j) = \begin{cases} 0 & j = 0 \\ 0 & j > 0 \wedge i = 0 \\ mochila(i-1, j) & w_i > j > 0 \wedge i > 0 \\ \max(mochila(i-1, j), v_i + mochila(i-1, j-w_i)) & j \geq w_i > 0 \wedge i > 0 \end{cases}$$

que puede ser exponencial debido a que tiene dos llamadas recursivas en el último caso.

Problema de la mochila

Confección de una tabla

- Habiendo dos parámetros, la tabla será nuevamente una matriz.
- Los casos base corresponden al llenado de la primera columna y primera fila de la matriz.
- Como todas las llamadas recursivas se realizan decrementando el “parámetro i ”, la única condición necesaria para el llenado de la tabla es proceder fila por fila, no importa el orden de llenado dentro de cada fila.

Problema de la mochila

Programación dinámica

```
fun mochila(v:array[1..n] of valor, w:array[1..n] of nat, W: nat)
                                                    ret r: valor

  var moch: array[0..n,0..W] of valor
  for i:= 0 to n do moch[i,0]:= 0 od
  for j:= 1 to W do moch[0,j]:= 0 od
  for i:= 1 to n do
    for j:= 1 to W do
      if w[i] > j then moch[i,j]:= moch[i-1,j]
      else moch[i,j]:= max(moch[i-1,j],v[i]+moch[i-1,j-w[i]])
      fi
    od
  od
  r:= moch[n,W]
end fun
```


Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0																	
1																	
2																	
3																	
4																	

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0																
2	0																
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0									
2	0																
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3								
2	0																
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3							
2	0																
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0																
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0												
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2									
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3								
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3				
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5			
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0																
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0	0	0	0	0	2	2										
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0	0	0	0	0	2	2	3									
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	0	2	2	2	3	3	3	3	5	5	5	5
3	0	0	0	0	0	0	2	2	3	3	3	3	3				
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0	0	0	0	0	2	2	3	3	3	3	3	5	5	5		
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0	0	0	0	0	2	2	3	3	3	3	3	5	5	5	6	
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0	0	0	0	0	2	2	3	3	3	3	3	5	5	5	6	6
4	0																

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0	0	0	0	0	2	2	3	3	3	3	3	5	5	5	6	6
4	0	0	0														

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0	0	0	0	0	2	2	3	3	3	3	3	5	5	5	6	6
4	0	0	0	2	2	2	2										

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0	0	0	0	0	2	2	3	3	3	3	3	5	5	5	6	6
4	0	0	0	2	2	2	2	3	4	4							

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0	0	0	0	0	2	2	3	3	3	3	3	5	5	5	6	6
4	0	0	0	2	2	2	2	3	4	4	5	5	5	5	5		

Problema de la mochila

Ejemplo con valores $v_1 = 3$, $v_2 = 2$, $v_3 = 3$, $v_4 = 2$, $w_1 = 8$, $w_2 = 5$, $w_3 = 7$ y $w_4 = 3$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	2	2	2	3	3	3	3	3	5	5	5	5
3	0	0	0	0	0	2	2	3	3	3	3	3	5	5	5	6	6
4	0	0	0	2	2	2	2	3	4	4	5	5	5	5	5	7	7

Algoritmos y Estructuras de Datos II

Recorriendo grafos

Clase de hoy

1 Repaso

2 Recorrida de grafos

- Generalidades
- Árboles binarios
- Árboles finitarios
- Grafos arbitrarios, DFS
- Grafos arbitrarios, BFS

Repaso

- cómo vs. qué
- 3 partes
 - 1 análisis de algoritmos
 - 2 tipos de datos
 - 3 técnicas de resolución de problemas
 - divide y vencerás
 - algoritmos voraces
 - backtracking
 - programación dinámica: problema de la moneda, problema de la mochila
 - recorrida de grafos

Clase de hoy

1 Repaso

2 Recorrida de grafos

- Generalidades
- Árboles binarios
- Árboles finitarios
- Grafos arbitrarios, DFS
- Grafos arbitrarios, BFS

Recorrida de grafos

Recorrer un grafo, significa **procesar** los vértices del mismo, de forma organizada de modo de asegurarse:

- que todos los vértices sean **procesados**,
- que ninguno de ellos sea **procesado** más de una vez.

Se habla de **procesar** los vértices, pero también utilizaremos la palabra **visitar** los vértices. En este contexto, son sinónimos. Puede haber más de una forma natural de recorrer un cierto grafo.

Clase de hoy

1 Repaso

2 Recorrida de grafos

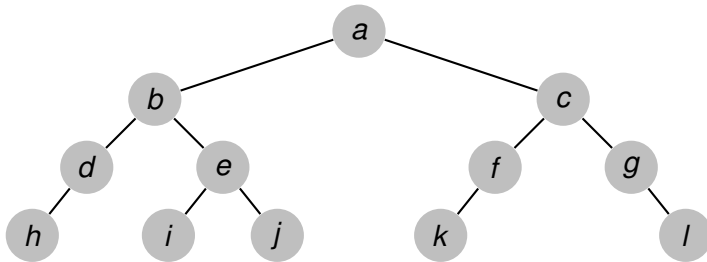
- Generalidades
- **Árboles binarios**
- Árboles finitarios
- Grafos arbitrarios, DFS
- Grafos arbitrarios, BFS

Recorrida de árboles binarios

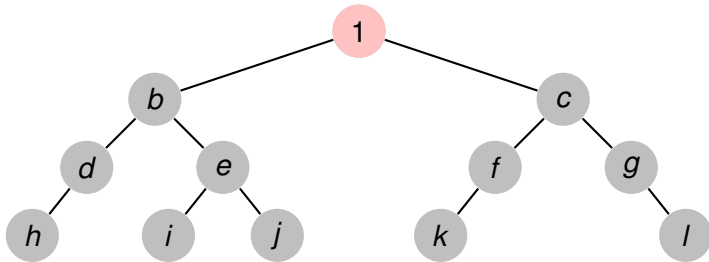
Un caso de grafo sencillo que ya han visto es el de árbol binario. Se han visto 3 maneras de **recorrerlo**:

- pre-order** Se **visita** primero el elemento que se encuentra en la raíz, luego se **recorre** el subárbol izquierdo y finalmente se **recorre** el subárbol derecho.
- in-order** Se **recorre** el subárbol izquierdo, luego se **visita** el elemento que se encuentra en la raíz y finalmente se **recorre** el subárbol derecho.
- pos-order** Se **recorre** el subárbol izquierdo, luego el derecho y finalmente se **visita** el elemento que se encuentra en la raíz.

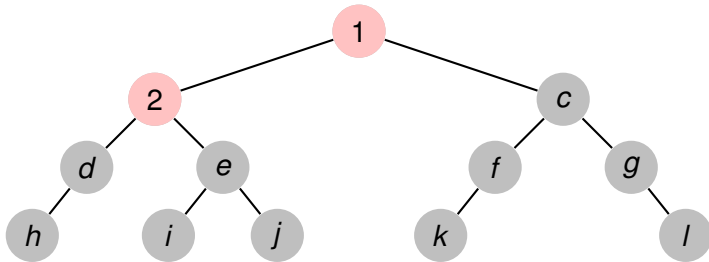
Ejemplo de árbol binario



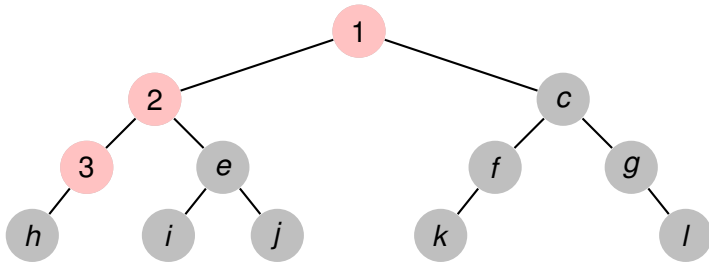
Ejemplo, recorrida pre-order



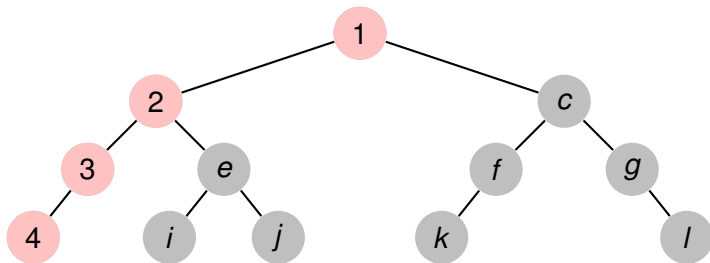
Ejemplo, recorrida pre-order



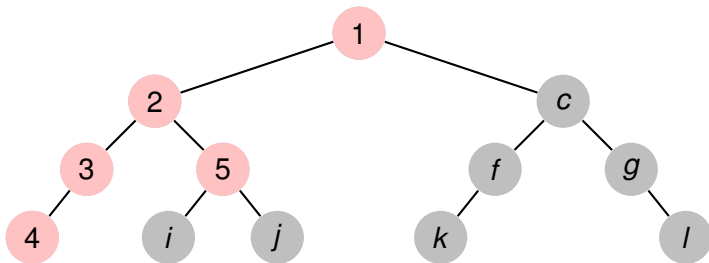
Ejemplo, recorrida pre-order



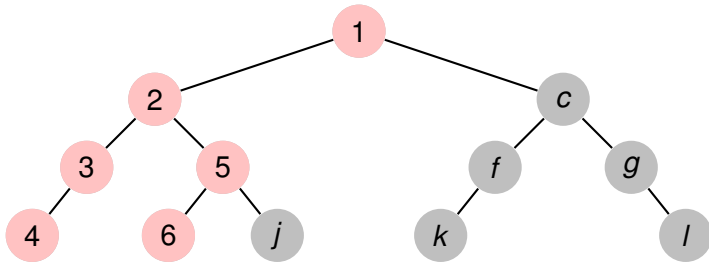
Ejemplo, recorrida pre-order



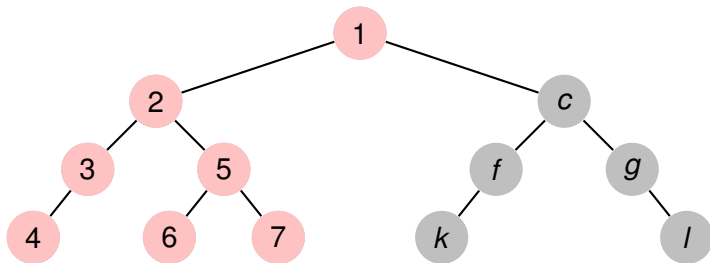
Ejemplo, recorrida pre-order



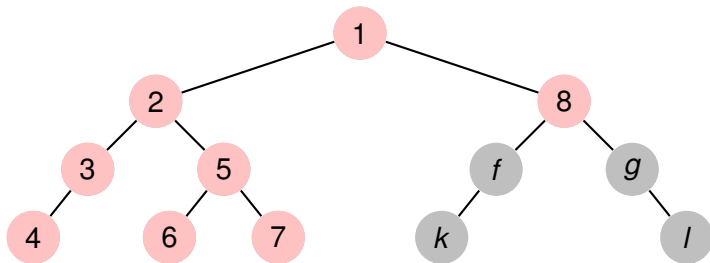
Ejemplo, recorrida pre-order



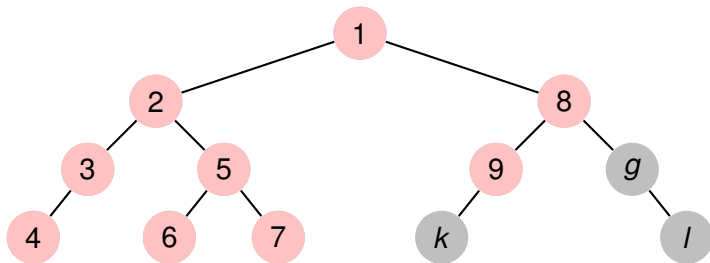
Ejemplo, recorrida pre-order



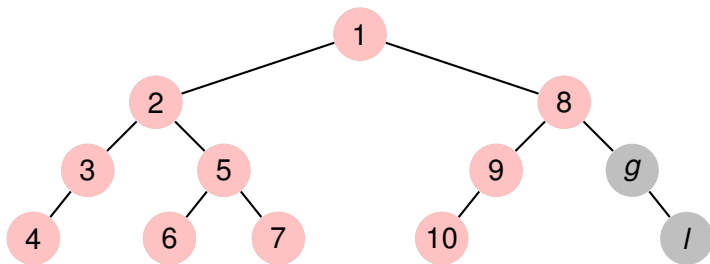
Ejemplo, recorrida pre-order



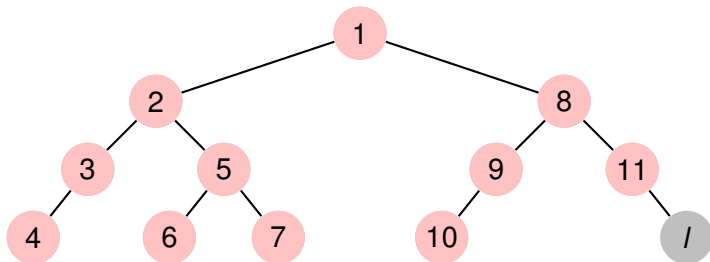
Ejemplo, recorrida pre-order



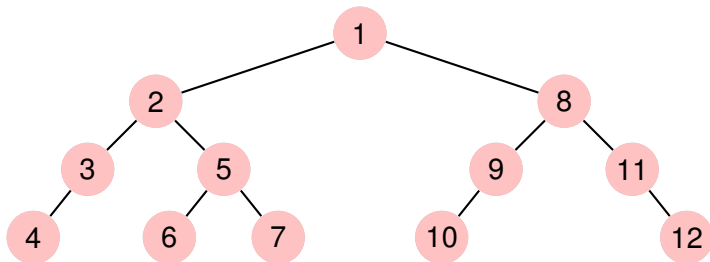
Ejemplo, recorrida pre-order



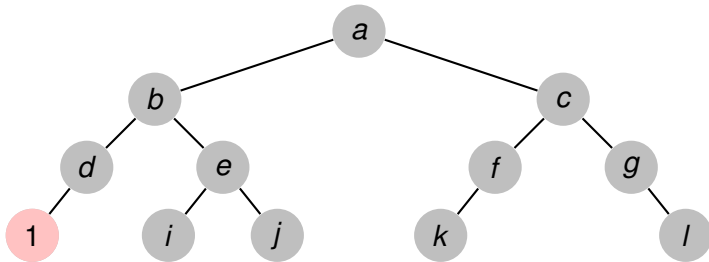
Ejemplo, recorrida pre-order



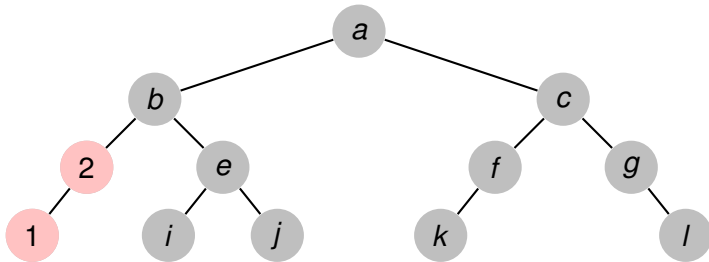
Ejemplo, recorrida pre-order



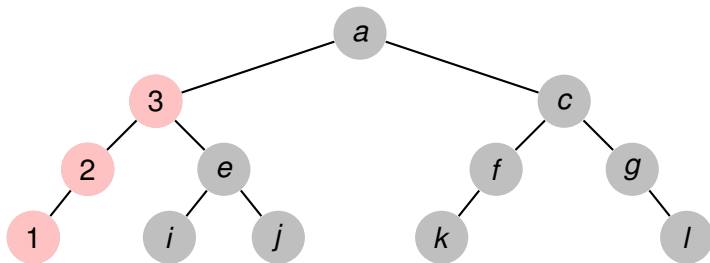
Ejemplo, recorrida in-order



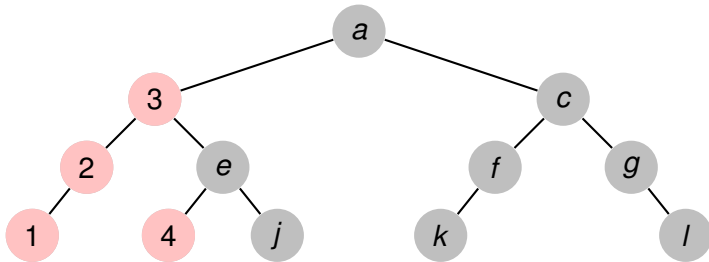
Ejemplo, recorrida in-order



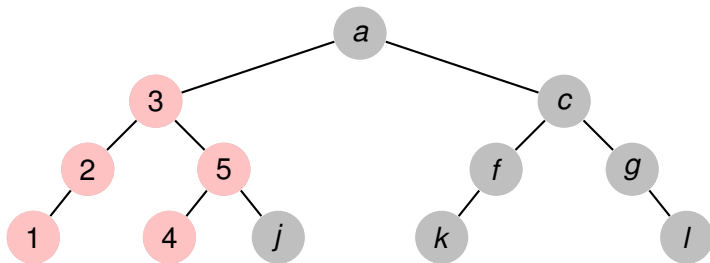
Ejemplo, recorrida in-order



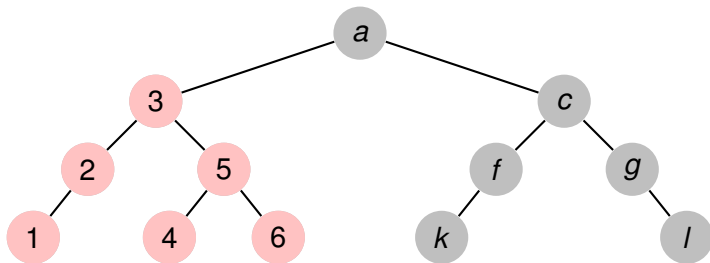
Ejemplo, recorrida in-order



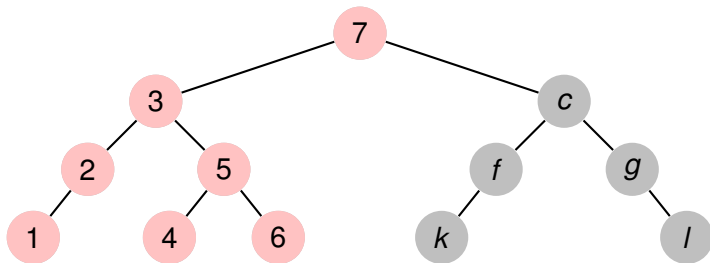
Ejemplo, recorrida in-order



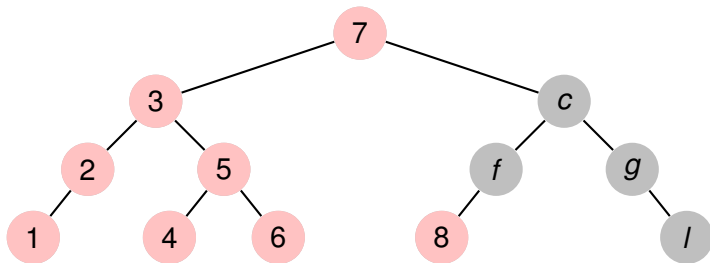
Ejemplo, recorrida in-order



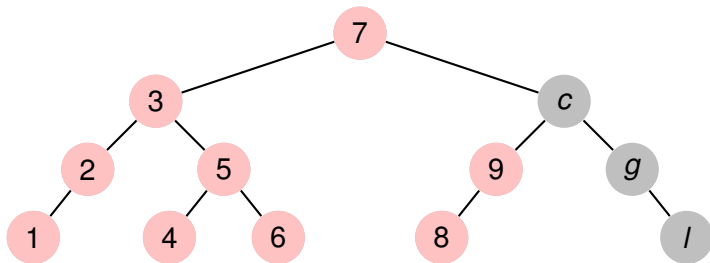
Ejemplo, recorrida in-order



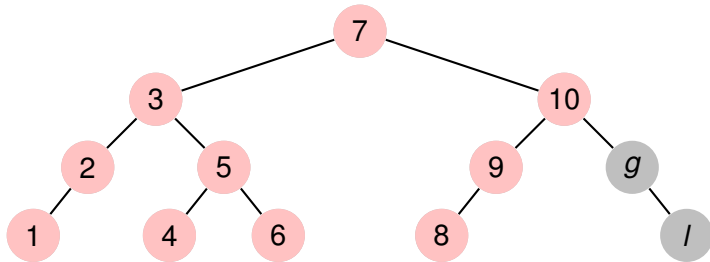
Ejemplo, recorrida in-order



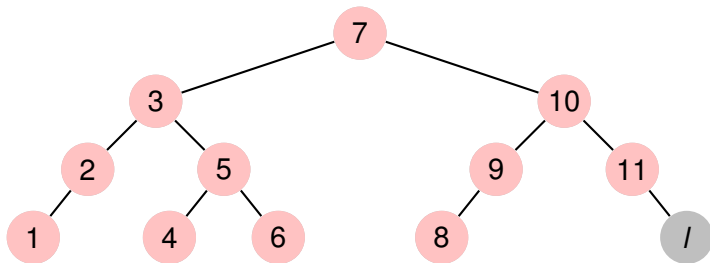
Ejemplo, recorrida in-order



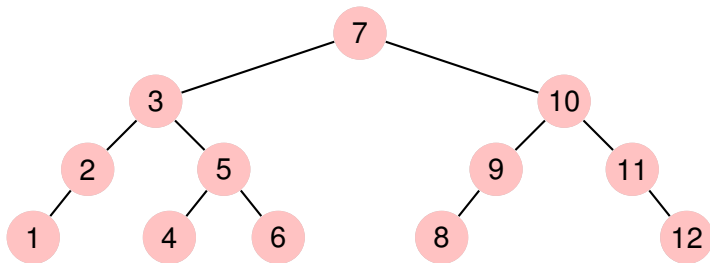
Ejemplo, recorrida in-order



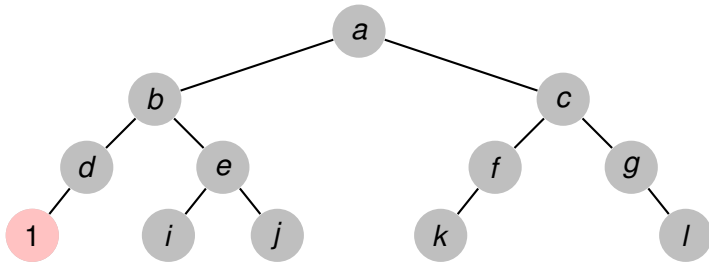
Ejemplo, recorrida in-order



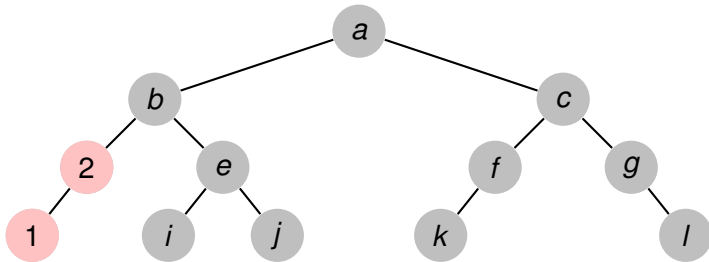
Ejemplo, recorrida in-order



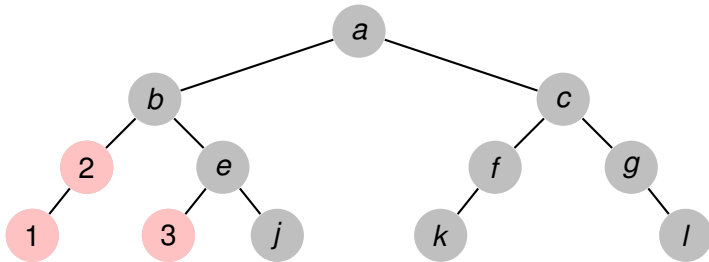
Ejemplo, recorrida pos-order



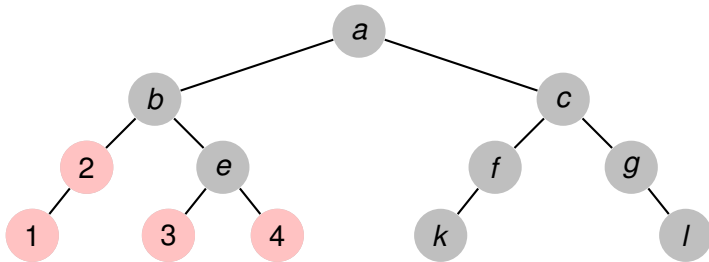
Ejemplo, recorrida pos-order



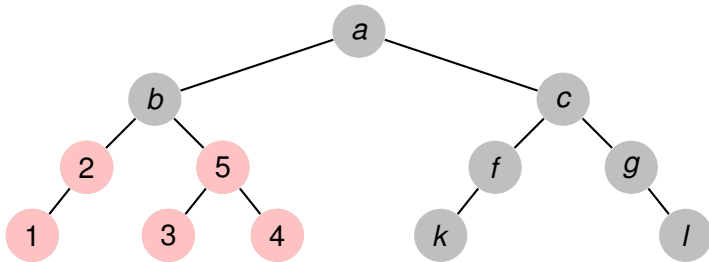
Ejemplo, recorrida pos-order



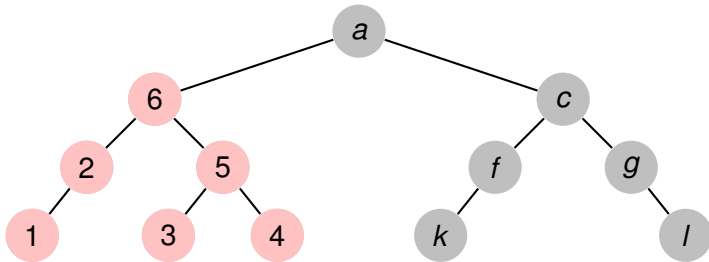
Ejemplo, recorrida pos-order



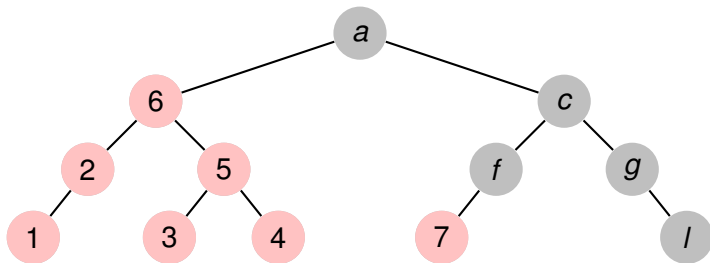
Ejemplo, recorrida pos-order



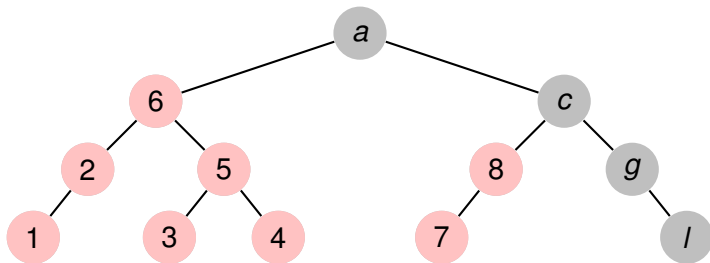
Ejemplo, recorrida pos-order



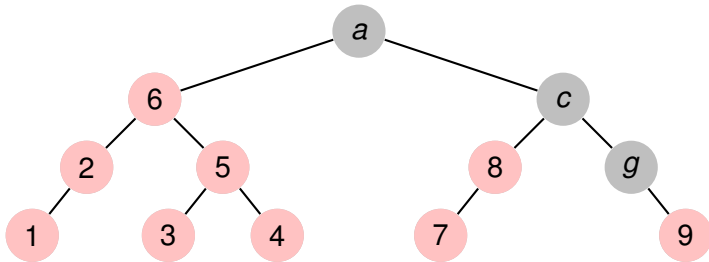
Ejemplo, recorrida pos-order



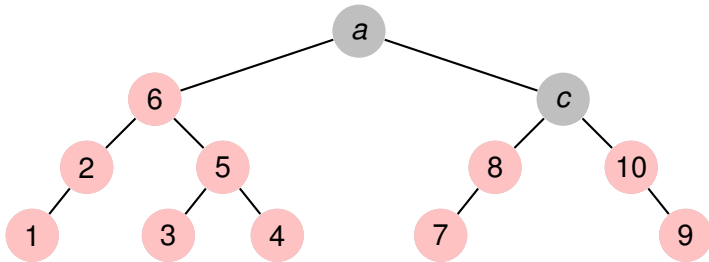
Ejemplo, recorrida pos-order



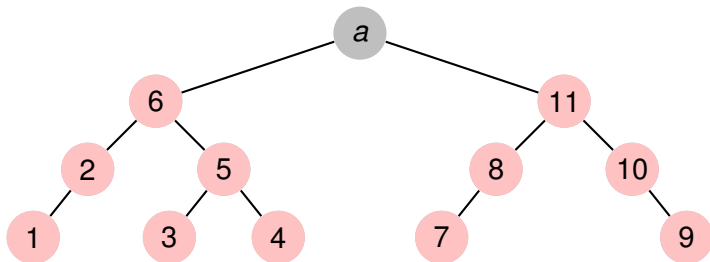
Ejemplo, recorrida pos-order



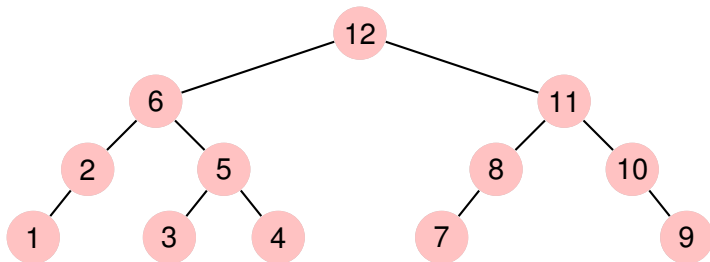
Ejemplo, recorrida pos-order



Ejemplo, recorrida pos-order



Ejemplo, recorrida pos-order

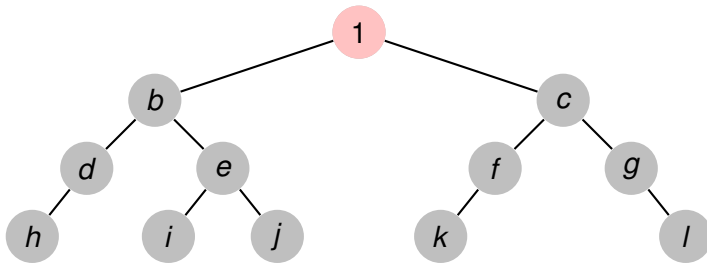


Otras 3 maneras de recorrer árboles binarios

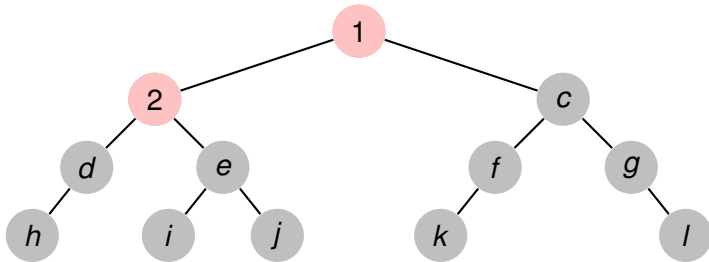
Hay otras tres maneras de recorrer: en cada una de las anteriores, intercambiar el orden entre las recorridas de los subárboles. Por ejemplo:

```
in_order_der_izqA Vacío = [ ]  
in_order_der_izqA (Nodo i r d) =  
    in_order_der_izqA d ++ (r : in_order_der_izqA i)
```

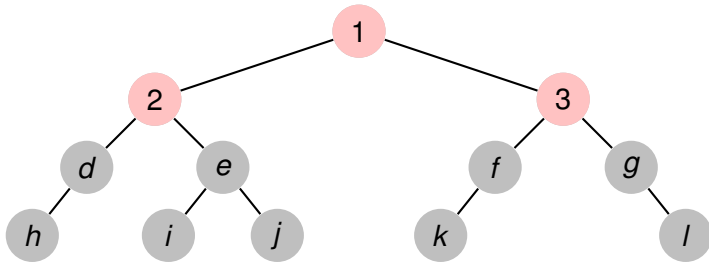
Otra manera más de recorrer árboles binarios



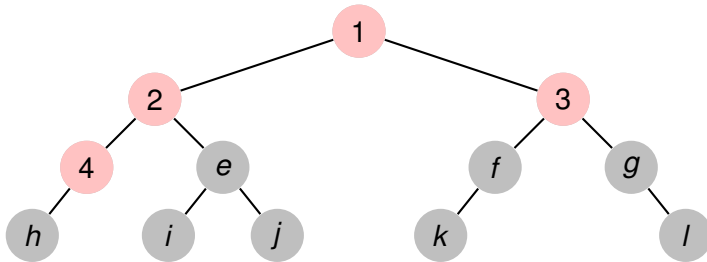
Otra manera más de recorrer árboles binarios



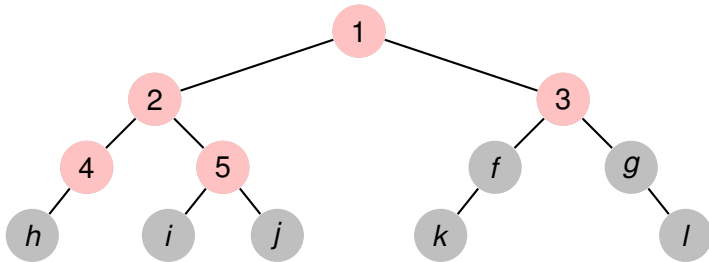
Otra manera más de recorrer árboles binarios



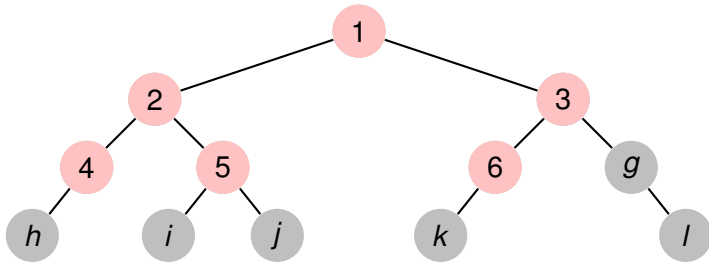
Otra manera más de recorrer árboles binarios



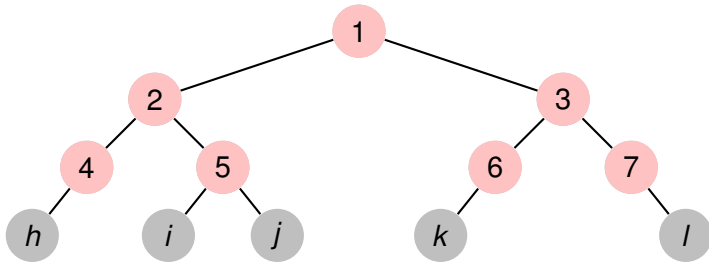
Otra manera más de recorrer árboles binarios



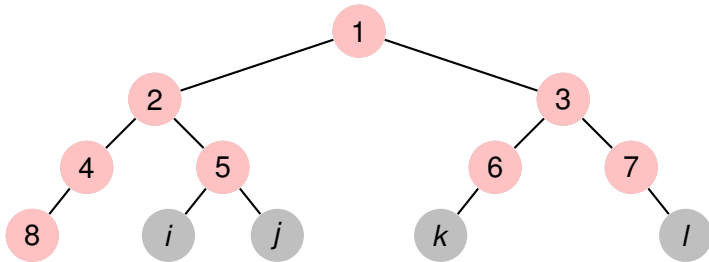
Otra manera más de recorrer árboles binarios



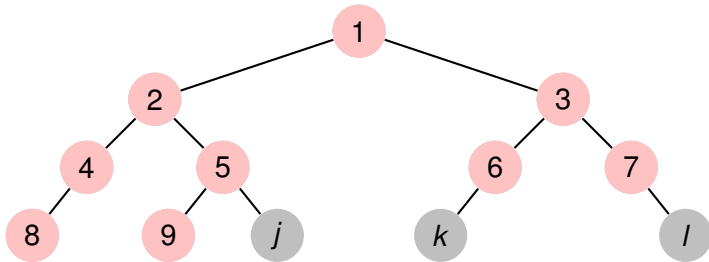
Otra manera más de recorrer árboles binarios



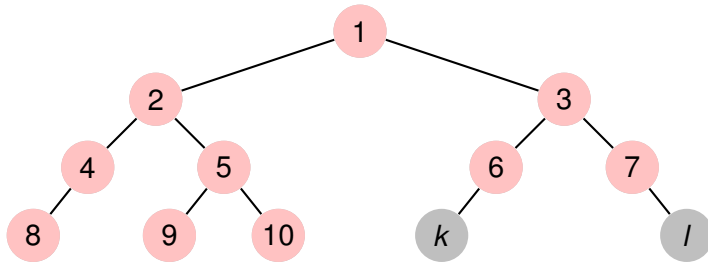
Otra manera más de recorrer árboles binarios



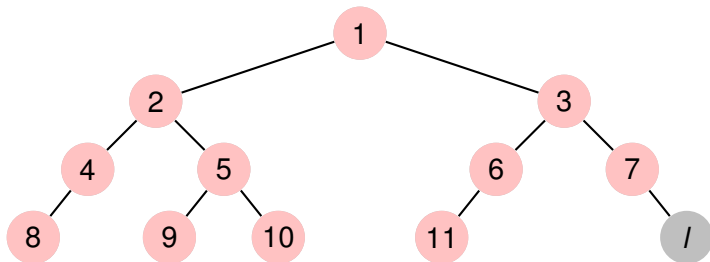
Otra manera más de recorrer árboles binarios



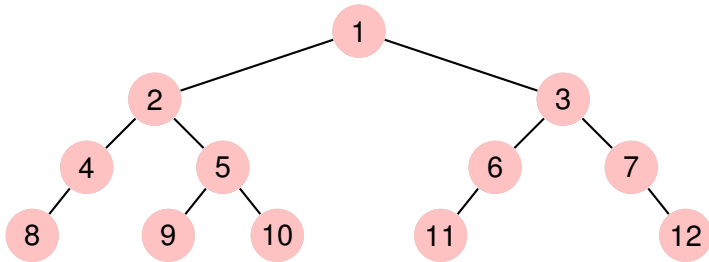
Otra manera más de recorrer árboles binarios



Otra manera más de recorrer árboles binarios



Otra manera más de recorrer árboles binarios



Otra manera más de recorrer árboles binarios

Algunas observaciones:

- salvo la última, todas las formas anteriores de recorrer, primero recorren **en profundidad**
- la última que presentamos, no,
- recorre **a lo ancho**.
- Todas las otras son ejemplo de DFS (Depth-first search).
- La última es ejemplo de BFS (Breadth-first search).
- Un programa que recorra en BFS es más difícil de escribir, se verá al final de la clase de hoy.

Clase de hoy

1 Repaso

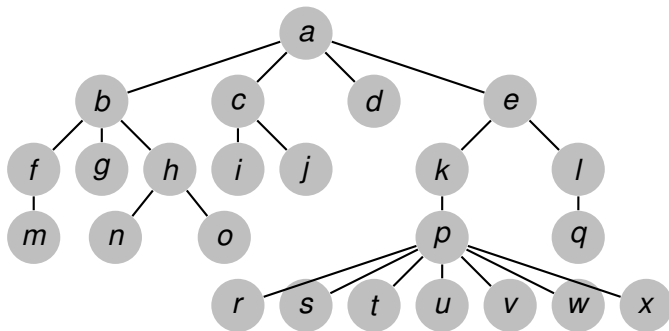
2 Recorrida de grafos

- Generalidades
- Árboles binarios
- **Árboles finitarios**
- Grafos arbitrarios, DFS
- Grafos arbitrarios, BFS

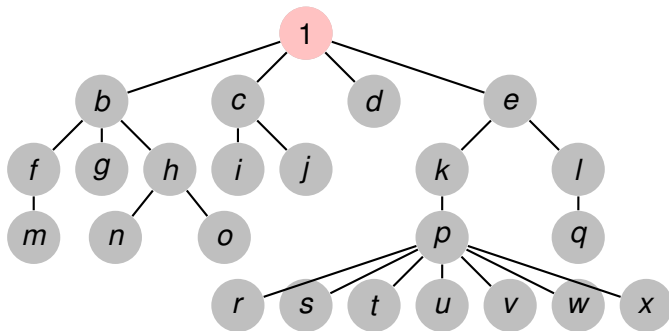
Recorrida de árboles finitarios

- Son árboles en los que cada vértice tiene una cantidad finita (pero puede ser variable) de hijos.
- La recorrida in-order deja de tener sentido (habiendo más de dos hijos, ¿en qué momento habría que visitar el elemento que se encuentra en la raíz?).
- Las recorridas pre-order y pos-order (DFS) y BFS siguen teniendo sentido.

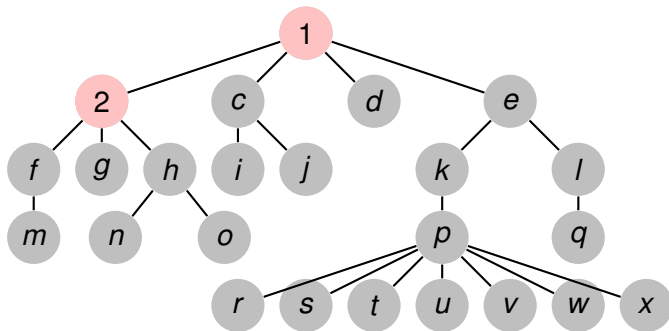
Ejemplo de árbol finitario



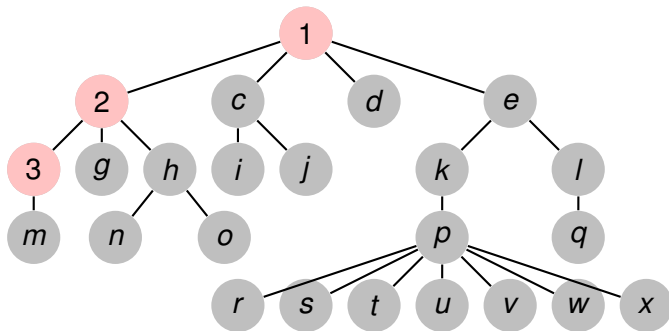
Ejemplo, recorrida pre-order



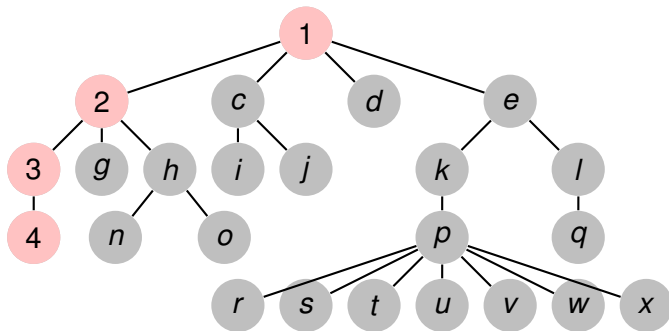
Ejemplo, recorrida pre-order



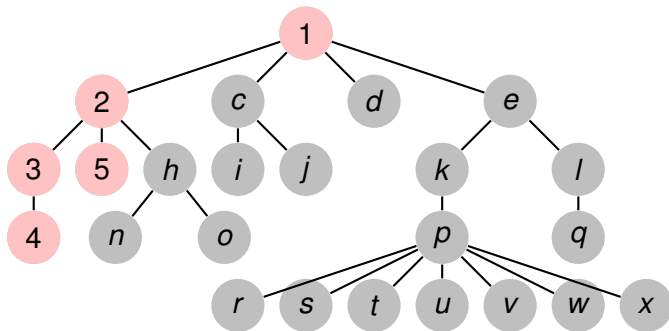
Ejemplo, recorrida pre-order



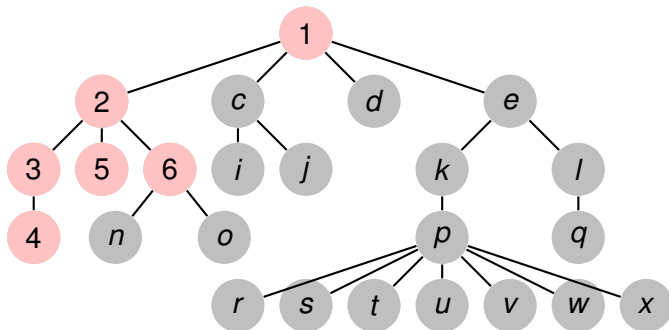
Ejemplo, recorrida pre-order



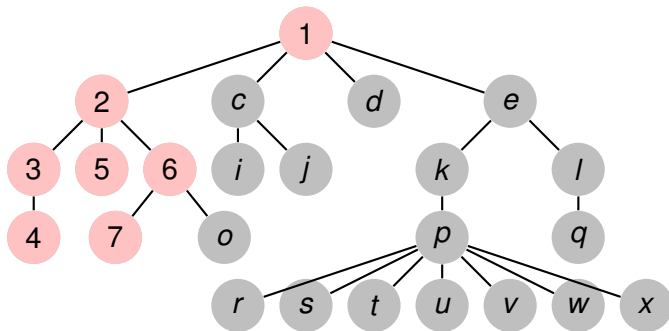
Ejemplo, recorrida pre-order



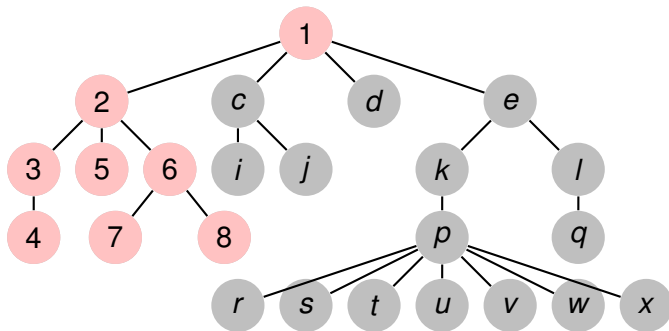
Ejemplo, recorrida pre-order



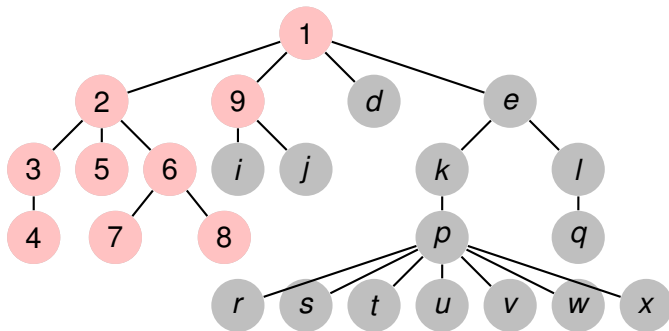
Ejemplo, recorrida pre-order



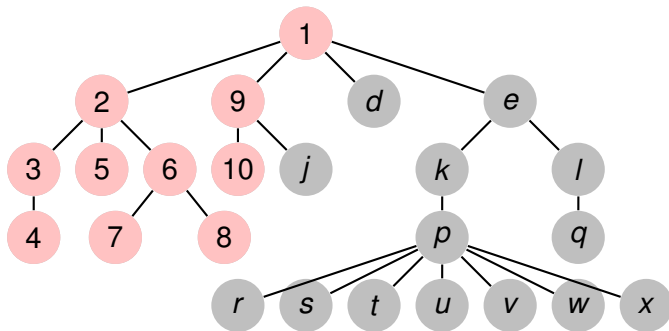
Ejemplo, recorrida pre-order



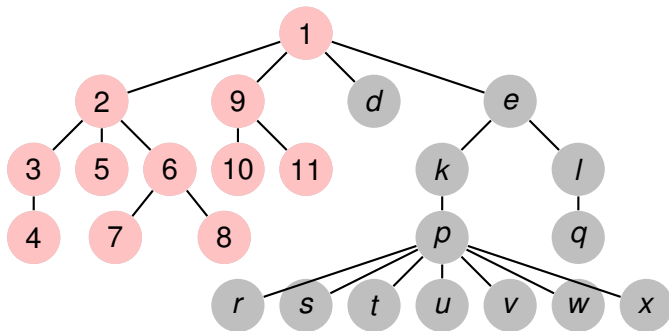
Ejemplo, recorrida pre-order



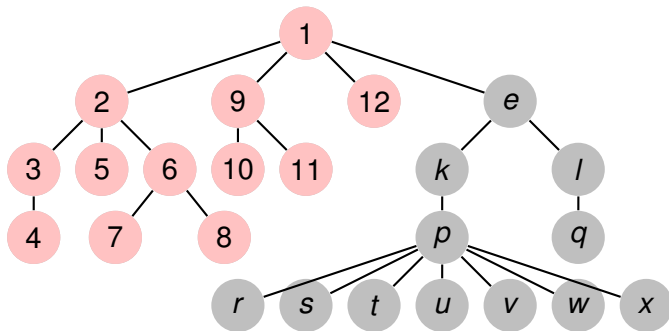
Ejemplo, recorrida pre-order



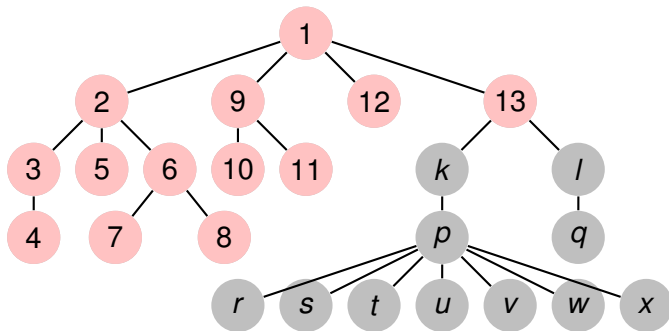
Ejemplo, recorrida pre-order



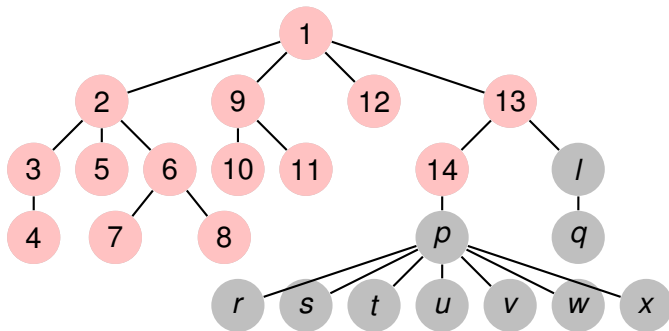
Ejemplo, recorrida pre-order



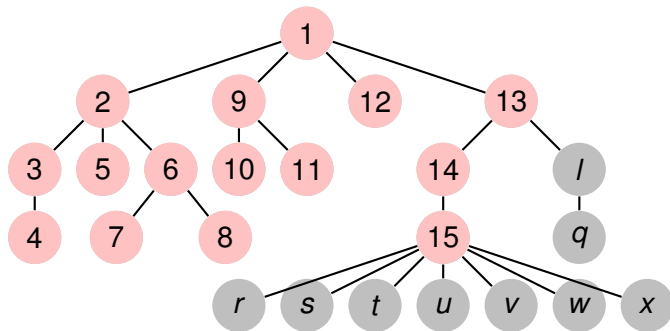
Ejemplo, recorrida pre-order



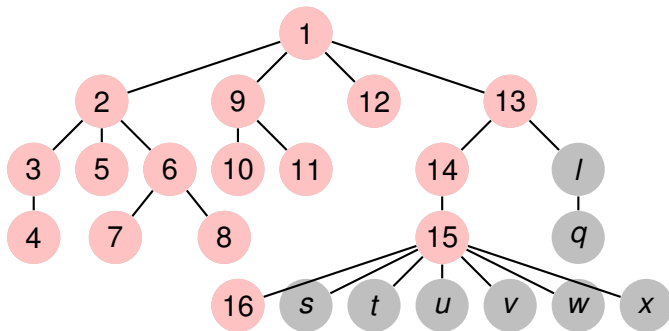
Ejemplo, recorrida pre-order



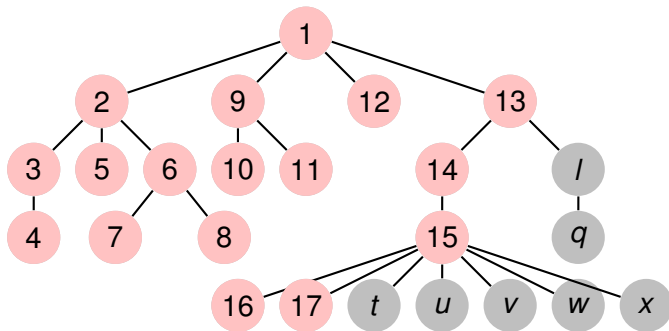
Ejemplo, recorrida pre-order



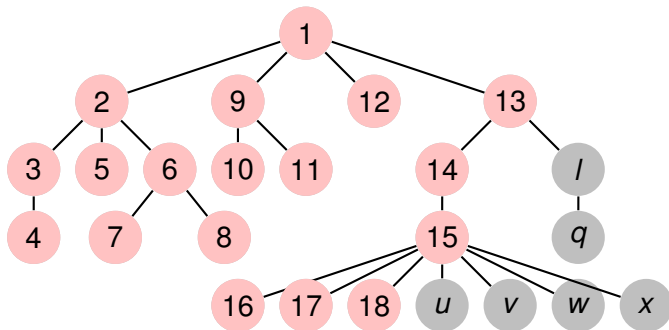
Ejemplo, recorrida pre-order



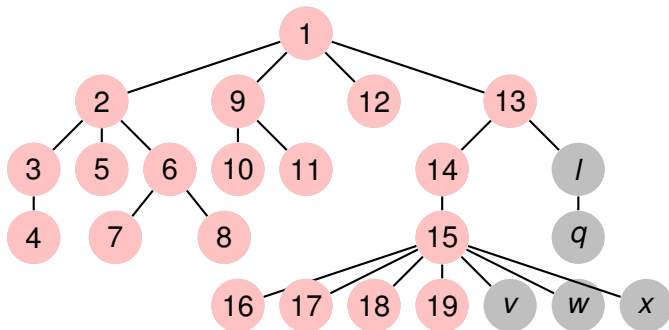
Ejemplo, recorrida pre-order



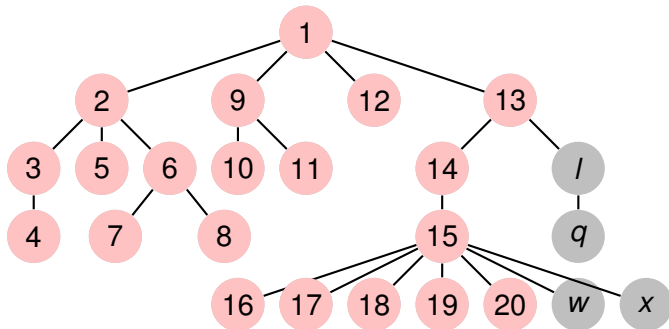
Ejemplo, recorrida pre-order



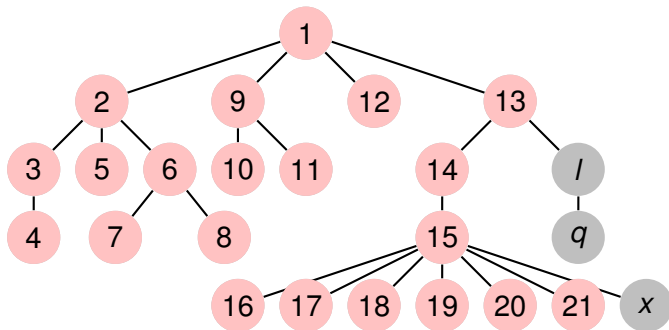
Ejemplo, recorrida pre-order



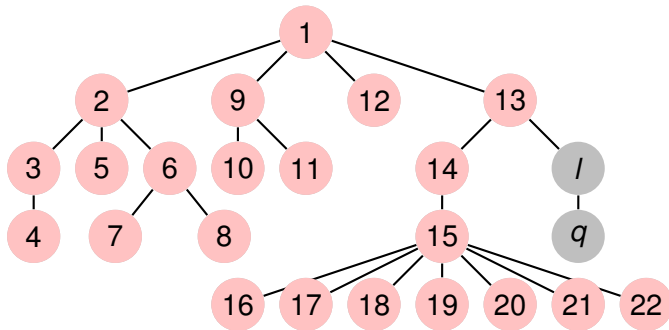
Ejemplo, recorrida pre-order



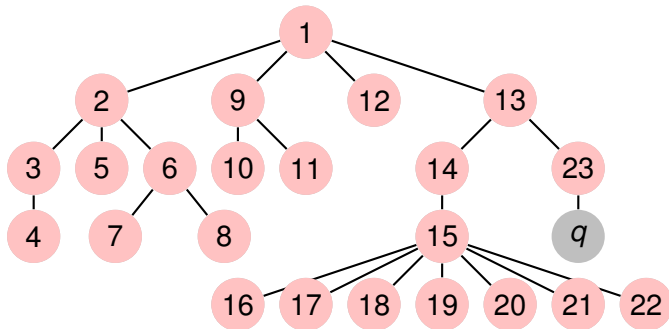
Ejemplo, recorrida pre-order



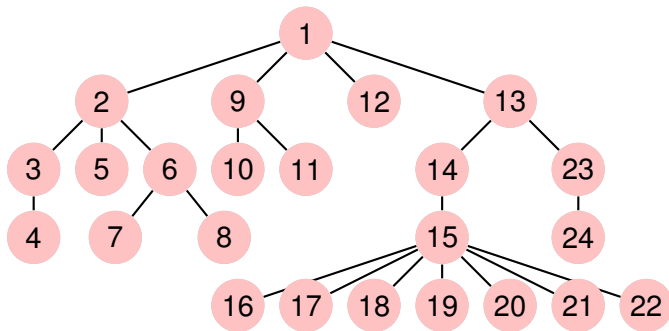
Ejemplo, recorrida pre-order



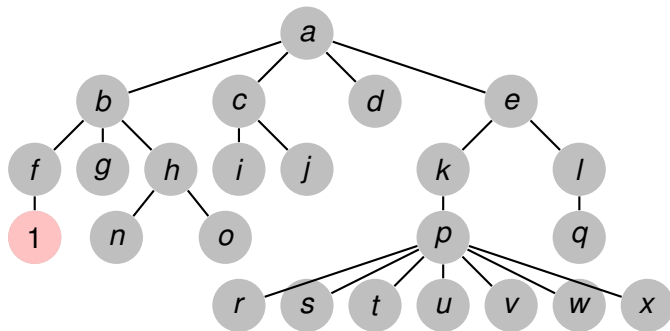
Ejemplo, recorrida pre-order



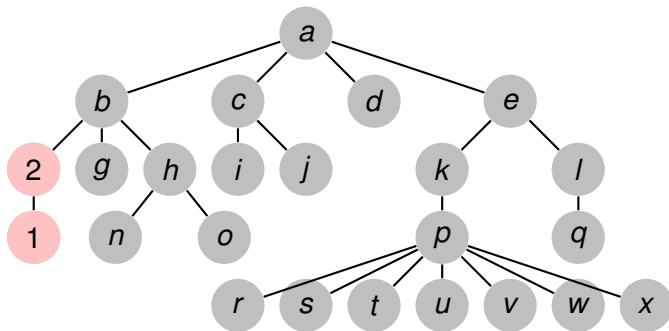
Ejemplo, recorrida pre-order



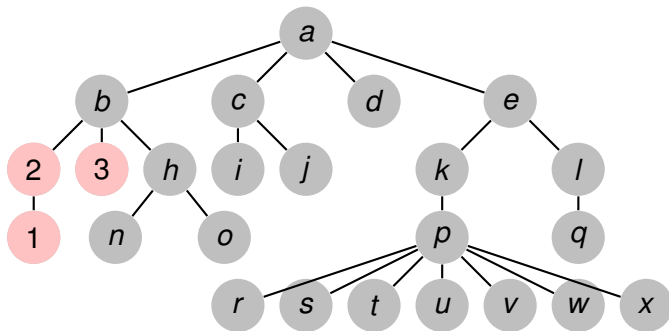
Ejemplo, recorrida pos-order



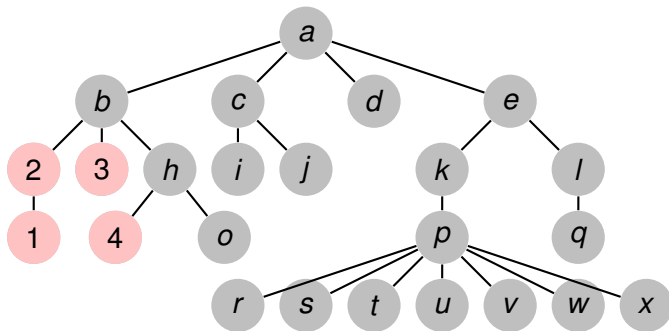
Ejemplo, recorrida pos-order



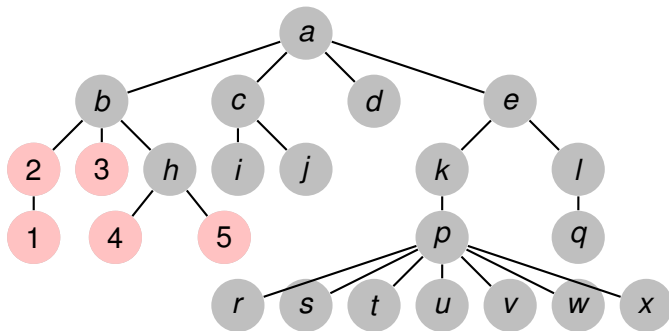
Ejemplo, recorrida pos-order



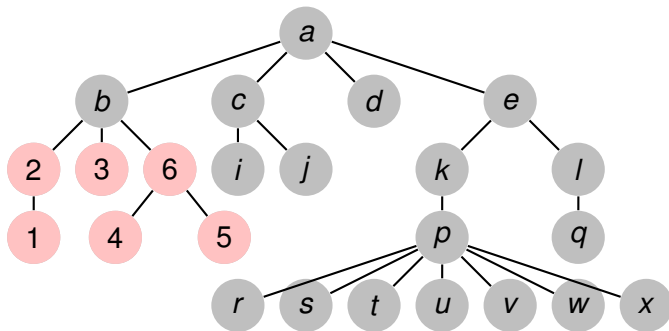
Ejemplo, recorrida pos-order



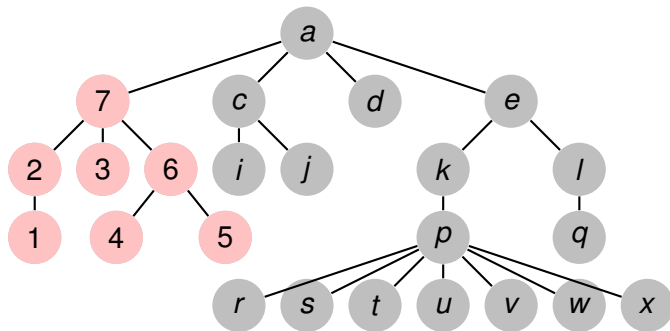
Ejemplo, recorrida pos-order



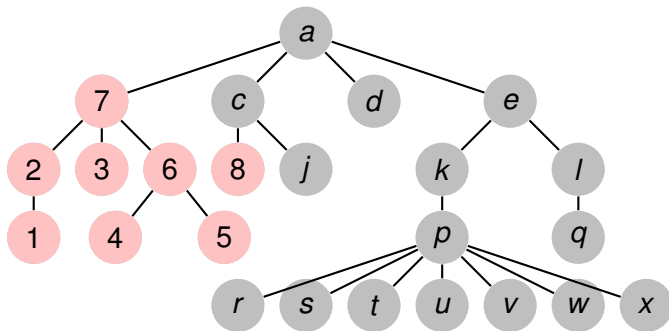
Ejemplo, recorrida pos-order



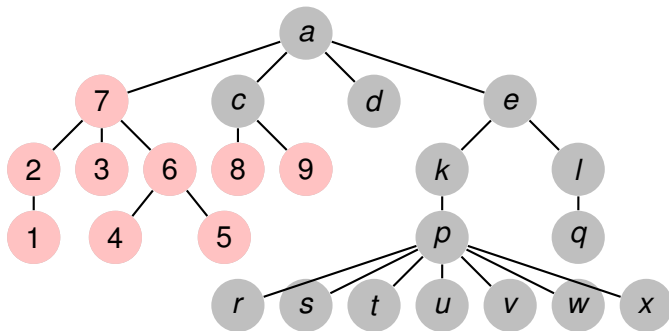
Ejemplo, recorrida pos-order



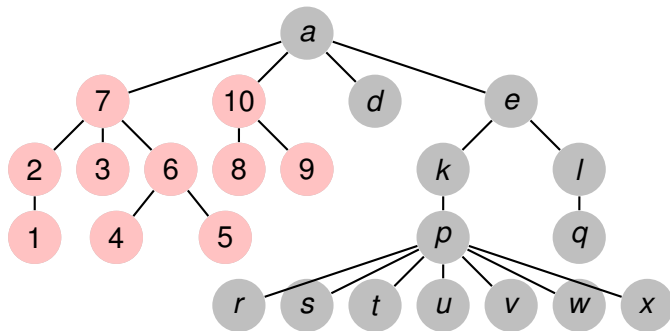
Ejemplo, recorrida pos-order



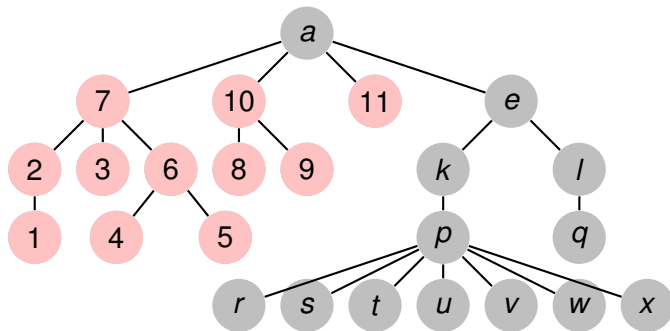
Ejemplo, recorrida pos-order



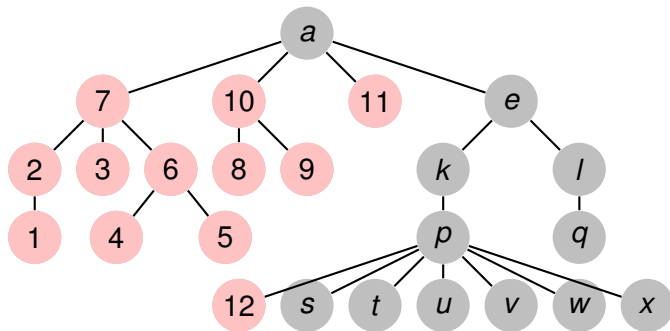
Ejemplo, recorrida pos-order



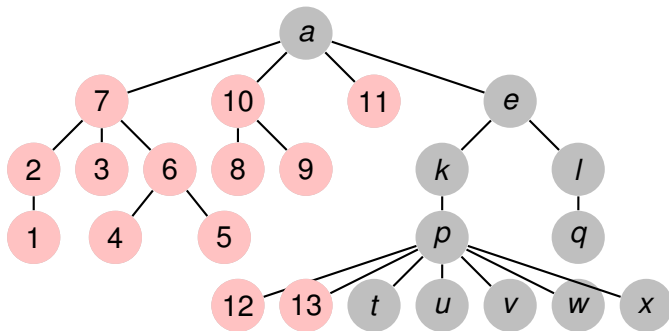
Ejemplo, recorrida pos-order



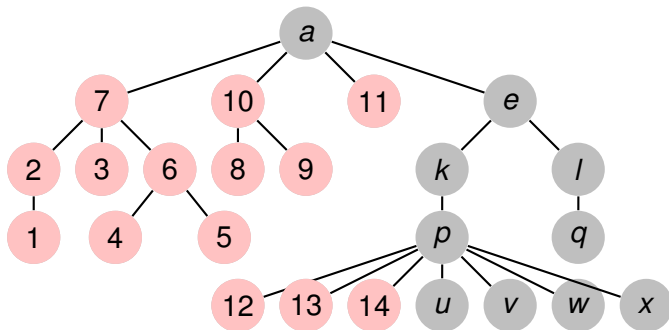
Ejemplo, recorrida pos-order



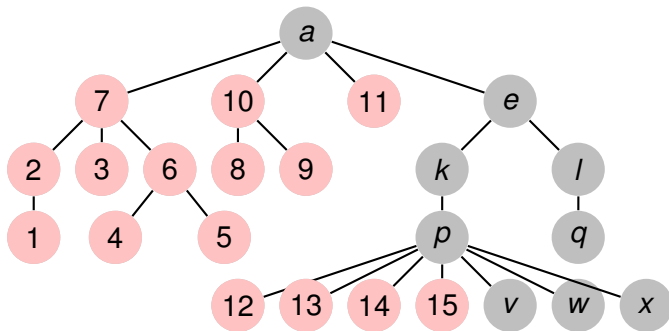
Ejemplo, recorrida pos-order



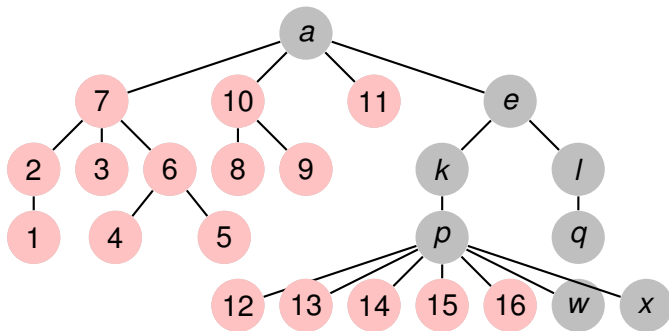
Ejemplo, recorrida pos-order



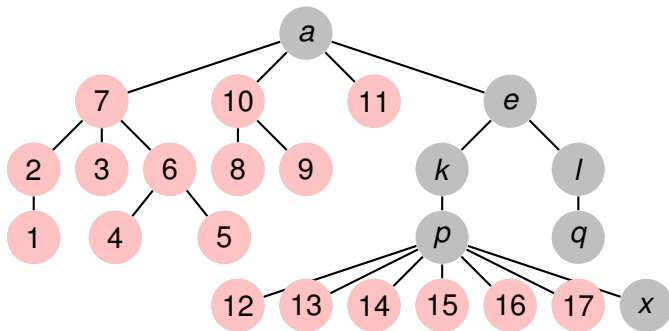
Ejemplo, recorrida pos-order



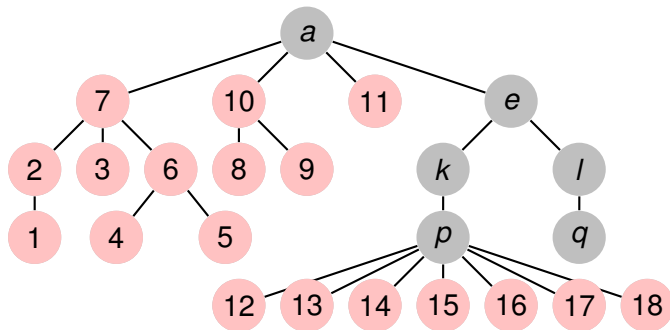
Ejemplo, recorrida pos-order



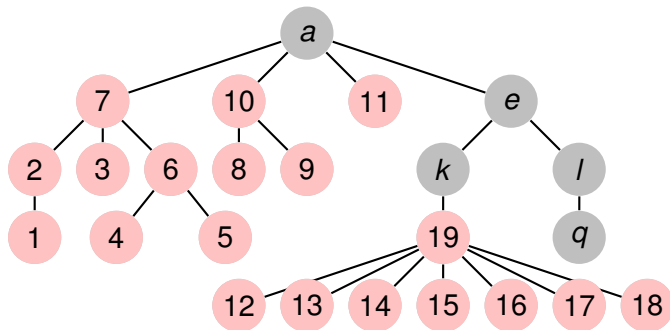
Ejemplo, recorrida pos-order



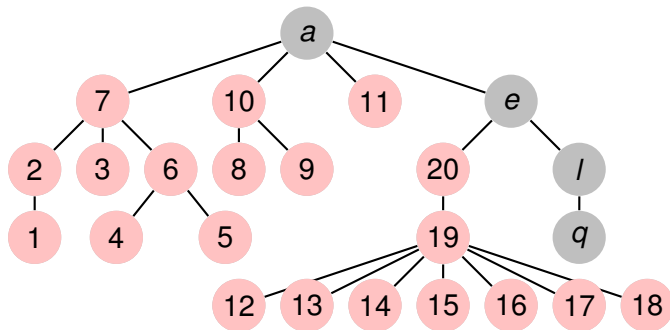
Ejemplo, recorrida pos-order



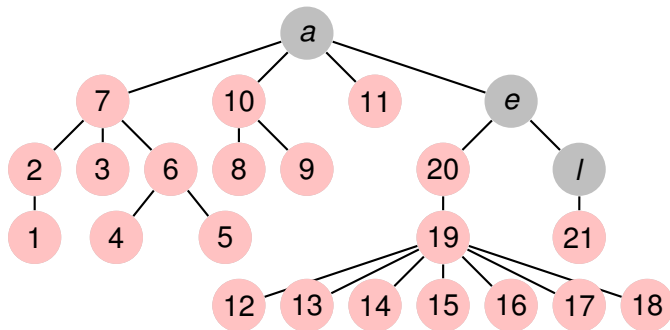
Ejemplo, recorrida pos-order



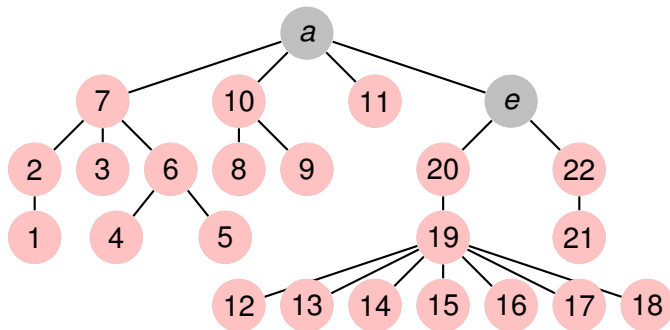
Ejemplo, recorrida pos-order



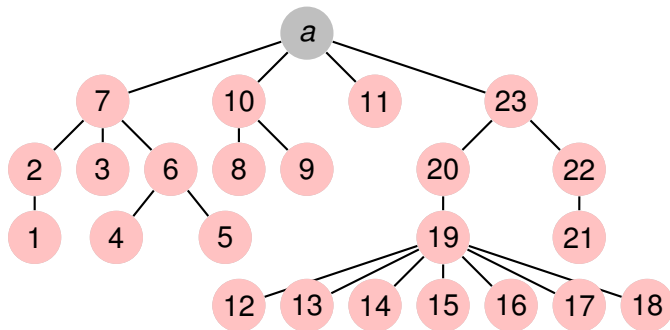
Ejemplo, recorrida pos-order



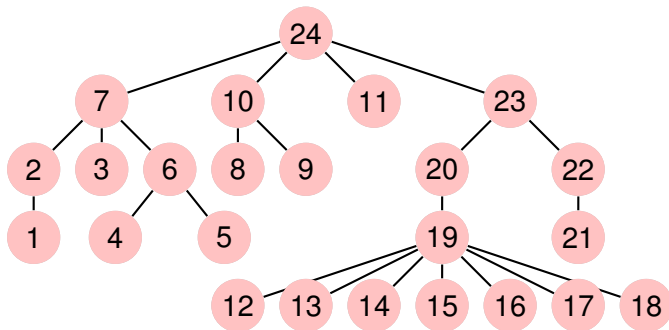
Ejemplo, recorrida pos-order



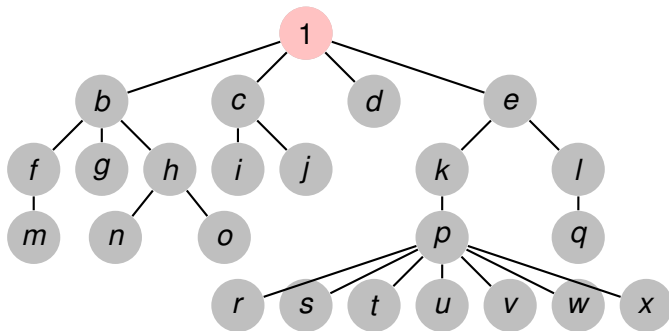
Ejemplo, recorrida pos-order



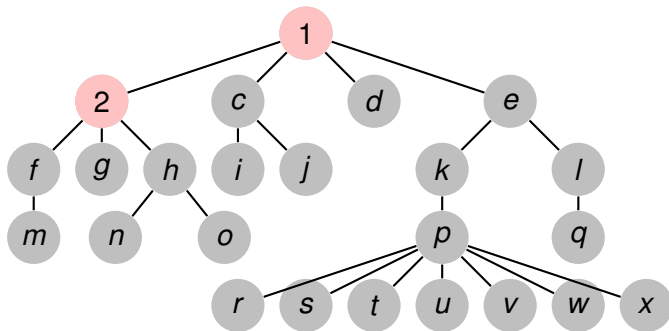
Ejemplo, recorrida pos-order



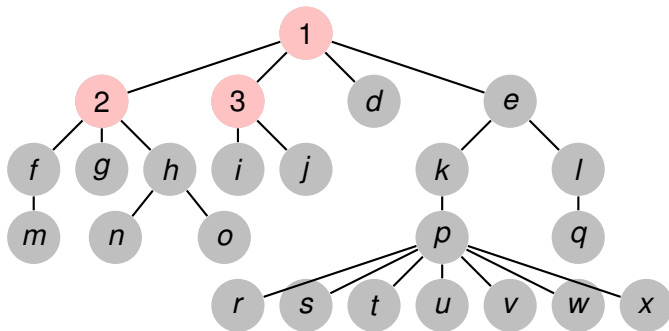
Ejemplo, recorrida BFS



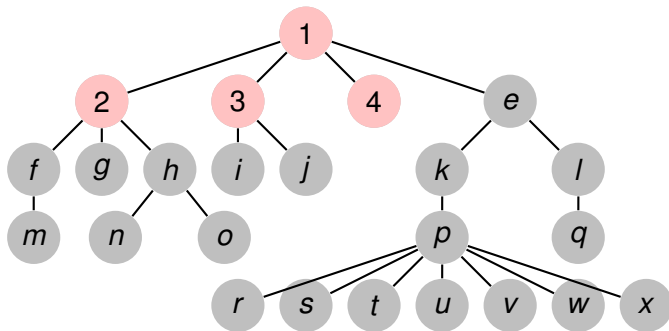
Ejemplo, recorrida BFS



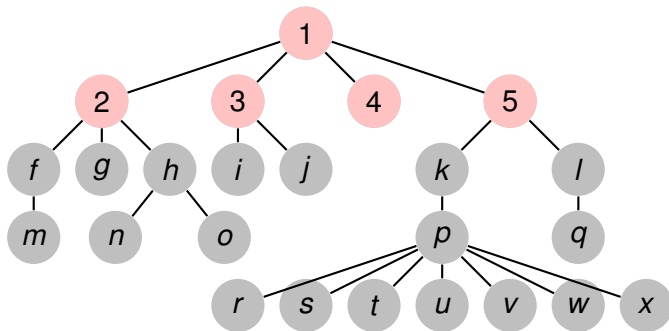
Ejemplo, recorrida BFS



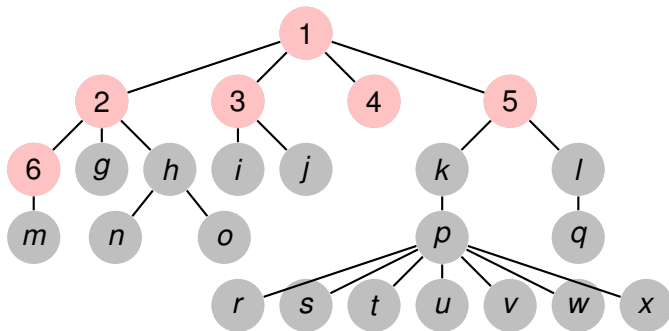
Ejemplo, recorrida BFS



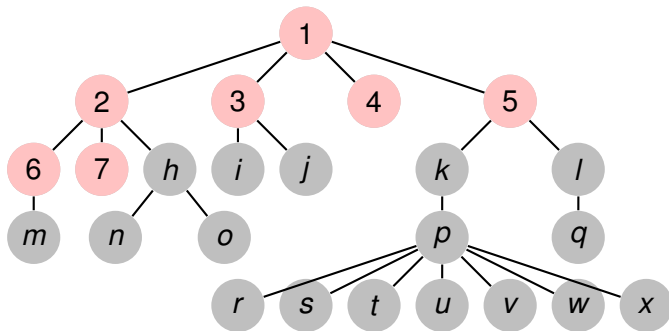
Ejemplo, recorrida BFS



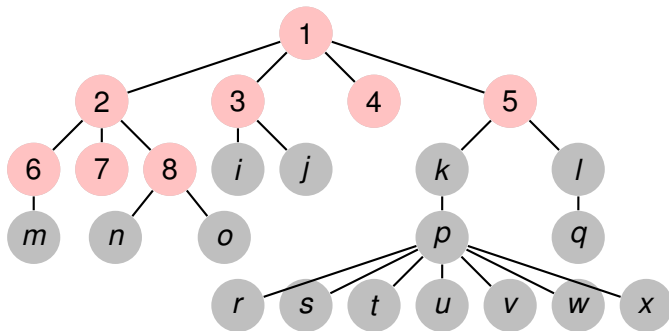
Ejemplo, recorrida BFS



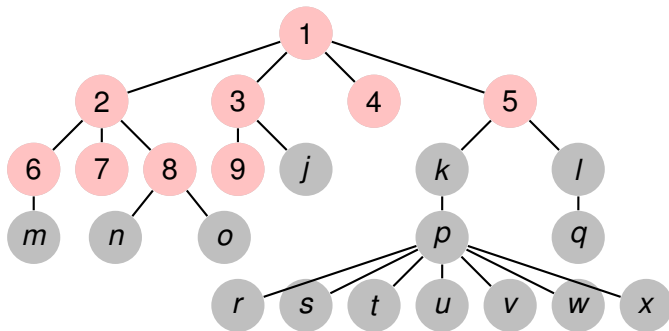
Ejemplo, recorrida BFS



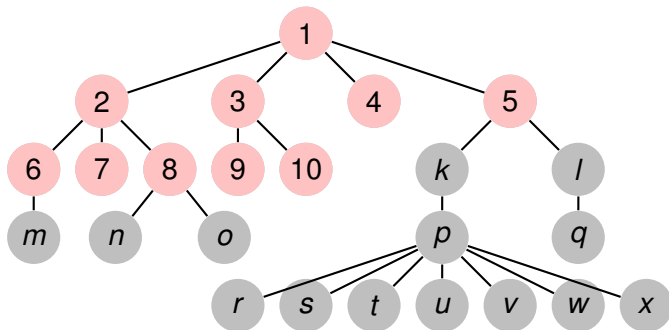
Ejemplo, recorrida BFS



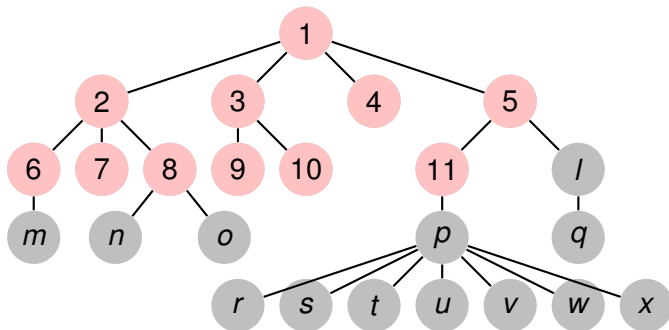
Ejemplo, recorrida BFS



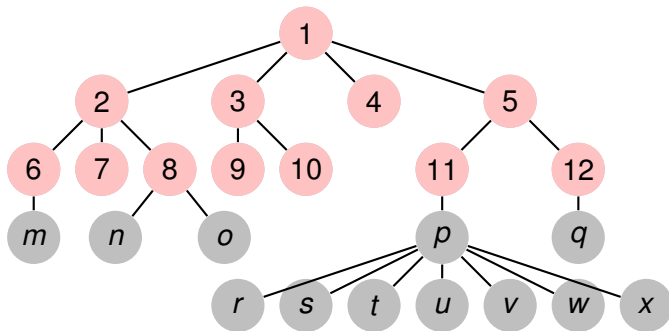
Ejemplo, recorrida BFS



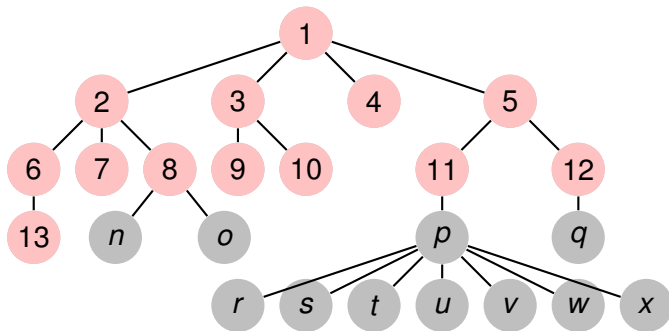
Ejemplo, recorrida BFS



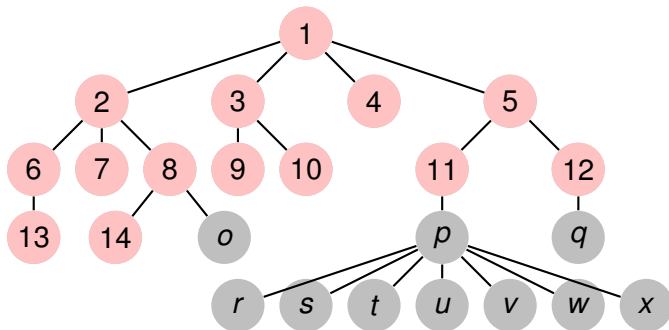
Ejemplo, recorrida BFS



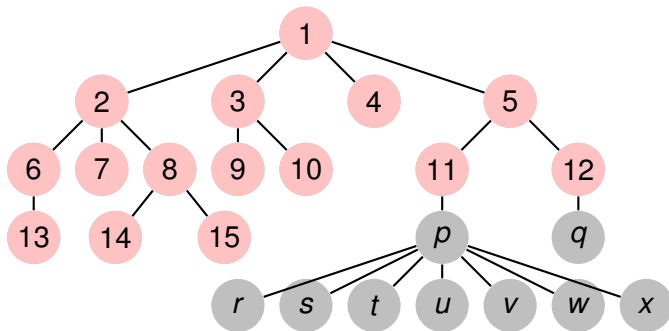
Ejemplo, recorrida BFS



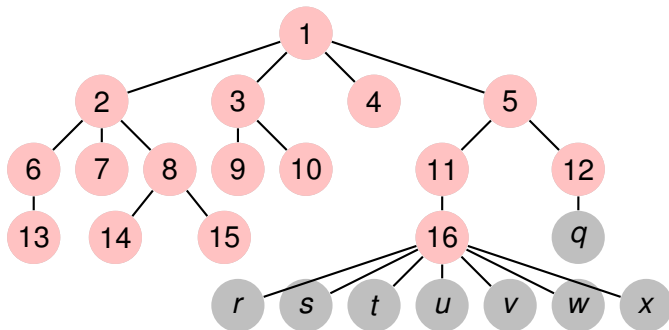
Ejemplo, recorrida BFS



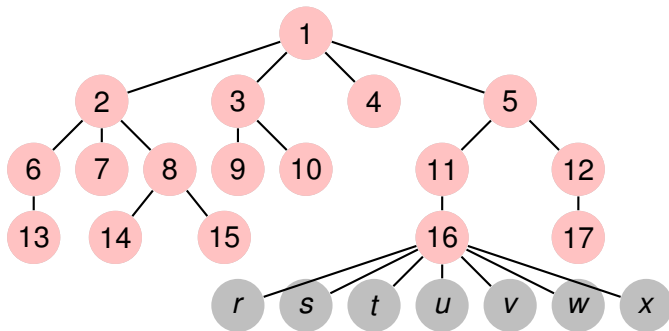
Ejemplo, recorrida BFS



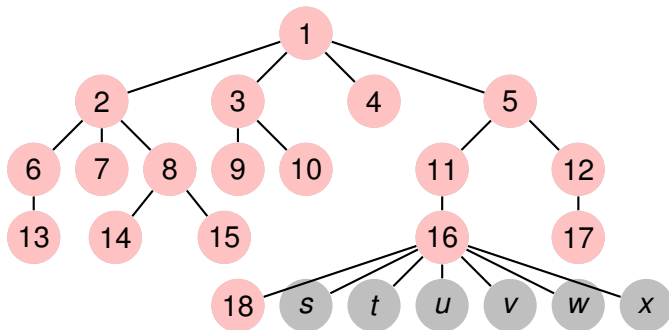
Ejemplo, recorrida BFS



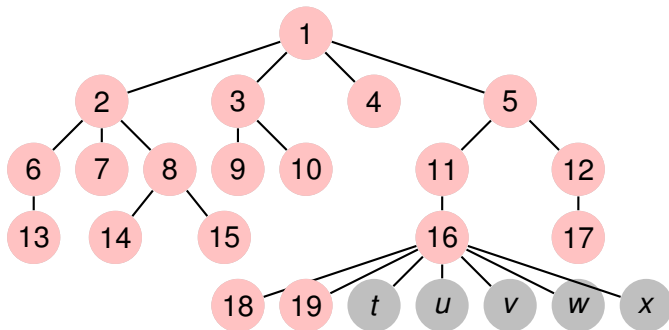
Ejemplo, recorrida BFS



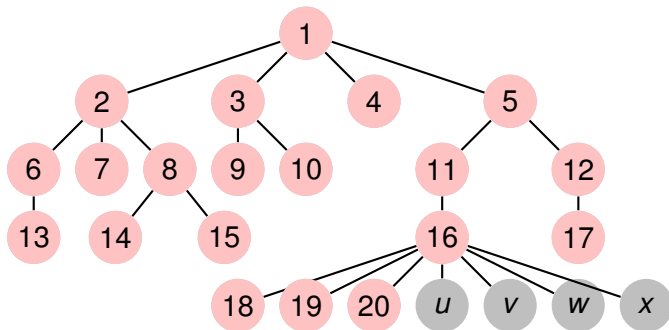
Ejemplo, recorrida BFS



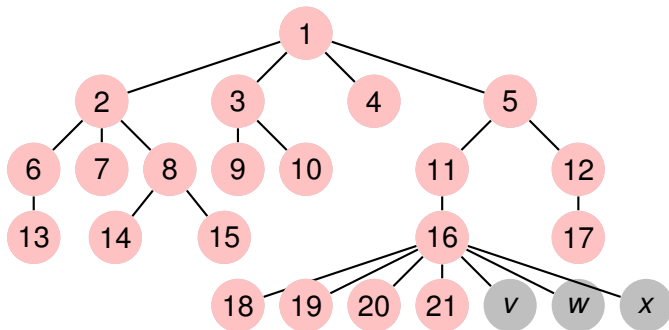
Ejemplo, recorrida BFS



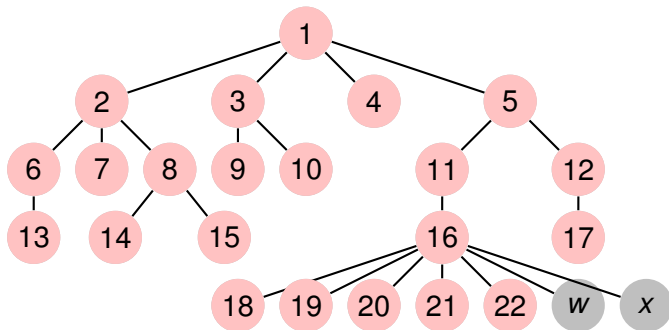
Ejemplo, recorrida BFS



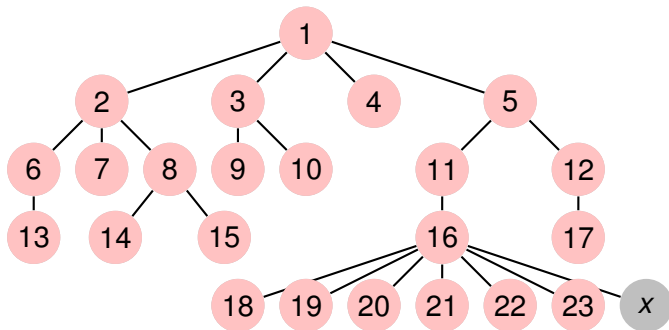
Ejemplo, recorrida BFS



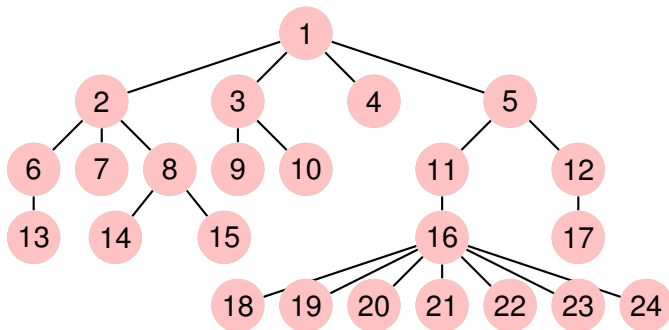
Ejemplo, recorrida BFS



Ejemplo, recorrida BFS



Ejemplo, recorrida BFS



Algoritmos

Marcas

Cuando se visita un vértice, se marca con un número positivo.

```
type tmark = tuple
    ord: array[V] of nat
    cont: nat
end

proc init(out mark: tmark)
    mark.cont:= 0
end

proc visit(in/out mark: tmark, in v: V)
    mark.cont:= mark.cont+1
    mark.ord[v]:= mark.cont
end
```

Algoritmos

pre-order

Asumimos que un árbol viene dado por su raíz (root) y una función (children) que devuelve (el conjunto o la lista de) los hijos de cada vértice.

```
fun pre_order(G=(V,root,children)) ret mark: tmark  
    init(mark)  
    pre_traverse(G, mark, root)  
end  
  
proc pre_traverse(in G, in/out mark: tmark, in v: V)  
    visit(mark,v)  
    for w  $\in$  children(v) do pre_traverse(G, mark, w) od  
end
```

Algoritmos

pos-order

```
fun pos_order(G=(V,root,children)) ret mark: tmark  
    init(mark)  
    pos_traverse(G, mark, root)  
end  
  
proc pos_traverse(in G, in/out mark: tmark, in v: V)  
    for w  $\in$  children(v) do pos_traverse(G, mark, w) od  
    visit(mark,v)  
end
```

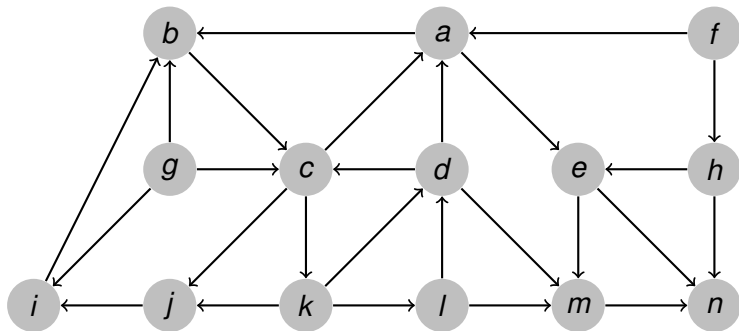
Clase de hoy

1 Repaso

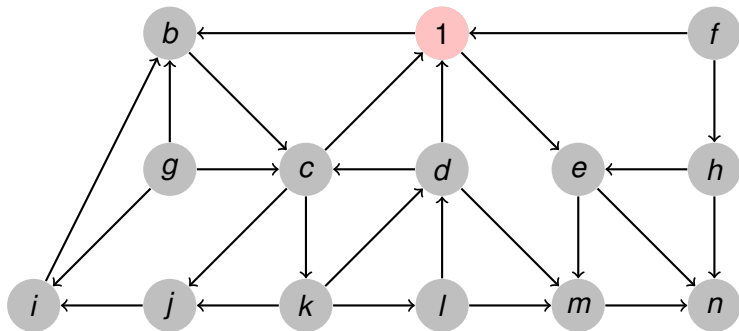
2 Recorrida de grafos

- Generalidades
- Árboles binarios
- Árboles finitarios
- **Grafos arbitrarios, DFS**
- Grafos arbitrarios, BFS

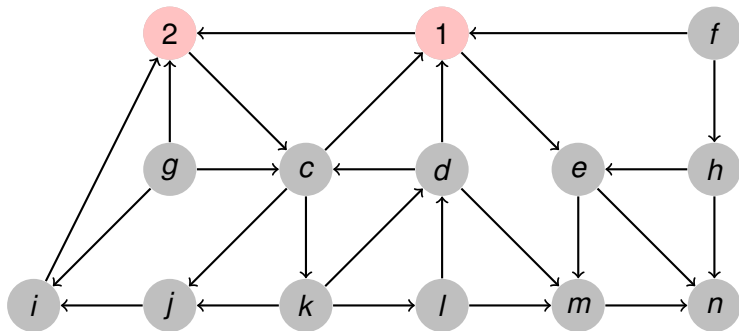
Ejemplo de grafo



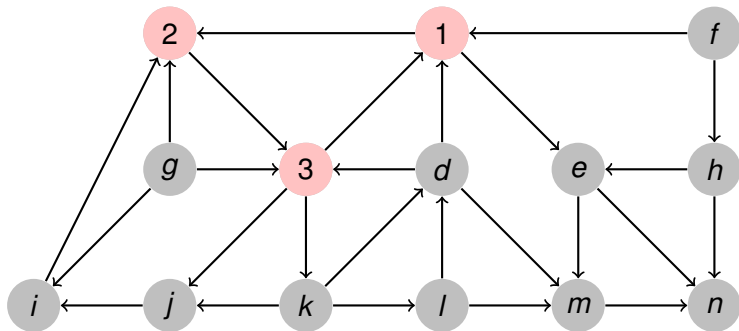
Ejemplo, DFS en pre-orden



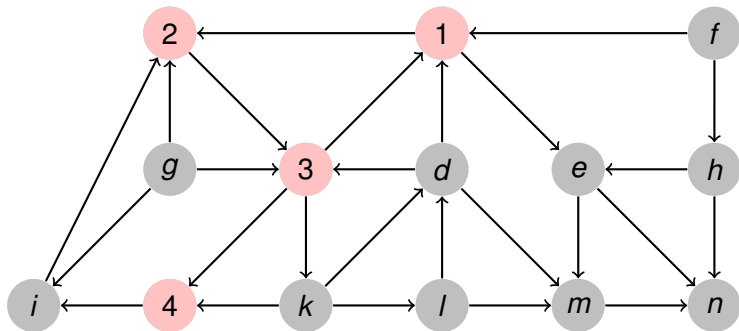
Ejemplo, DFS en pre-orden



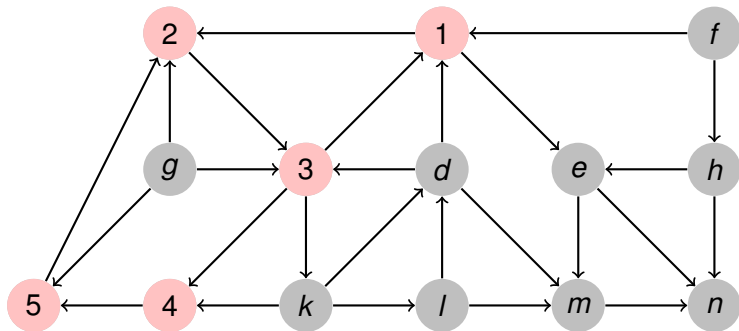
Ejemplo, DFS en pre-orden



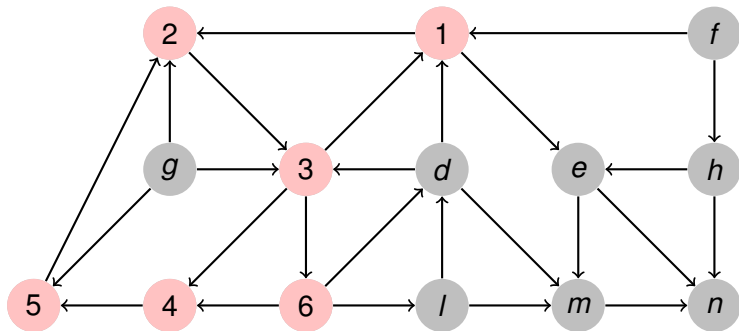
Ejemplo, DFS en pre-orden



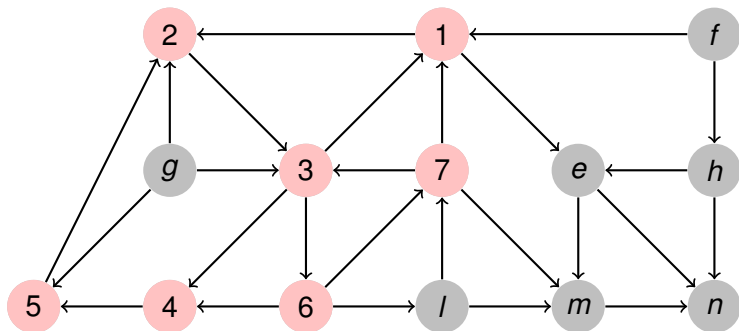
Ejemplo, DFS en pre-orden



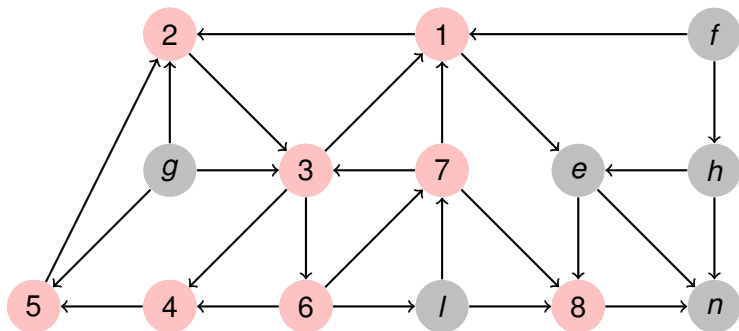
Ejemplo, DFS en pre-orden



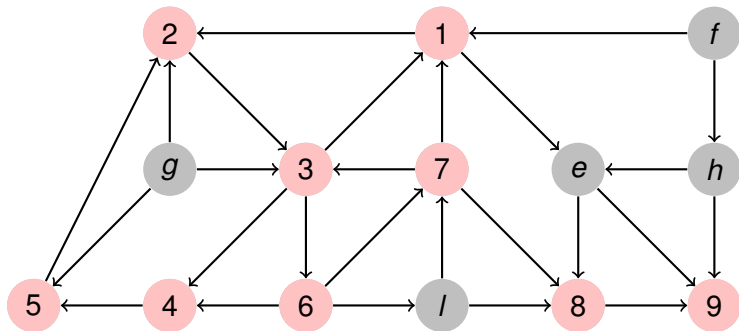
Ejemplo, DFS en pre-orden



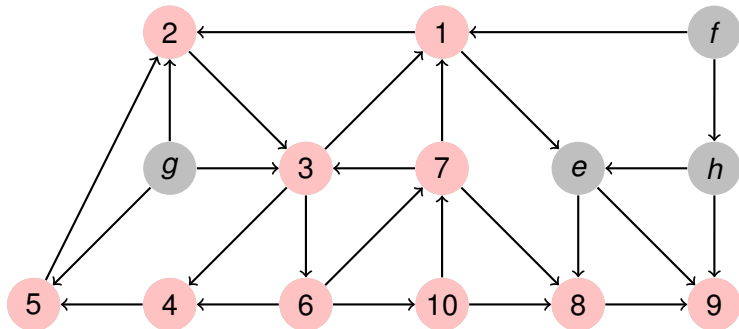
Ejemplo, DFS en pre-orden



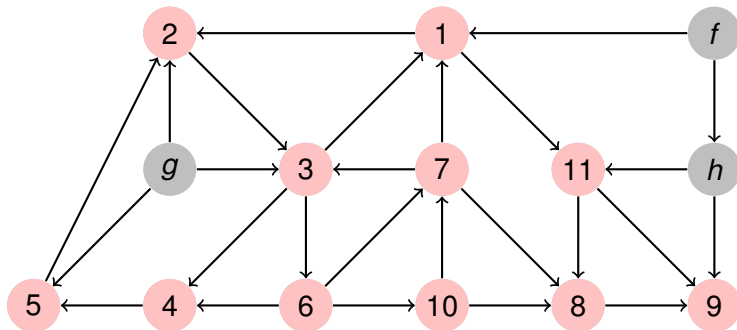
Ejemplo, DFS en pre-orden



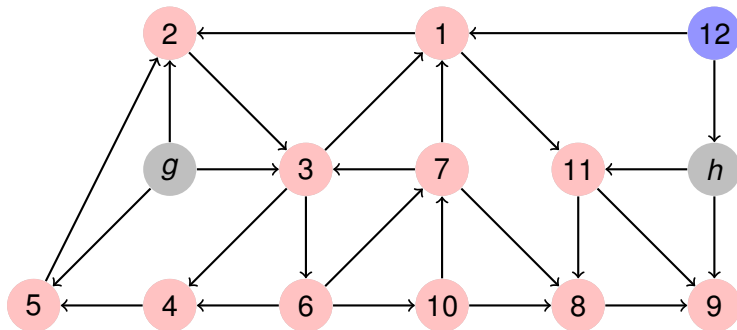
Ejemplo, DFS en pre-orden



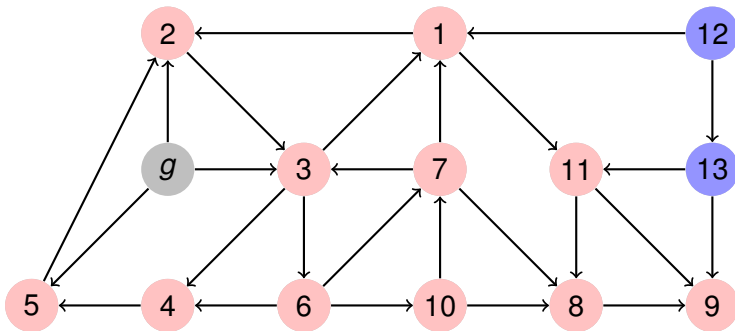
Ejemplo, DFS en pre-orden



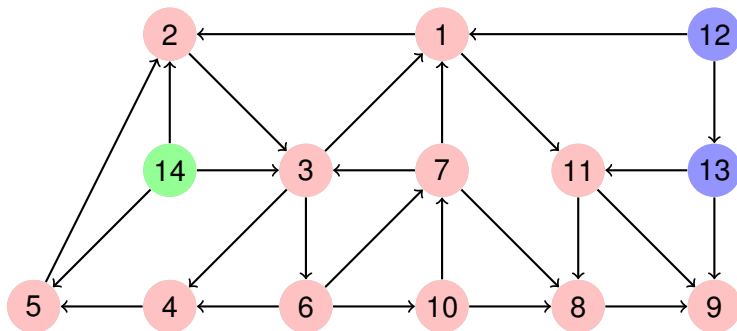
Ejemplo, DFS en pre-orden



Ejemplo, DFS en pre-order



Ejemplo, DFS en pre-orden



Algoritmos

Marcas

Como ahora puede haber ciclos, es necesario poder averiguar si un vértice ya fue visitado.

```
proc init(out mark: tmark)
    mark.cont:= 0
    for  $v \in V$  do mark.ord[v]:= 0 od
end

fun visited(mark: tmark, v: V) ret b: bool
    b:= (mark.ord[v]  $\neq$  0)
end
```


Algoritmo DFS

```
fun dfs( $G=(V, \text{neighbours})$ ) ret mark: tmark  
    init(mark)  
    for  $v \in V$  do  
        if  $\neg \text{visited}(\text{mark}, v)$  then dfsearch( $G, \text{mark}, v$ ) fi  
    od  
end  
  
proc dfsearch(in  $G$ , in/out mark: tmark, in  $v: V$ )  
    visit(mark,  $v$ )  
    for  $w \in \text{neighbours}(v)$  do  
        if  $\neg \text{visited}(\text{mark}, w)$  then dfsearch( $G, \text{mark}, w$ ) fi  
    od  
end
```

DFS iterativo

Introducimos una pila para evitar recursión

```
proc dfsearch(in G, in/out mark: tmark, in v: V)
  var p: stack of V
  empty(p)
  visit(mark,v)
  push(v,p)
  while  $\neg$ is_empty(p) do
    if existe  $w \in \text{neighbours}(\text{top}(p))$  tal que  $\neg \text{visited}(\text{mark}, w)$  then
      visit(mark,w)
      push(w,p)
    else pop(p)
    fi
  od
end
```

Clase de hoy

1 Repaso

2 Recorrida de grafos

- Generalidades
- Árboles binarios
- Árboles finitarios
- Grafos arbitrarios, DFS
- **Grafos arbitrarios, BFS**

BFS

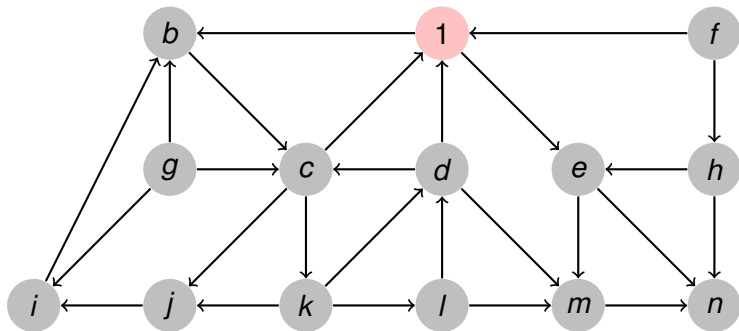
Si cambiamos la pila por una cola obtenemos BFS

```
proc bfsearch(in G, in/out mark: tmark, in v: V)
  var q: queue of V
  empty(q)
  visit(mark,v)
  enqueue(q,v)
  while  $\neg$ is_empty(q) do
    if existe  $w \in \text{neighbours}(\text{first}(q))$  tal que  $\neg \text{visited}(\text{mark}, w)$  then
      visit(mark,w)
      enqueue(q,w)
    else dequeue(q)
    fi
  od
end
```

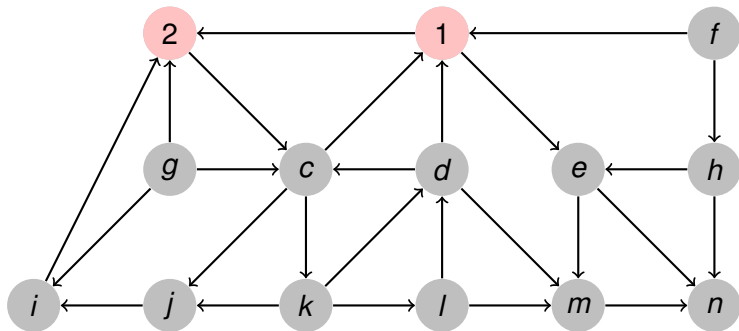
BFS, procedimiento principal

```
fun bfs( $G=(V, \text{neighbours})$ ) ret mark: tmark  
    init(mark)  
    for  $v \in V$  do  
        if  $\neg \text{visited}(\text{mark}, v)$  then bfsearch( $G, \text{mark}, v$ ) fi  
    od  
end
```

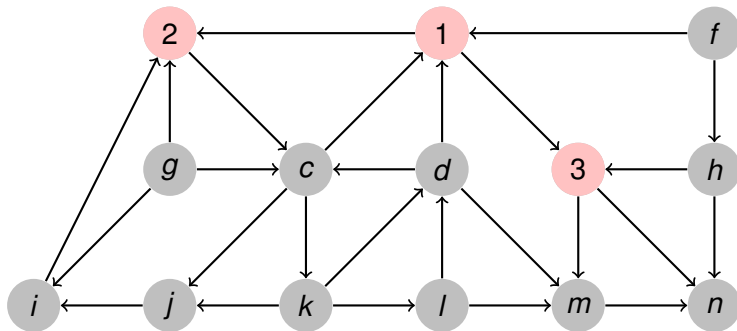
Ejemplo, BFS



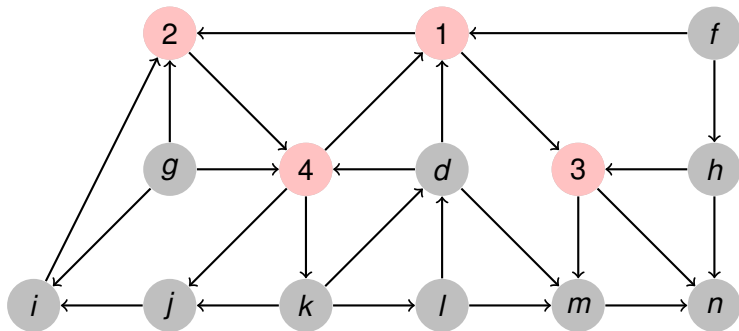
Ejemplo, BFS



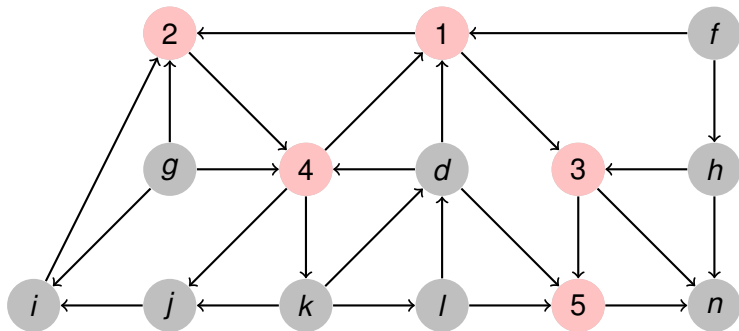
Ejemplo, BFS



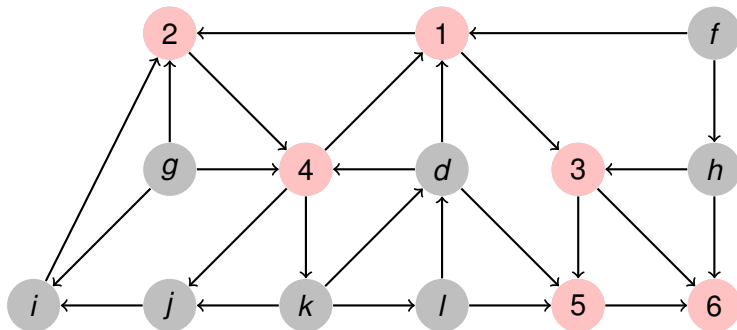
Ejemplo, BFS



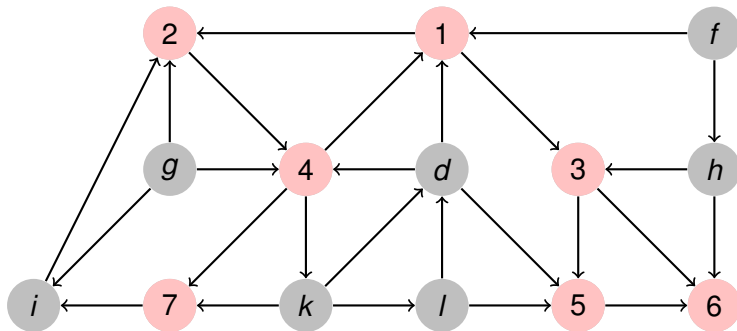
Ejemplo, BFS



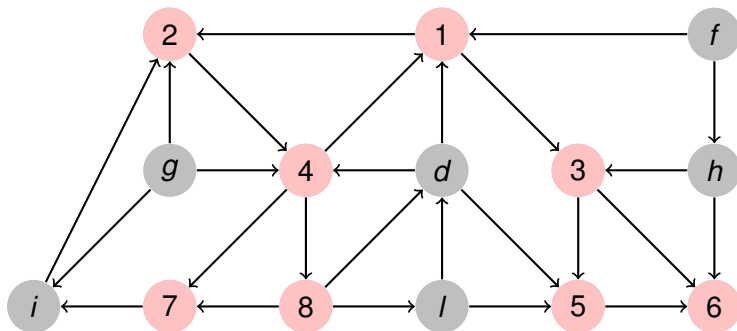
Ejemplo, BFS



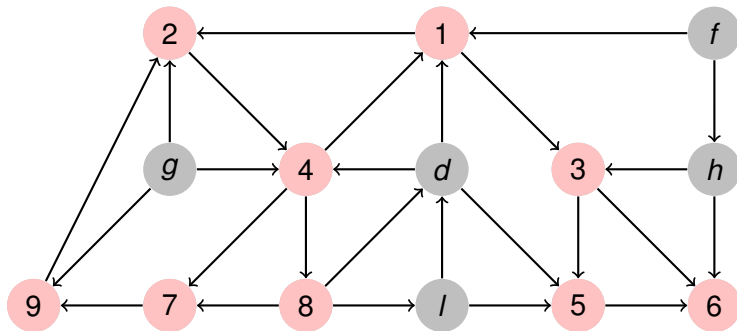
Ejemplo, BFS



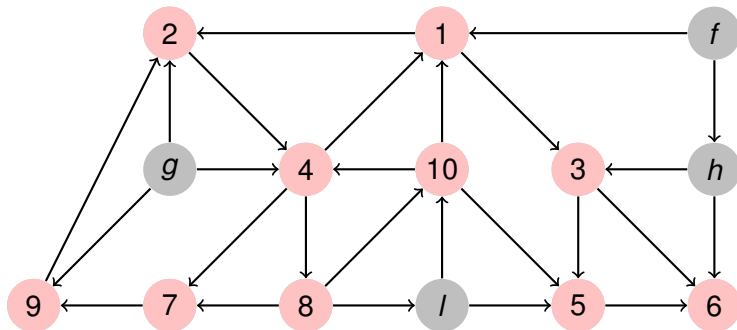
Ejemplo, BFS



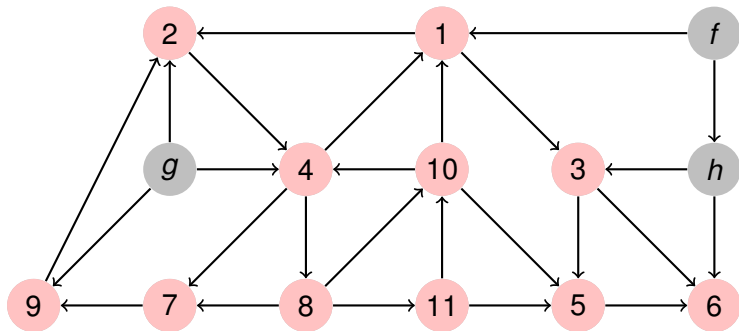
Ejemplo, BFS



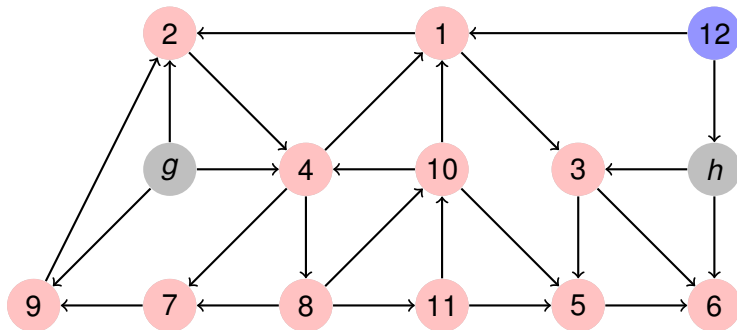
Ejemplo, BFS



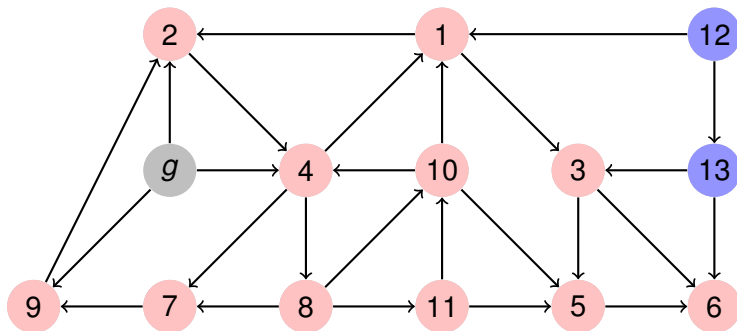
Ejemplo, BFS



Ejemplo, BFS



Ejemplo, BFS



Ejemplo, BFS

